

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Aplicación para la gestión del hogar en piso de estudiantes

Autor

Sergio Muñoz Gómez

Directores

María Luisa Rodríguez Almendros



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, Enero de 2025

Aplicación para la gestión del hogar en piso de estudiantes

Sergio Muñoz Gómez

Palabras clave: gestión, hogar, software, iteraciones

Resumen

Poner aquí el resumen.

Project Title: Project Subtitle

Sergio Muñoz Gómez

Keywords: management, home, software, iterations

Abstract

Write here the abstract in English.

Yo, **Sergio Muñoz Gómez**, alumno de la titulación INGENIERÍA INFORMÁTICA de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 76062609E, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Sergio Muñoz Gómez

Granada a 20 de Enero de 2025 .

D. **María Luisa Rodríguez Almendros**, Profesora del Área de Ingeniería del Software del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Aplicación para la gestión del hogar en piso de estudiantes***, ha sido realizado bajo su supervisión por **Sergio Muñoz Gómez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 20 de Enero de 2025 .

Los directores:

María Luisa Rodríguez Almendros

Agradecimientos

En primer lugar quiero dar mis agradecimientos a mi familia, gracias por apoyarme siempre en todas las decisiones que he tomado durante mi vida. Gracias por confiar en mí y por estar siempre a mi lado en los momentos más difíciles. Gracias por hacerme sentirme orgulloso de todos los logros que he conseguido y por darme ánimos en los momentos en los que las cosas no salían como esperaba. Gracias por hacerme ver que todo es posible si se lucha por ello. Gracias por enorgullecerme de ser parte de esta familia.

En segundo lugar quiero agradecer a los amigos que me ha dado esta carrera, en concreto a Julia, Juan y Salma. Gracias por haberme enseñado lo que es la amistad incondiconal, gracias por todo el aprendizaje que me habeis dado y por haberme ayudado en los momentos en los que más lo he necesitado. Gracias por haberme hecho reir y por haberme hecho ver que la vida es mucho más bonita si se comparte con amigos. Gracias por haberme hecho sentirme orgulloso de ser quien soy. Estoy muy orgullo de ser vuestro amigo.

No quiero olvidarme de mis compañeras de piso. Carla, Elena, Sandra, gracias por haberme hecho sentir como en casa desde el primer día. Gracias por haberlo hecho todo tan fácil y por haberme ayudado en todo lo que he necesitado. Lo que empezó siendo una convivencia ha pasado a ser una familia.

Por último quiero agradecer a mi tutora, María Luisa Rodríguez Almendros, por haberme guiado y ayudado en la realización de este trabajo. Este proyecto no hubiera sido posible sin su ayuda y sin su apoyo.

Gracias a todos por haberme ayudado a llegar hasta aquí. Este trabajo es también vuestro.

Índice general

1. Introducción **19**

1.1. Motivación 19

1.2. Objetivos del proyecto 20

1.3. Estructura de la memoria 21

2. Planificación y presupuesto **23**

2.1. Fases 23

2.1.1. Fase Inicial 23

2.1.2. Fase de Investigación 23

2.1.3. Fase de Analisis 24

2.1.4. Fase de Diseño 24

2.1.5. Fase de Desarrollo 24

2.1.6. Documentación 24

2.1.7. Diagrama de Gantt 24

2.2. Presupuesto 26

2.2.1. Recursos Hardware 26

2.2.2. Recursos Software 26

2.2.3. Licencias 27

2.2.4. Costes indirectos 27

2.2.5. Presupuesto Total 27

3. Estado del arte **29**

3.1. Estudios sobre la gestión del hogar y las finanzas en viviendas compartidas 29

3.2. Aplicaciones similares y comparativas 32

3.2.1. Tricount 32

3.2.2. Todoist 36

3.2.3. MeToca 41

3.2.4. Cozi Family Organizer 42

3.2.5. OurHome 47

3.2.6. Comparativa 48

3.3. Legislación actual de protección de datos 50

3.4. Estudio de usabilidad y accesibilidad 52

3.4.1. Estudio de usabilidad 52

3.4.2. Estudio de accesibilidad 53

4. Análisis de tecnologías **56**

4.1. Análisis de las tecnologías Frontend 56

4.1.1. Desarrollo nativo 56

4.1.2. Desarrollo Híbrido 57

4.1.3. Tabla comparativa de tecnologías front-end 59

4.2. Analisis de las tecnologías Backend 61

4.2.1. Firebase 61

4.2.2. Node.js con Express 61

4.2.3. Django 62

4.2.4. Ruby on Rails 62

4.2.5. Tabla comparativa de tecnologías back-end 63

5. Análisis de metodologías de desarrollo 64

5.1. Metodologías de desarrollo de software 64

5.1.1. Modelo en cascada (Waterfall) 64

5.1.2. Modelo en espiral 65

5.1.3. Metodología iterativa 65

5.1.4. Desarrollo iterativo e Incremental 65

5.1.5. Metodología Ágil 66

5.1.6. DevOps 66

5.2. Comparativa de metodologías 66

6. Propuesta 68

6.1. Explicación de la propuesta 68

6.2. Análisis de la aplicación 68

6.2.1. Especificación de requisitos 68

6.2.2. Modelos de casos de uso 86

6.2.3. Modelo conceptual 94

6.3. Diseño 96

6.3.1. Diseño de la arquitectura 96

6.3.2. Diseño de la interfaz gráfica 97

7. Desarrollo e implementación de la aplicación 104

7.1. Elección de tecnologías 104

7.2. Iteracciones de desarrollo 105

7.2.1. Iteración 1 105

7.2.2. Iteración 2 121

7.2.3. Iteración 3 142

8. Validación de la aplicación por usuarios reales 155

A. Descripción de casos de uso 156

A.1. Casos de uso de la gestión de sesión 156

A.2. Casos de uso de la gestión de hogar 162

A.3. Casos de uso de la gestión de tareas 167

A.4. Casos de uso de la gestión de gastos 174

A.5. Casos de uso de la gestión compra 180

Referencias 186

Índice de figuras

3.1. Número de personas que residen en la vivienda. 29
3.2. Pantalla Inicio. 33
3.3. Lista de grupos. 33
3.4. Añadir un nuevo grupo. 34
3.5. Pantalla del perfil. 34
3.6. Detalles de un nuevo grupo. 35
3.7. Enviar invitación a los miembros del grupo. 35
3.8. Añadir un gasto. 35
3.9. Resumen de los gastos y deudas. 35
3.10. Pantalla de registro. 38
3.11. Elegir el ámbito de la aplicación. 38
3.12. Pantalla principal. 39
3.13. Añadir una tarea. 39
3.14. Pantalla con la tarea añadida. 40
3.15. Calendario de tareas. 40
3.16. Registro. 44
3.17. Crear una familia. 44
3.18. Añadir un evento. 45

3.19. Calendario. 45
3.20. Listas de la compra. 46
3.21. Recetas. 46

4.1. Logo de Kotlin 57
4.2. Logo de Swift 57
4.3. Logo de Ionic 58
4.4. Logo de Flutter 58
4.5. Logo de React Native 59
4.6. Logo de Xamarin 59
4.7. Logo de Firebase 61
4.8. Logo de Node.js 62
4.9. Logo de Django 62
4.10. Logo de Ruby on Rails 63

6.1. Diagrama de paquetes de casos de uso 87
6.2. Diagrama de casos de uso de la gestión de sesión 87
6.3. Diagrama de casos de uso de la gestion de hogar 88
6.4. Diagrama de casos de uso de la gestion de tareas 89
6.5. Diagrama de casos de uso de la gestion de gastos 90
6.6. Diagrama de casos de uso de la gestion de compra 91
6.7. Modelo conceptual 95
6.8. Diagrama de la arquitectura del sistema 96
6.9. Diseño de la pantalla de inicio de sesión. 99
6.10. Diseño de la pantalla de cierre de sesión y datos de usuario. . 99
6.11. Diseño de la pantalla para unirte a un hogar o crear uno nuevo. 100

6.12. Diseño de la pantalla para unirte a un hogar. 100
6.13. Diseño de la pantalla principal de tareas. 101
6.14. Diseño de la pantalla del calendario. 101
6.15. Diseño de la pantalla principal de gastos. 102
6.16. Diseño de la pantalla de deudas. 102
6.17. Diseño de la pantalla de la lista de la compra. 103
6.18. Diseño de la pantalla de productos. 103

7.1. Diseño de la pantalla de bienvenida. 107
7.2. Diseño de la pantalla de registro. 107
7.3. Diseño de la pantalla de editar usuario. 108
7.4. Diseño de la pantalla de editar hogar. 108
7.5. Interfaz gráfica pantalla bienvenida. 109
7.6. Interfaz gráfica pantalla de registro. 111
7.7. Interfaz gráfica pantalla de inicio de sesión. 112
7.8. Interfaz gráfica pantalla de ver datos de hogar y usuario. . . . 114
7.9. Interfaz gráfica pantalla de editar datos de usuario. 115
7.10. Interfaz gráfica pantalla de editar hogar. 116
7.11. Interfaz gráfica pantalla para participar en un hogar. 117
7.12. Interfaz gráfica pantalla para crear un hogar. 118
7.13. Interfaz gráfica pantalla de unirse a un hogar. 119
7.14. Diseño de la pantalla de crear tareas. 123
7.15. Diseño de la pantalla de editar tarea. 123
7.16. Interfaz gráfica pantalla de tareas. 126
7.17. Interfaz gráfica pantalla de crear tarea puntual. 127
7.18. Interfaz gráfica pantalla de crear tarea semanal. 128

7.19. Interfaz gráfica pantalla de ver datos de una tarea. 129

7.20. Interfaz gráfica pantalla para editar o eliminar una tarea. . . 130

7.21. Interfaz gráfica pantalla de editar tarea. 131

7.22. Interfaz gráfica pantalla de calendario. 133

7.23. Interfaz gráfica pantalla de lista de la compra. 134

7.24. Interfaz gráfica pantalla de crear lista de la compra. 135

7.25. Interfaz gráfica pantalla para eliminar lista de la compra. . . 136

7.26. Interfaz gráfica pantalla de productos de la lista de la compra. 137

7.27. Interfaz gráfica pantalla de añadir producto a la lista de la compra. 138

7.28. Diseño de la pantalla para saldar deudas. 143

7.29. Interfaz gráfica pantalla de gastos. 145

7.30. Interfaz gráfica pantalla de crear gasto. 146

7.31. Interfaz gráfica pantalla de deudas. 147

7.32. Interfaz gráfica pantalla de ver detalles de un gasto. 148

7.33. Interfaz gráfica pantalla para saldar deuda. 149

7.34. Interfaz gráfica pantalla principal de la aplicación. 151

7.35. Interfaz gráfica barra de navegación. 152

7.36. Diagrama final de clases. 153

8.1. Puntuaciones individuales SUS 155

Índice de cuadros

2.1. Tabla de tareas y horas 25

2.2. Presupuesto detallado del proyecto 28

3.1. Comparativa de aplicaciones para la gestión de tareas y gastos. 48

4.1. Comparativa de tecnologías para desarrollo de aplicaciones móviles. 60

4.2. Comparativa de tecnologías para el desarrollo back-end. . . . 63

5.1. Comparativa de metodologías de desarrollo de software. . . . 66

6.1. RF_01 70

6.2. RF_02 70

6.3. RF_03 71

6.4. RF_04 71

6.5. RF_05 71

6.6. RF_06 72

6.7. RF_07 73

6.8. RF_08 73

6.9. RF_09 74

6.10. RF_10 74

6.11. RF_11 75
6.12. RF_12 76
6.13. RF_13 76
6.14. RF_14 77
6.15. RF_15 77
6.16. RF_16 78
6.17. RF_17 78
6.18. RF_18 78
6.19. RF_19 79
6.20. RF_20 79
6.21. RF_21 80
6.22. RF_22 80
6.23. RF_23 81
6.24. RF_24 81
6.25. RF_25 82
6.26. RF_26 82
6.27. RF_27 83
6.28. RF_28 83
6.29. RF_29 84
6.30. RF_30 84
6.31. RNF_01 85
6.32. RNF_02 85
6.33. RNF_03 85
6.34. RNF_04 85
6.35. RNF_05 86

6.36. Caso de uso - Registro 92
6.37. Caso de uso - Iniciar Sesión 93
6.38. Caso de uso - Cerrar Sesión 94

7.1. RF_31 122

A.1. Caso de uso - Registro 157
A.2. Caso de uso - Iniciar Sesión 158
A.3. Caso de uso - Cerrar Sesión 159
A.4. Caso de uso - Ver Usuario 160
A.5. Caso de uso - Editar Usuario 161
A.6. Caso de uso - Crear Hogar 162
A.7. Caso de uso - Unir Miembro a Hogar 163
A.8. Caso de uso - Abandonar Hogar 164
A.9. Caso de uso - Modificar Hogar 165
A.10. Caso de uso - Ver Hogar 166
A.11. Caso de uso - Crear Tarea 167
A.12. Caso de uso - Consultar Planificación Semanal 168
A.13. Caso de uso - Modificar Tarea 169
A.14. Caso de uso - Marcar Tarea Realizada 170
A.15. Caso de uso - Eliminar Tarea 171
A.16. Caso de uso - Ver Tarea 172
A.17. Caso de uso - Ver Calendario 173
A.18. Caso de uso - Crear Gasto 174
A.19. Caso de uso - Modificar Gasto 175
A.20. Caso de uso - Eliminar Gasto 176

A.21.Caso de uso - Saldar Deuda 177
A.22.Caso de uso - Ver Deudas 178
A.23.Caso de uso - Ver Gasto 179
A.24.Caso de uso - Crear Lista 180
A.25.Caso de uso - Añadir Producto 181
A.26.Caso de uso - Eliminar Producto 182
A.27.Caso de uso - Eliminar Lista 183
A.28.Caso de uso - Ver Producto 184
A.29.Caso de uso - Editar Producto 185

Capítulo 1

Introducción

1.1. Motivación

La mayoría de estudiantes enfrentan dificultades para gestionar adecuadamente su hogar debido a la falta de tiempo, ya que gran parte de su jornada está dedicada a sus estudios. Esta situación convierte la organización y la realización de tareas domésticas en un problema, pues estas actividades requieren una planificación cuidadosa y tiempo para llevarlas a cabo. Por otro lado, la gestión de los gastos compartidos puede resultar especialmente compleja en un entorno de estudiantes, ya que es común que cada miembro del hogar se encargue de diferentes pagos. Esto puede generar confusiones y equivocaciones, respectoa a quién ha pagado ciertos gastos y qué cantidades están pedientes, lo que agrava aún más la dificultad de una correcta administación del hogar.

Después de 5 años viviendo de manera independiente, he experimentado de primera mano las dificultades de gestionar un hogar compartido con otros estudiantes o trabajadores. En numerosas ocasiones el tiempo libre del que disponíamos no se ha destinado a la organización del hogar. Aunque las tareas se asignaban semanalmente, no siempre se cumplían, ya sea por falta de tiempo y o por descuidos. Además, la administación de los gastos ha sido, en muchas ocasiones, caótica, ya que no siempre se llevaba un control adecuado sobre quién ha pagado qué, y algunas personas solían olvidar devolver el dinero a tiempo.

En la actualidad, aproximadamente el **87 % de los jovenes emancipados en España viven comparten vivienda**, segun un estudio del portal HuffPost. (Huffington Post, 2025). Además, el 31 % de quienes alquilan una habitación en un piso compartido cohabitan con otras tres personas. (FC Research, 2024). Por tanto, la necesidad de una aplicación que facilite la organización y gestión del hogar en pisos de estudiantes es evidente.

Actualmente no existe ninguna aplicación que ofrezca una solución integral tanto para la gestión de las tareas domesticas como para la gestión de los gastos. Las aplicaciones disponibles abordan estos aspectos por separado. Además la mayoría de ellas están orientadas a hogares familiares donde la distribución de las tareas no se gestiona de la misma forma que en los pisos de estudiantes. Por eso es fundamental desarrollar una aplicación que integre todas estas funcionalidades, dirigida específicamente a estudiantes o trabajadores que compartan vivienda.

La aplicación que quiero desarrollar estará diseñada para ayudar a estudiantes y trabajadores que compartan vivienda, proporcionando una solución integrada que facilite tanto la organización de las tareas domésticas como la administración de gastos compartidos. En un entorno de convivencia, la asignación de tareas suele complicarse debido a los horarios irregulares y las diferentes cargas de estudio o trabajo de cada persona. Esta aplicación permitirá a asignar tareas de manera equitativa y flexible, estableciendo recordatorios y notificando a cada miembro sobre las responsabilidades asignadas para mejorar el cumplimiento y la colaboración en el hogar.

Por otra parte, la aplicación incluirá un sistema de gestión de gastos compartidos, donde se registrarán todos los pagos realizados y las cantidades que se deben por cada persona. Esto reducirá las confusiones y facilitará un control transparente y detallado de los gastos del hogar. En lugar de depender de otras aplicaciones o de anotaciones manuales, esta herramienta unificará ambos aspectos (tareas y gastos) en un solo lugar, simplificando la administración y ayudando a la organización en hogares compartidos.

1.2. Objetivos del proyecto

El objetivo principal del proyecto es desarrollar una aplicación móvil para facilitar a los estudiantes la organización y gestión eficiente de su hogar, ofreciendo soluciones que aborden las dificultades que puedan surgir en la planificación y distribución de responsabilidades. Para poder alcanzar este objetivo, podemos diferenciar una serie de objetivos específicos:

- **Objetivo 1: Análisis de la gestión de hogares compartidos y gestión de gastos.**
  - **Descripción:** Búsqueda de información sobre los problemas más comunes que surgen en la gestión de hogares compartidos y en la gestión de gastos.
- **Objetivo 2: Estudio de aplicaciones similares**

    - **Descripción:** Estudio del mercado actual de aplicaciones similares con el fin de identificar funcionalidades innovadoras que puedan ser incorporadas en nuestra aplicación.

- **Objetivo 3: Análisis de las tecnologías a utilizar**
    - **Descripción:** Investigar cuáles son las tecnologías más adecuadas para la implementación del proyecto.
- **Objetivo 4: Desarrollo e implementación de la aplicación**
    - **Descripción:** Analizar, desarrollar y realizar pruebas en la aplicación con el fin de obtener los mejores resultados.
- **Objetivo 5: Análisis de usabilidad y accesibilidad**
    - **Descripción:** Realizar un análisis de usabilidad y accesibilidad para identificar posibles mejoras en la interfaz de usuario.
- **Objetivo 6: Validación de la aplicación por usuarios reales**
    - **Descripción:** Evaluar la aplicación en un entorno real para verificar si satisface las expectativas del usuario.

1.3. Estructura de la memoria

A continuación, se ofrece una visión general de la estructura y organización de la memoria. El contenido se ha dividido en los siguientes apartados:

1. **Introducción.** En este apartado se presenta la motivación del proyecto y los objetivos que se pretenden alcanzar.
2. **Planificación y presupuesto.** Descripción de la planificación del proyecto, que incluye las diferentes fases que se han llevado a cabo durante su desarrollo, así como el presupuesto estimado.
3. **Estado del arte.** Estudio sobre los principales temas relacionados con este proyecto, cómo analisis de aplicaciones similares, temas legales de protección de datos, etc.
4. **Análisis de tecnologías.** Análisis de las posibles tecnologías que se pueden usar en la actualidad y eleeciión de las más adecuadas para el desarrollo de la aplicación.
5. **Análisis de metodologías de desarrollo** Análisis de las posibles metodologías que existen y elección de las más adecuadas para el desarrollo de la aplicación.

6. **Propuesta.** Se presenta la aplicación propuesta, detallando su análisis y diseño. Se detallan los requisitos funcionales y no funcionales, asi como los diagramas de casos de uso y de clases. Se incluyen también los mockups de la aplicación.

7. **Desarrollo e implementación de la aplicación.** Se detalla el proceso de desarrollo de la aplicación. Se realizará siguiendo una metodología iterativa y se detallarán sus iteraciones.

8. **Validación de la aplicación por usuarios reales.** Evaluación de la aplicación en un entorno real para verificar si satisface las expectativas del usuario.

9. **Conclusiones y trabajo futuro.** Se presentan las conclusiones obtenidas a lo largo del proyecto y se proponen posibles líneas de trabajo futuro.

10. **Bibliografía.** Referencias bibliográficas utilizadas en la elaboración de la memoria.

Capítulo 2

Planificación y presupuesto

2.1. Fases

La planificación del proyecto se estructurará en fases con el objetivo de garantizar un desarrollo organizado y eficiente, asegurando que cada etapa esté enfocada en el cumplimiento de objetivos específicos.

2.1.1. Fase Inicial

En esta fase se recopilará y definirá toda la información relacionada con el Capitulo 1, incluyendo la motivación para la realización del proyecto y la determinación de los objetivos que se pretenden alcanzar a lo largo del mismo.

2.1.2. Fase de Investigación

Esta fase estará dedicada a la recopilación de información relevante para el desarrollo del proyecto. Se analizarán aplicaciones similares para demostrar la necesidad de una aplicación integral que cubra todas las funciones propuestas. Además se investigará aspectos como la gestión del tiempo en hogares compartidos y la administración de gastos entre personas que comparten un mismo domicilio.

Para garantizar que la aplicación cumpla con los requisitos legales, también se revisará la normativa vigente de protección de datos. Asímismo, se realizará un estudio de usabilidad y accesibilidad para asegurar que el diseño sea inclusivo y fácil de usar para todos los usuarios.

2.1.3. Fase de Analisis

En esta fase se llevará a cabo un análisis de las diferentes tecnologías que se pueden utilizar para el desarrollo de la aplicación, tanto para el frontend como para el backend, con el objetivo de selecionar las más adecuadas para el proyecto. Además, se definirán los requisitos que debe cumplir la aplicación para satisfacer las necesidades del usuario objetivo. Como parte de esta fase, se elaborarán los casos de uso y los diagramas de secuencia del sistema.

2.1.4. Fase de Diseño

Esta fase estará enfocada en la creación de los prototipos de la aplicación. Se desarrollará un diagrama de arquitectura del sistema y bocetos para la interfaz gráfica. Además, se definirán los colores y las tipografías que se emplearán en la aplicación, garantizando un diseño responsive que permita su visualización en cualquier dispositivo.

2.1.5. Fase de Desarrollo

En esta fase se procederá a la implementación de la aplicación, siguiendo las tecnologías, metodologías y diseños definidos en las fases previas de análisis y diseño. Esta fase se dividirá en cuatro iteraciones, ya que se empleará una metodología ágil para el desarrollo del proyecto. En cada iteración se determinarán las funcionalidades que se implementarán y se realizarán pruebas para garantizar que la aplicación cumple con los requisitos definidos en la fase de análisis.

Las pruebas serán realizadas por usuarios reales, concretamente jóvenes que comparten piso, para verificar que la aplicación satisface sus necesidades y resulta fácil de utilizar.

2.1.6. Documentación

Aunque no se considera una fase independiente, la documentación será un proceso continuo a lo largo de todo el proyecto. Se registrarán de manera detallada todas las actividades y resultados obtenidos en cada fase.

2.1.7. Diagrama de Gantt

En esta tabla se específica la duración en horas de cada una de las tareas representadas en el diagrama de Gantt. Este diagrama representa la planificación temporal de las actividades del proyecto, donde cada celda corresponde a una semana del mes. La tabla presenta la cantidad de horas

dedicadas a cada tarea, mientras que el diagrama de Gantt muestra el periodo en el que se llevaron a cabo.

Tarea	**Horas**
Motivación	1
Objetivos del proyecto	2
Estructura de la memoria	2
Planificación	3
Presupuesto	4
Estudios sobre la gestión del hogar y las finanzas en viviendas compartidas	20
Aplicaciones similares y comparativas	25
Legislación actual de protección de datos	7
Estudio de usabilidad y accesibilidad	12
Análisis de las tecnologías Frontend	20
Analisis de las tecnologías Backend	15
Análisis de las metodologías de desarrollo	12
Explicación de la propuesta	2
Especificación de los requisitos	
Modelos de casos de uso	
Modelo conceptual	
Diseño de la arquitectura	
Diseño de la interfaz gráfica	
Elección de tecnologias	2
Iteración 1	30
Iteración 2	45
Iteración 3	45

Cuadro 2.1: Tabla de tareas y horas

2.2. Presupuesto

En este apartado se detallarán los costes asociados al desarrollo del proyecto. Se incluirán los costes de hardware, software, licencias, servicios en la nube y otros gastos necesarios para la realización del proyecto. Realizar un presupuesto en este proyecto es clave para planificar y gestionar los recursos de forma eficiente. Permite asegurarse de que las fases de desarrollo se puedan llevar a cabo con los medios disponibles. Además, ayuda a justificar las decisiones tomadas y evitar problemas por faltas de recursos asegurando que el proyecto sea viable y esté bien estructurado.

2.2.1. Recursos Hardware

Los componentes hardware necesarios para el desarrollo del proyecto son un ordenador portátil y un teléfono móvil para realizar pruebas con la aplicación.

- **Ordenador:** TUF Gaming FX505DT_FX505DT
- **Teléfono Móvil:** Samsung Galaxy A51

2.2.2. Recursos Software

Las herramientas software que se utilizarán en el proyecto son las siguientes:

- **Sistema Operativo:** Windows 11
- **IDE:** Visual Studio Code
- **Control de Versiones:** Git
- **Lenguajes de Programación:** Flutter, Dart
- **Diseño de diagramas UML:** Visual Paradigm
- **Diseño de Interfaces:** Figma
- **Sistema de Gestión de Base de Datos:** Firebase
- **Sistema de composición de texto:** LaTeX

2.2.2.1. Recursos Humanos

En este apartado se detallarán los gastos por la contratación de personal. Este proyecto será desarrollado por una única persona, un programador junior, que según (Talent.com, 2025) tiene un coste de 21.000€al año, como la duración de este proyecto es de 6 meses, el coste total será de 10.500€.

2.2.3. Licencias

En este apartado se detallan las licencias software necesarias para llevar a cabo el desarrollo del proyecto. Aunque en este caso no supone ningún coste, ya que se utilizarán versiones gratuitas o herramientas sin coste, en un proyecto real sería imprescindible considerar el pago de las licencias. Esto garantizaría el acceso completo a todas las funcionalidades de las herramientas, además de cumplir con las normativas legales y éticas relaciondas con el uso del sotware. Las licencias de software que deberían considerarse en un proyecto real son las siguientes:

- **Windows 11:** Windows 11 Home License
- **Visual Paradigm:** Visual Paradigm Standard Edition
- **Figma:** Figma Professional

El resto de licencias necesarias para el desarrollo del proyecto son gratuitas, por lo que no se incluirán en el presupuesto.

2.2.4. Costes indirectos

En este apartado se engloba todos los gastos indirectos que incluyen los costes de electricidad, internet, espacio de trabajo, etc. Estos gastos indirectos suelen estimarse como un porcentaje adicional del coste directo total *(entre 10 % y 20 %)*. En este caso será un 15 % de los recursos humanos.

2.2.5. Presupuesto Total

El coste total del proyecto se obtiene sumando los costes de los recursos hardware, software, humanos, licencias y costes indirectos.

Concepto	Coste Mensual	Coste Total
Recursos Hardware	-	1.000 €
Ordenador portátil	-	800 €
Teléfono móvil para pruebas	-	200 €
Recursos Humanos	1.750€	10.500 €
Programador Junior (6 meses)	1.750€	10.500 €
Licencias	52,68€	316,12€
Windows 11 Home License	-	145 €
Visual Paradigm Standard Edition	12,23€	73,38€
Figma Professional	20,45€	122,7€
Costes Indirectos	262,5€	1.575€
Electricidad	50€	300€
Internet	30€	180€
Espacio de trabajo (estimado)	182,5€	1.095€
Coste Total	2.065,18€	**13.391,12€**

Cuadro 2.2: Presupuesto detallado del proyecto

Capítulo 3

Estado del arte

3.1. Estudios sobre la gestión del hogar y las finanzas en viviendas compartidas

Según el informe "Un problema como una casa" del consejo de la Juventud de España (CJE) el **87 % de los jovenes emancipados en España comparten vivienda** (Huffington Post, 2025). Esta tendencia se debe a la necesidad de reducir los gastos, ya que los altos costos de alquiler y la precariedad laboral dificultan que los jóvenes puedan afrontar los gastos mensuales de una vivienda en solitario. Como podemos ver en la grafica de la imagen 3.1 dentro de los jóvenes emancipados que comparten vivienda, **alrededor del 42,4 % la comparten con 3 o 4 personas** y el 37,9 % con dos personas más, siendo el restante personas que vivien con una persona más o con más de cuatro.

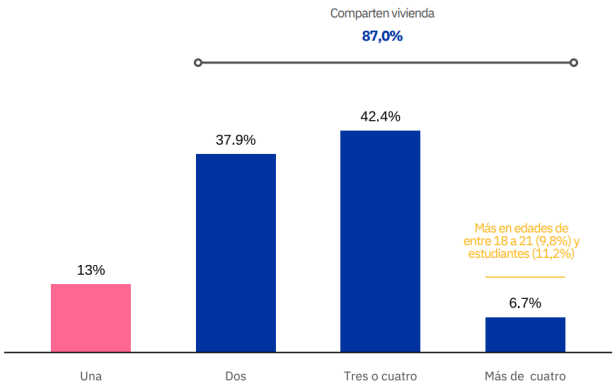


Figura 3.1: Número de personas que residen en la vivienda.

Para hacer frente a esta situación, muchos jóvenes recurren al apoyo económico familiar: alrededor del 30 % necesita ayuda de sus familias para cubrir los gastos mensuales. Esta dependencia económica refleja la insuficiencia de los ingresos juveniles para sostener una vida independiente.

La calidad de las viviendas compartidas también es motivo de preocupación. Casi la mitad de los jóvenes que viven de alquiler deben afrontar una fianza equivalente a dos meses de renta, y un 41,5 % paga más de 600 euros en este concepto. Además, el 50 % ha tenido problema con sus caseros, principalmente con la falta de reparación de desperfectos.

Un estudio del *American Time Use Survey*, (American Time Use Survey, 2022) analiza la distribución del tiempo que las personas dedican a la administración del hogar y cómo esta tarea se realiza en intervalos dispersos. Este estudio revela patrones en las actividades domésticas y ayuda a entender los desafío de coordinación y planificación en hogares donde las responsabilidades se dividen entre varias personas. El estudio muestra que las tareas del hogar, como la limpieza, la preparación de comidas, la organización y la gestión de finanzas, tienden a realizarse en intervalos de tiempo que varían significativamente entre los distintos miembros del hogar. Esto es problemático en hogares compartidos, como los estudiantes o trabajadores, donde la disponibilidad de cada persona depende de su carga académica o laboral. La falta de tiempos fijos y la irregularidad en la dedicación a las tareas puede generar problemas de coordinación, ya que cada miembro tiene su propio horario, lo que impide una rutina doméstica unificada.

Otro hallazgo importante de este estudio es que las tareas domésticas se realizan de manera inconsistente debido a limitaciones de tiempo y energía, especialmente entre aquellos con empleos de tiempo completo o estudiante con cargas académicas pesadas. Esta incosistencia es común en los hogares compartidos, donde la carga se divide entre varias personas sin un sistema claro de asignación de responsabilidades o seguimiento. Sin herramientas adecuadas para la organización, es posible que algunas tareas queden sin realizar o que recaigan siempre en las mismas personas, generando frustración y posibles conflictos.

El estudio también resalta que, a medida que los individuos se ven cada vez más presionados por sus estudios o trabajos, tienden a posponer las tareas del hogar, lo cual se agrava en entornos de convivencia compartida. Este aplazamiento puede llevar a una acumulación de tareas, que a su vez aumenta el nivel de estrés de los residentes y genera tensiones en el hogar.

Por último, los resultados también sugieren que la organización efectiva del hogar es crucial para el bienestar de sus miembros. En el contexto de hogares compartidos, el uso de herramientas de planificación o aplicaciones específicas podría mejorar significativamente la administración de tareas y reducir el estrés relacionado con el hogar.

El estudio basado en datos de HUD, (U.S. Department of Housing and Urban Development, 2022) examina los desafíos financieros que enfrentan los estudiantes en viviendas compartidas. Uno de los hallazgos principales de este estudio es que los contratos de arrendamiento de las viviendas a menudo requieren compromisos anuales. Los estudiantes deben seguir pagando el alquiler durante todo el año, incluso en períodos en que la vivienda está vacía, como los meses de verano. Este requisito genera una carga financiera adicional, ya que los estudiantes deben encontrar formas de cubrir los pagos durante esos meses.

También señala que la organización de los gastos compartidos es una tarea compleja para los estudiantes. Dado que los contratos incluyen puede incluir diferentes servicios, la variación en las facturas mensuales añade dificultad al reparto equitativo de costes. Muchos estudiantes carecen de herramientas y sistemas de organización, lo que provoca confusión y conflictos en cuanto a las cantidades que cada miembro debe pagar. Esto puede ser díficil de gestionar cuando uno o varios de los residentes enfrentan problemas financieros, ya que retrasos o impagos afectan a todos los miembros del hogar.

Por último, este estudio también examina cómo las dificultades para gestionar los gastos afectan no solo la situación financiera de los estudiantes, sino también su bienestar general y rendimiento académico. La preocupación por la estabilidad financiera y la tensión asociada con la administración de los costos compartidos afectan a los estudiantes a nivel psicológico y académico, ya que deben dedicar su tiempo y energía a resolver estos problemas, lo que limita su concentración en los estudios.

3.2. Aplicaciones similares y comparativas

3.2.1. Tricount

Tricount es una aplicación diseñada para facilitar la gestión y el reparto de gastos entre varias personas. Es útil en sistuaciones, donde un grupo de personas realiza gastos comunes, como en viajes o cenas. Es una aplicación bastante sencilla de usar, lo que la hace aún más popular entre los grupos de personas. Está disponible para Android e iOS y tiene también soporte web. (Tricount, 2025)

3.2.1.1. Funciones Principales

1. **Creación de un Tricount**
    a) Configuración Inicial. Al iniciar un nuevo "Tricount"(grupo) se le da un nombre, se elige un tipo de divisa y se añade a los participantes.
    b) Acceso Compartido. Puede ser compartido entre todos los participantes a través de un enlace, lo que permite que cada uno pueda añadir sus gastos.
2. **Añadir Gastos**
    a) Registro de Gastos. Cualquier participante puede añadir un gasto en el tricount, especificando la cantidad, la categoría, quien pagó y para quien es ese gasto.
    b) División Personalizada. Tricount permite dividir los gastos de forma equitatica o personalizada, es decir, si el gasto es para todos o solo para algunos.
    c) Comentarios. Se pueden añadir comentarios a cada pago.
3. **Cálculo Automático**
    a) Resumen de Balances. Se calcula automáticamente cúanto debe cada persona y a quien.
    b) Balances Individuales. Cada participante puede ver su valance individual y cuánto ha gastado, cuánto le deben y cuánto le debe a otros.
4. **Saldar Deudas**
    a) Recomendaciones de Pagos. Tricount recomienda la mejor manera de saldar las deudas dentro del grupo, optimizando el número de transsaciones necesarias.

b) Pagos Parciales o Totales. Los participantes pueden registrar pagos parciales o totales dentro de la aplicación para mantener un seguimiento claro de que ha sido pagado y lo que queda pendiente.

3.2.1.2. Modo de Uso

Una vez nos hayamos descargado la aplicación ya sea desde Google Play o App Store, lo primero que nos muestra es una pantalla de inicio como nos muestra la *Figura 3.2*.

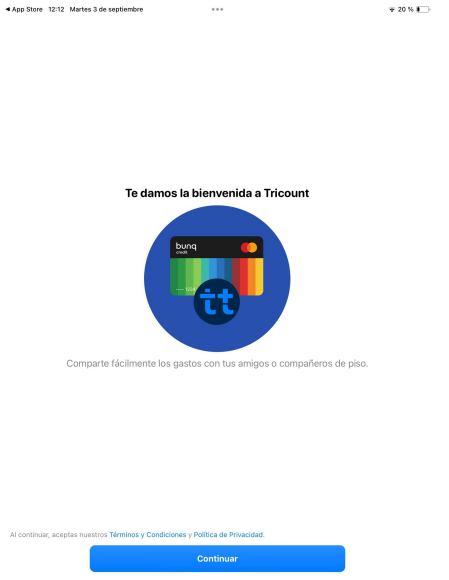


Figura 3.2: Pantalla Inicio.

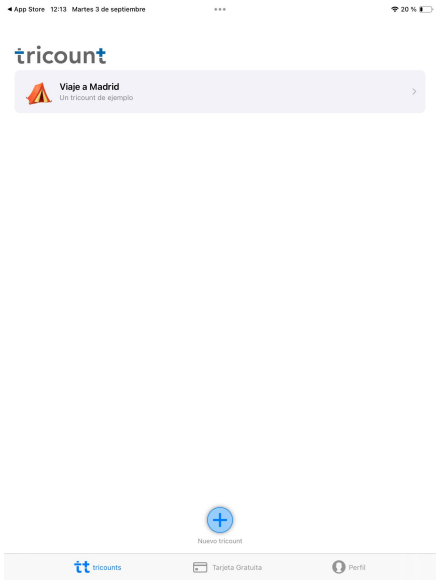


Figura 3.3: Lista de grupos.

Pulsado el botón de *Continuar* nos muestra una pantalla donde nos sale una lista de los grupos que hemos creado o en los que estamos participando. En este caso como es la primera vez que entramos solo nos muestra un grupo de ejemplo (*Figura 3.3*).

Para añadir un nuevo grupo, pulsamos el botón de *Nuevo Tricount* y nos muestra una pantalla emergente donde nos da dos opciones, la primera es iniciar un Tricount (grupo) desde 0 y la otra es unirnos a un grupo ya existente (*Figura 3.4*).

Otra de las pestañas que nos muestra es la del perfil, donde nos permite iniciar sesión y en caso de que ya estuvisemos registrado, se nos importan todos los grupos que estuvieran asociados a nuestra cuenta (*Figura 3.5*). Ahora vamos a hablar de lo realmente importante de la aplicación y es de cómo crear un grupo y añadir gastos. Para ello después de pulsar el botón de *Nuevo Tricount* nos da la opción de crear un grupo desde 0, le damos un

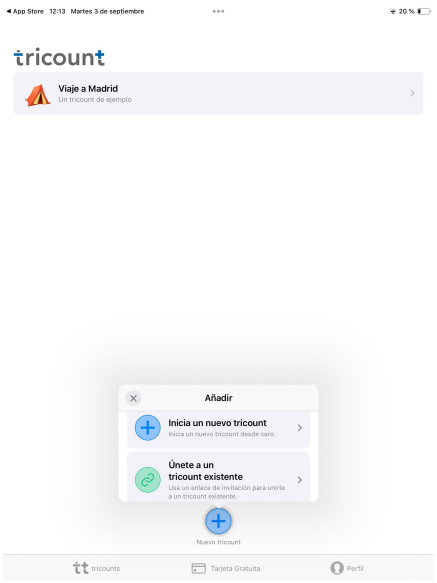


Figura 3.4: Añadir un nuevo grupo.

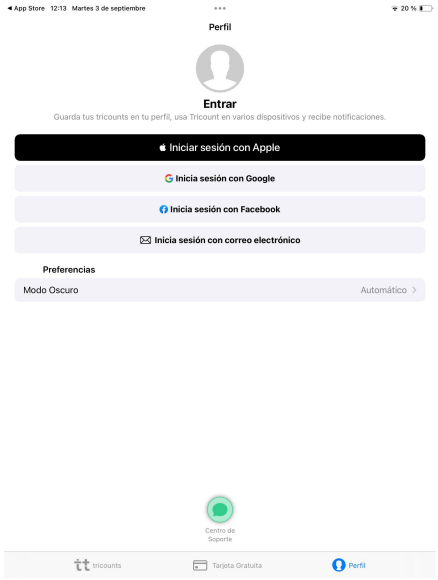


Figura 3.5: Pantalla del perfil.

nombre, añadimos los miembros que van a participar en el grupo y elegimos la divisa.(*Figura 3.6*). Una vez creado nos permite enviar un enlace a la gente que va a participar en el grupo (*Figura 3.7*).

Lo siguiente que vamos a hacer es añadir un gasto, para ello pulsamos el botón de *Añadir un gasto* y nos muestra una pantalla donde podemos añadir el nombre del gasto, la cantidad, la fecha, quien ha realizado el pago y a quien le corresponde pagar (*Figura 3.8*). Por último, nos muestra un resumen de los gastos que se han realizado y las deudas que tiene cada miembro del grupo (*Figura 3.9*).

3.2.1.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 4.8/5 estrellas y cuenta con más de 10 millones de descargas y en la tienda oficial de Apple (*App Store*) tiene una calificación de 4.7/5 estrellas. La mayoría de comentarios son positivos y destacan la facilidad de uso y la utilidad de la aplicación.

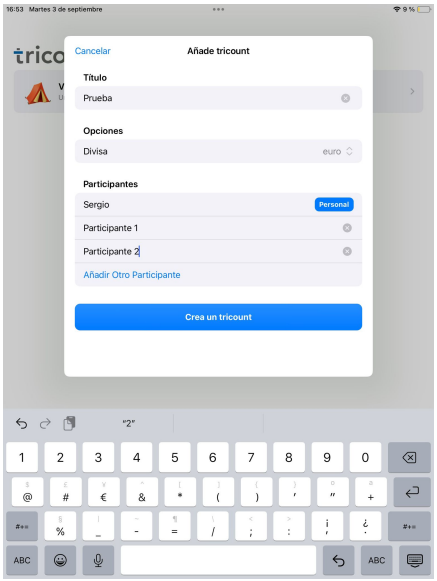


Figura 3.6: Detalles de un nuevo grupo.

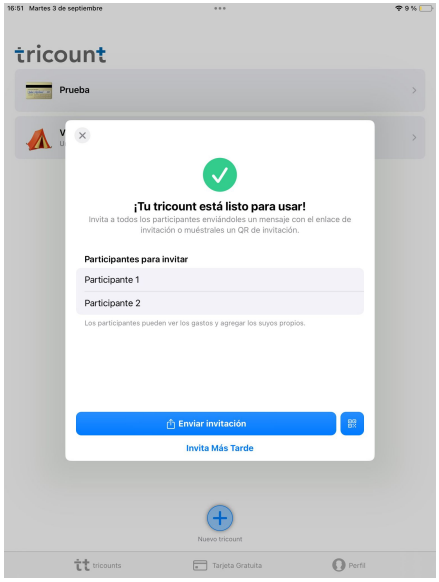


Figura 3.7: Enviar invitación a los miembros del grupo.

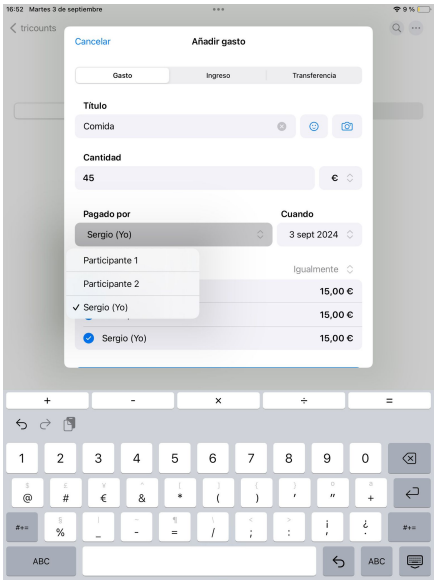


Figura 3.8: Añadir un gasto.

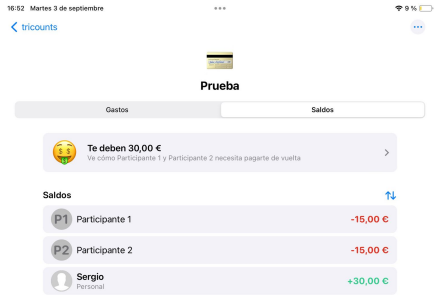


Figura 3.9: Resumen de los gastos y deudas.

3.2.2. Todoist

Todoist es una aplicación de gestión de tareas y proyectos que ayuda a los usuarios a organizar sus actividades de manera diaria. Es ampliamente utilizada tanto por personas como por equipos, debido a su flexibilidad, interfaz intuitiva y capacidad para gestionar desde simples listas de tareas hasta proyectos complejos. Está disponible para Android, iOS, Windows, Mac y la web. (Todoist, 2025)

3.2.2.1. Funciones Principales

1. **Gestión de Tareas**

    a) Añadir Tareas. Pueden crear tareas rápidamente. Cada tarea puede tener un título, una descripción y se puede asignar un proyecto o etiqueta para mantener todo organizado.

    b) Fechas de Vencimiento y Recordatorios. Se puede establecer una fechas límites para las tareas, además de recordatorios específicos, lo que ayuda a asegurarse de que las tareas se hagan a tiempo.

    c) Tareas Recurrentes. Si hay tareas que se repiten, se pueden configurar como tareas recurrentes.

2. **Proyectos y Subtareas**:

    a) Proyectos. Permite organizar las tareas dentro de proyectos. Los proyectos son carpetas donde se agrupan las tareas relacionadas con un objetivo o área específica.

    b) Subtareas. Para tareas más complejas, pueden descomponerlas en subtareas, permitiendo gestionar pasos más pequeños.

3. **Etiquetas y Filtros**

    a) Etiquetas. Permiten agrupar tareas de diferentes proyectos a través de características comunes.

    b) Filtros. Permite crear vistas personalizadas de las tareas, según criterios como fechas, etiquetas, etc.

4. **Prioridades**. Permite asignar niveles de prioridad a cada tarea, de 1 a 4

5. **Colaboración y Gestión de Equipos**

    a) Asignar Tareas. En proyectos compartidos, puedes asignar tareas a otros miembros del equipo, facilitando la colaboración en proyectos grupales.

b) Comentarios y Archivos. Los usuarios pueden añadir comentarios.

c) Compartir Proyectos. Puedes compartir proyectos con otros usuarios, lo que permite que todos los miembros vean, añadan o editen tareas dentro de ese proyecto.

6. **Integraciones**. Se integra con una amplia gama de aplicaciones y servicios, como Google Calendar, Slack, Dropbox, etc.

3.2.2.2. Modo de Uso

Lo primero que tenemos que hacer la primera que entramos en la aplicación es registrarnos a través de las diferentes opciones que nos permite la aplicación (*Figura 3.10*). Una vez completado este paso nos permite elegir el ámbito, en el que planeas usar la aplicación (he elegido personal porque es la que más se ajusta a la aplicación que estamos buscando). (*Figura 3.11*). Hecho todo esto nos lleva a la pantalla principal donde nos muestra las tareas que vencen hoy, en este caso no aparece ninguna porque aún no hemos añadido nada (*Figura 3.12*). Si le damos a añadir nos permite poner una descripción a la tarea, una fecha de vencimiento, una etiqueta, una prioridad, etc. (*Figura 3.13*). Una vez añadida nos aparece la tarea en nuestra lista de tareas pendientes donde la podemos marcar como realiza o no realizada. (*Figura 3.14*). También tenemos otra pestaña donde nos muestra un calendario con las tareas que tenemos pendientes por días, además de la tarea que hemos añadido, nos muestra tareas de ejemplo (*Figura 3.15*). Por último, cabe añadir que la aplicación cuenta con otra pestaña donde se puede buscar tareas y otra para explorar ya que podemos tener tareas en diferentes bandejas.

3.2.2.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 4.5/5 estrellas y cuenta con más de 10 millones de descargas y en la tienda oficial de Apple (*App Store*) tiene una calificación de 4.7/5 estrellas. La mayoría de comentarios son positivos y destacan la facilidad de uso y la utilidad de la aplicación. La versión de pago tiene un coste de 4.99€/mes o 47.99€/año.



Figura 3.10: Pantalla de registro.

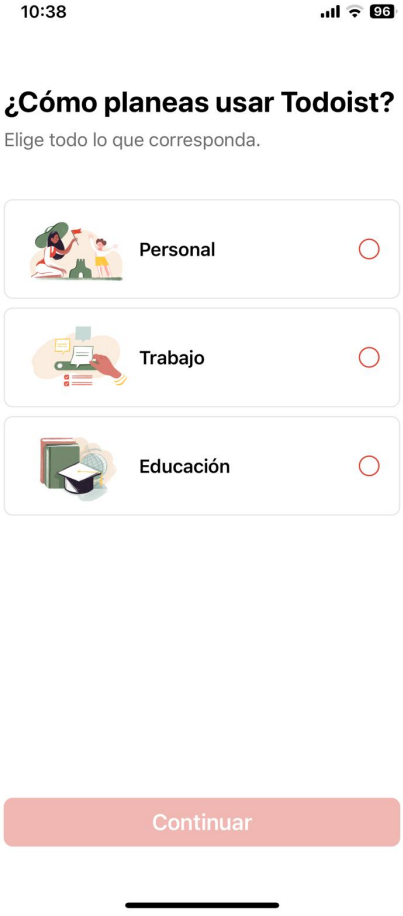


Figura 3.11: Elegir el ámbito de la aplicación.

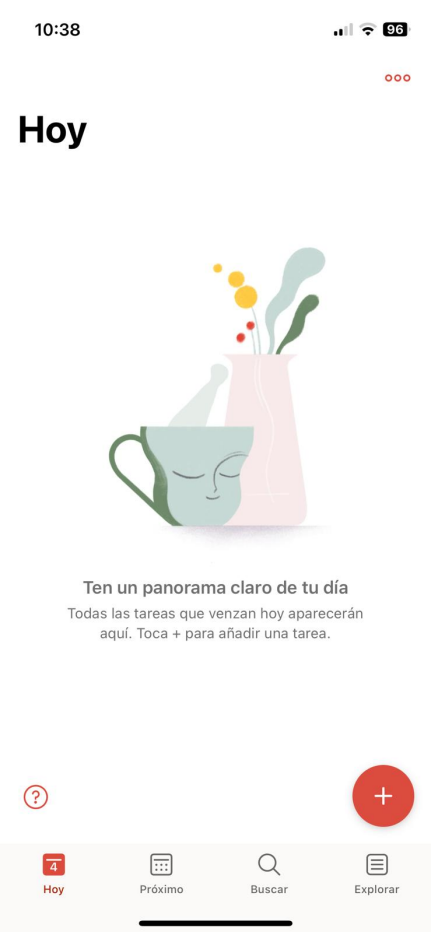


Figura 3.12: Pantalla principal.



Figura 3.13: Añadir una tarea.

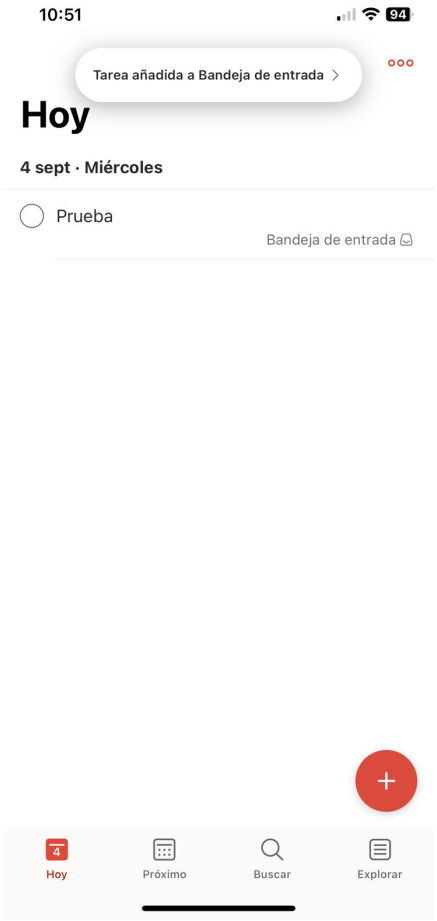


Figura 3.14: Pantalla con la tarea añadida.

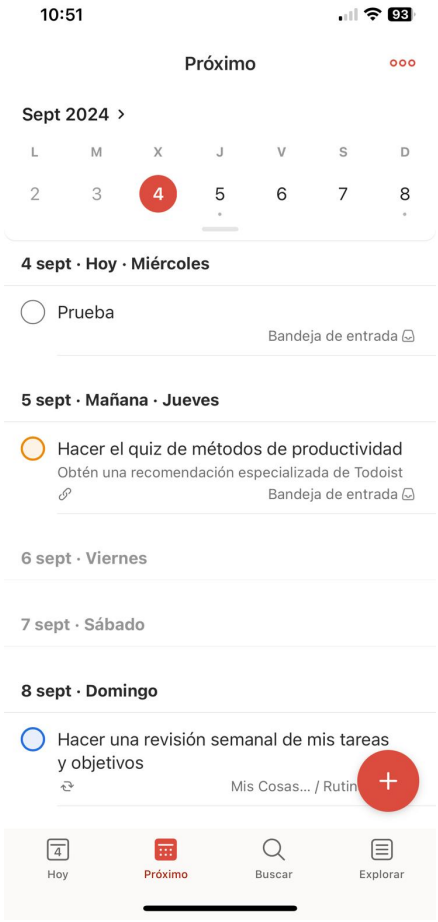


Figura 3.15: Calendario de tareas.

3.2.3. MeToca

Es una aplicación desarrollada por el gobierno de España con el objetivo de facilitar la organización de tareas del hogar de manera equitativa y eficiente. Es una iniciativa para promover la corresponsabilidad en el hogar, asegurando que las responsabilidades domésticas sean distruidas de manera justa entre todos los miembros de la familia. Es gratuita y en un principio estaba disponible para Android e iOS, pero en la actualidad solo está disponible para Android. (Ministerio de Igualdad, 2023)

3.2.3.1. Funciones Principales

1. **Gestión de Tareas del Hogar**

    a) Asignación de tareas. Permite asignar tareas a cada miembro del hogar. Las tareas pueden ser de cualquier tipo

    b) Calendario y Recordatorios. Incluye un calendario integrado donde se pueden programar las tareas frecuentes o únicas. Además envía recordatorios en caso de no hacerlas a tiempo.

2. **Seguimiento y Cumplimiento**

    a) Registro de Tareas Completadas. Cada usuario puede marcar sus tareas completadas y así poder llevar unn control del cumplimiento de responsabilidades.

    b) Estadísticas de Participación. Genera informes y estadísticas sobre cómo se están distribuyendo las tareas y si se están cumpliendo.

3. **Distribución Equitativa**

    a) Corresponsabilidad. Fomenta la corresponsabilidad en el hogar a través de la visualización y el seguimiento de tareas todos los miembros pueden ver si las responsabilidades se están distribuyendo de manera equitativa.

4. **Soporte para diferentes tipos de hogares**

    a) Adaptabilidad. Está diseñada para ser inclusiva y adapatable para cualquier tipo de hogar o de familia

3.2.3.2. Modo de Uso

En este caso no se ha podido probar la aplicación porque esta cerrada temporalmente, debido a la falta de usuarios.

3.2.3.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 1.6/5 estrellas, con apenas más de 10.000 descargas, aunque los creadores aseguran que tienen muchisimas funcionalidades la mayoría de comentarios aseguran que no les resulta muy útil.

3.2.4. Cozi Family Organizer

Es una aplicación diseñada para ayudar a las familias a organizar y gestionar sus actividades diarias y así organizar y gestionar sus actividades de manera colaborativa. Cozi se enfoca en la simplicidad y en proporcionar funciones esenciales para coordinar las actividades de varios miembros de una familia ocupada. Está disponible para Android, iOS y la web. Dispone de una versión gratuita que incluye la mayoría de las funcionalidades principales y una versión de pago con funciones adicionales. (Cozi Inc., 2025)

3.2.4.1. Funciones Principales

1. **Calendario Familiar**

   a) Calendario Unificado. El calendario de Cozi hace que todos los miembros de la familia tengan acceso a un calendario compartido. Cada miembro de la familia tiene un color asignado, lo que facilita ver quién tiene cada actividad y cuando tiene que hacerla.

   b) Eventos y Recordatorios. Se pueden añadir eventos y establecer recordatorios para asegurar que nadie olvide compromisos importantes, cómo citas médicas, actividades escolares o eventos familiares.

   c) Notificaciones. La aplicación envía notificaciones a los miembros de la familia sobre los eventos próximos, asegurando que todos estén informados de los planes.

2. **Lista de Compras Compartidas**

   a) Lista de Compras Colaborativas. Permite crear y compartir listas de compras entre todos los miembro de la familia, lo que permite que varias personas puedan contribuir a la lista y cualquier miembro puede marcar articulos que ya se han comprado.

   b) Actualizaciones en Tiempo Real. La lista de compras se actualiza en tiempo real, lo que significa que cualquier cambio que se haga se refleja inmediatamente en todos los dispositivos.

c) Organización por Tienda. Las listas pueden organizarse por tipo de tienda o categoría.

3. **Listas de Tareas**

   a) Tareas Pendientes. Permite crear listas para proyectos domésticos, responsabilidades diarias o cualquier tipo de actividades pendientes

   b) Marcar Completadas. Los usuarios pueden marcarla como hecha, lo que ayuda a mantener un seguimiento claro de lo que se ha logrado

4. **Planificación de Comidas**

   a) Organizador de Menús. Facilita organizar las comidas con antelación y evita el estrés diario de decidir qué cocinar.

   b) Recetas Integradas. Puedes guardar y organizar recetas directamente en la aplicación y con solo un clic.

   c) Sincronización con el calendario. Los menús planificados pueden sincronizarse con el calendario para que toda la familia lo vea.

5. **Diario Familiar**. Es una función donde los miembros pueden registrar eventos o recuerdos importantes, como fotos y notas sobre actividades familiares. Además se puede compartir con otro familiares y amigos.

3.2.4.2. Modo de Uso

Después de descargar e instalar esta aplicación, lo primero que nos muestras es una pantalla de registro/inicio de sesión (*Figura 3.16*). donde nos pide un nombre, el correo electrónico y una contraseña. Después de crear nuestra cuenta nos muestra una pantalla donde nos pide el nombre de nuestra familia y los miembros que la componen (*Figura 3.17*). Después de esto nos permite añadir un evento al calendario, con un nombre, la fecha, quién la tiene que realizar, si se repite o no y cúando queremos que nos lo recuerde (*Figura 3.18*). Una vez completado este paso nos muestra el un calendario con todas las tareas que tenemos pendientes (*Figura 3.19*). Si nos vamos a la pestaña de "List"nos muestras todas las listas de compras o de cosas que hacer ya que están divididas en dos pestañas, en la (*Figura 3.20*) podemos ver un ejemplo de lista de la compra con un árticulo añadido a una de ellas. Otra de las pestañas que nos ofrece la aplicación es la de Recetas"donde nos muestra recetas de cocina y además nos deja añadir más recetas a través de una url de la receta encontrada (*Figura 3.21*). Por último, la pestaña final es la de "Familia"donde nos permite configurar información sobre la familia, por ejemplo, añadir una foto de perfil o añadir a un miembro más a la familia.

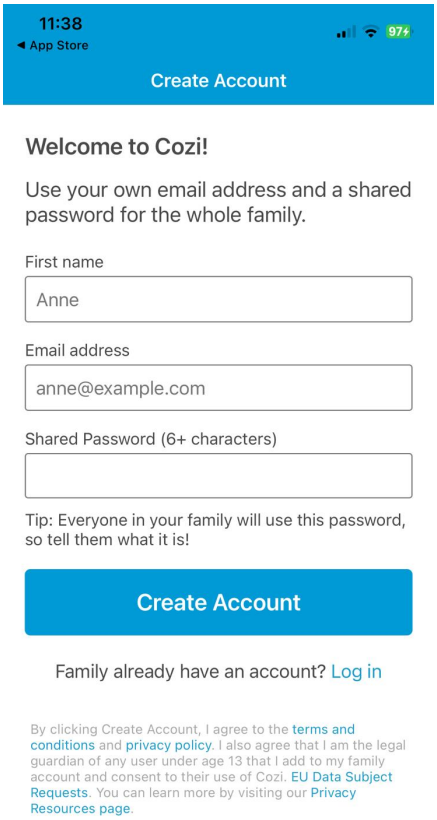


Figura 3.16: Registro.

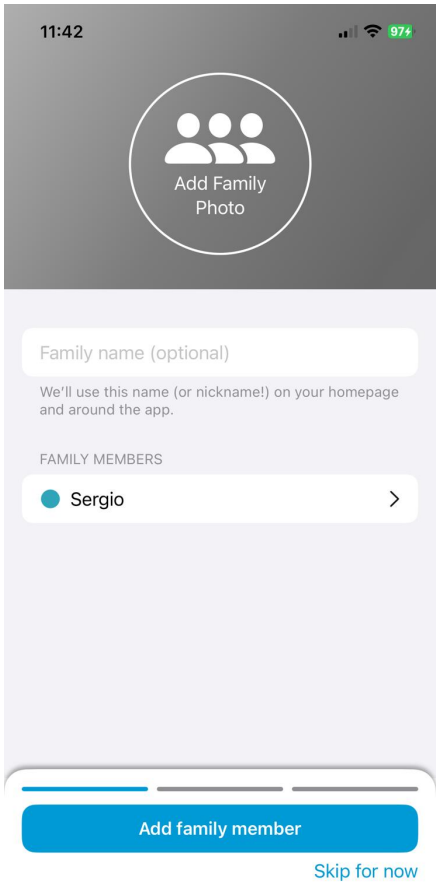


Figura 3.17: Crear una familia.

3.2.4.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 4.1/5 estrellas y cuenta con más de 5 millones de descargar. En la tienda oficial de Apple (*App Store*) tiene una calificación de 4.8/5 estrellas. La mayoría de comentarios son positivos y destacan la facilidad de uso y la utilidad de la aplicación. Por lo que he podido observar la página esta completamente en inglés.

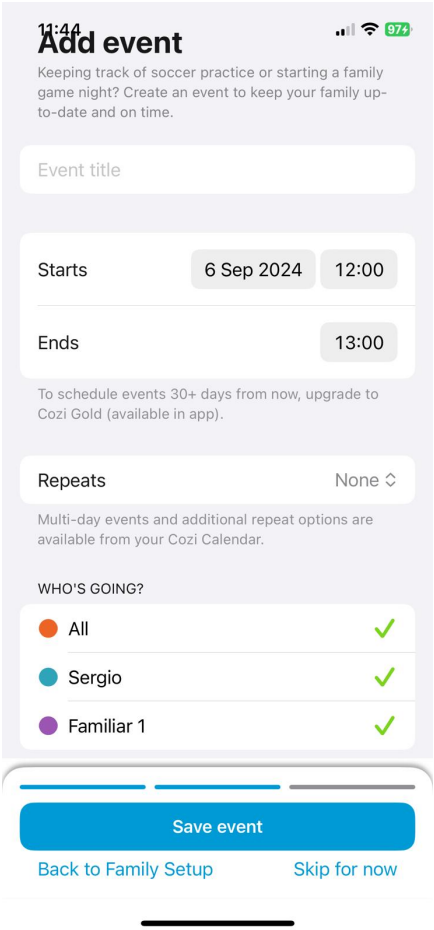


Figura 3.18: Añadir un evento.

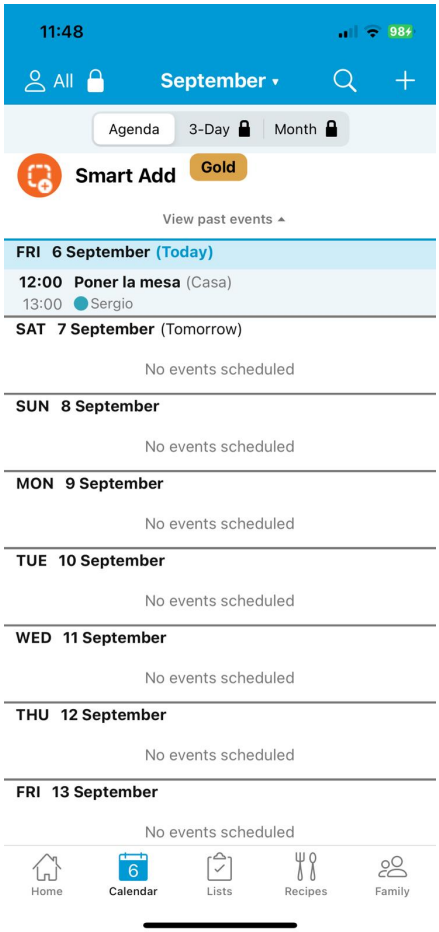


Figura 3.19: Calendario.

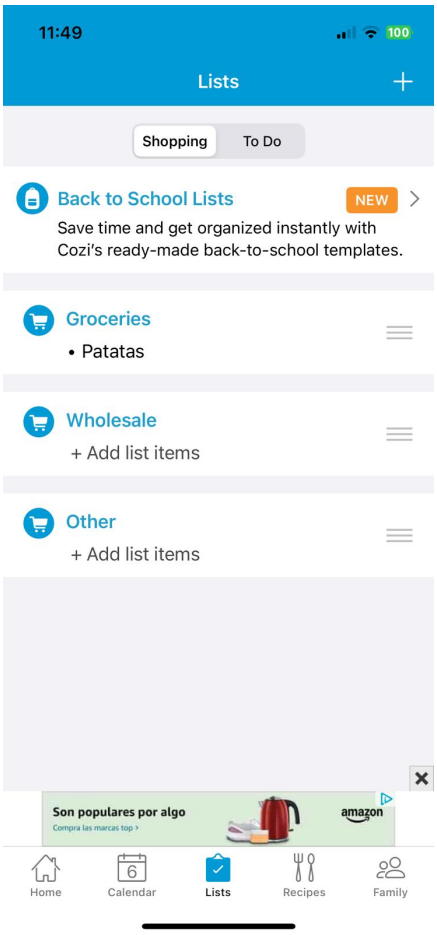


Figura 3.20: Listas de la compra.

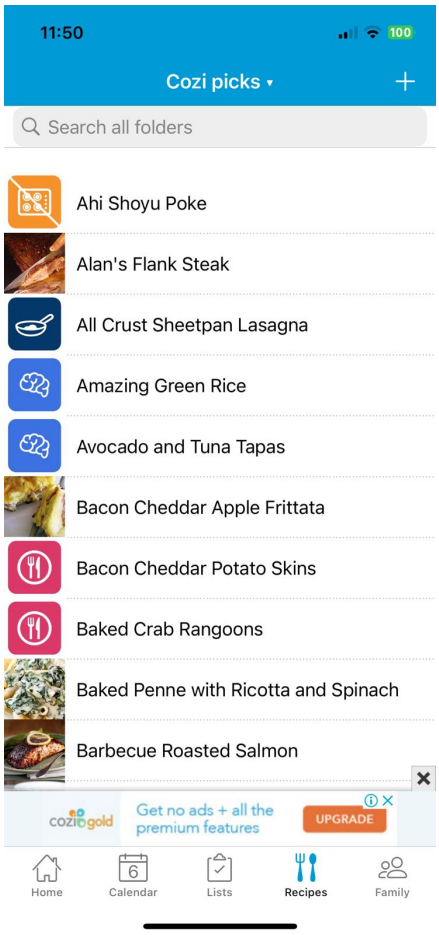


Figura 3.21: Recetas.

3.2.5. OurHome

Es una aplicación que ayuda a familias a organizar y gestionar tareas, compras y responsabilidades diarias de manera colaborativa. Está diseñada para facilitar la administración de tareas en el hogar y fomentar la cooperación familiar mediante asignación de tareas, el seguimiento de recompensas y la planificación de compras. (OurHome, 2025)

3.2.5.1. Funciones Principales

1. **Asignación de Tareas**. Permite asignar y crear tareas a los miembros de la familia, definiendo la frecuencia y quien debe hacer cada tarea y en qué plazo. Además incluye un sistema de recompensas basado en puntos, donde los miembros ganan puntos al completar tareas.
2. **Lista de Compra Compartida**. Incluye una lista de compras integrada que puede ser gestionada por cualquier miembro de la familia. Permite categorizar artículos y marcarlos como comprados.
3. **Calendario Familiar**. Incluye un calendario compartido donde se pueden añadir eventos y recordatorios, donde cada miembro puede ver y agregar eventos.
4. **Seguimiento de Objetivos**. Permite a los padres establecer objetivos específicos para los niños o cualquier miembro de la familia, cómo hábitos de estudio o rutinas diarias
5. **Sistema de notificaciones**. Los usuarios reciben notificaciones cuando se les asigna una tarea, se acerca la fecha límite o se actualiza la lista de compras.

3.2.5.2. Modo de Uso

Si nos metemos desde la página web oficial de la aplicación, nos aparece un enlace para que la descarguemos en nuestro dispositivo iOS, sin embargo, en mi dispositivo iOS no me da la opción de instalarmela, por lo tanto, tras investigar, llegué a la conclusión de que la aplicación no está disponible para nuestra región.

3.2.5.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene no tiene ningún tipo de calificación y solo cuanta con un poco más de 500 descargas. No aparece ningún tipo de comentario por lo tanto se deduce, que no es una aplicación muy popular.

3.2.6. Comparativa

A continuación se muestra una tabla comparativa de las aplicaciones analizadas, donde se resumen las principales características y funcionalidades de cada una:

Características	Tricount	Todoist	MeToca	Cozi Family Organizer	OurHome
Plataforma	Android, iOS, Web	Android, iOS, Windows, Mac, Web	Android	Android, iOS, Web	Android
Coste	Gratuito	Gratuito y Pago	Gratuito	Gratuito y Pago	Gratuito
Calificación Media	4.75	4.6	1.6	4.45	No disponible
Control de Gastos	✓	–	–	–	–
Gestión de Tareas	–	✓	✓	✓	✓
Calendario	–	✓	–	✓	✓
Lista de Compras	–	–	–	✓	✓
Asignación de Tareas	–	–	✓	✓	✓

Cuadro 3.1: Comparativa de aplicaciones para la gestión de tareas y gastos.

La Tabla 3.1 presenta una comparativa de diversas aplicaciones utilizadas para la gestión de tareas y organización en hogares compartidos. Se han seleccionado cinco aplicaciones populares que ofrecen diferentes funcionalidades para facilitar la administración de gastos, la planificación de tareas y la coordinación entre los miembros del hogar.

Cada fila de la tabla representa una característica que permite evaluar las diferencias entre las aplicaciones. En primer lugar, se muestra la **plataforma** en la que cada aplicación está disponible, lo que puede influir en la accesibilidad y compatibilidad para los usuarios. Posteriormente, se indica el **coste**, diferenciando entre aplicaciones completamente gratuitas y aquellas que ofrecen versiones premium con funcionalidades adicionales.

La columna de **calificación media** refleja la puntuación promedio dada por los usuarios en las tiendas de aplicaciones, lo que proporciona una referencia sobre la satisfacción y confiabilidad de cada herramienta. En cuanto a las funcionalidades, se incluyen características clave como el **control de gastos**, útil para administrar pagos y contribuciones entre compañeros de piso; la **gestión de tareas**, que permite asignar y organizar responsabilidades dentro del hogar; **el calendario**, que facilita la planificación de eventos o recordatorios compartidos; **la lista de compras**, ideal para coordinar la adquisición de productos necesarios; y **la asignación de tareas**, que permite distribuir responsabilidades entre los miembros del hogar.

A partir de esta comparativa, se puede observar que **Tricount** es la única aplicación que se centra exclusivamente en el control de gastos, mientras que **Todoist** destaca en la gestión de tareas, aunque no ofrece funcionalidades

específicas para hogares compartidos. Por otro lado, **Cozi Family Organizer** y **OurHome** son aplicaciones más completas, ya que integran múltiples funcionalidades como listas de compras, asignación de tareas y calendarios. Sin embargo, en términos de calificación, **MeToca** presenta una puntuación baja, lo que puede indicar posibles problemas de usabilidad o funcionalidad en comparación con el resto.

3.3. Legislación actual de protección de datos

Al desarrollar una aplicación para la gestión del hogar compartido, hay que tener en cuenta la privacidad y seguridad de los datos de los usuarios. En un entorno donde varias personas comparten responsabilidades y gastos, es inevitable manejar información como nombres, correos electrónicos, deudas pendientes e incluso hábitos de convivencia. Sin una adecuada protección de estos datos, podrían generarse situaciones de vulnerabilidad. Por ello, es fundamental que la aplicación cumpla con la Ley de Protección de Datos, garantizando que los usuarios tengan control sobre su información y que esta sea almacenada y procesada de manera segura.

La **Ley de Protección de Datos** está diseñada para garantizar el derecho a la privacidad y la protección de los datos personales, evitando el uso indebido o la divulgación de la información personal de personas. En la Unión Europea, la normativa más relevante es el **Reglamento General de Protección de Datos (GDPR)**, que entró en vigor en 2018 y establece un estándar estricto para el manejo de datos en todos los países miembros, aplicando también a empresas fuera de la UE que traten datos de ciudadanos europeos. En España, esta ley se complementa con la **Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPD-GDD)**, que adapta el GDPR pero al contexto español. (Parlamento Europeo, 2016)

Las principales medidas de este reglamento son: (Jefatura del Estado, 2018)

1. **Consentimiento explícito:** Las organizaciones deben obtener el consentimiento explícito de los usuarios antes de recopilar sus datos. Este consentimiento debe ser claro, específico e informado, lo que significa que los usuarios deben saber exactamente qué datos se recogen y para qué fines se usarán.

2. **Derecho al Acceso, Rectificación y Supresión:** Los usuarios tienen el derecho de acceder a sus datos personales, solicitar correciones o eliminarlos. Esto se conoce como *derecho al olvido* y permite a los usuarios mantener control sobre su información en línea.

3. **Transparencia y Finalidad:** Los empresas deben informar a los usuarios sobre qué datos recopilan, el propósito de dicha recopilación y durante cuánto tiempo los conservarán. Esto busca evitar el almacenamiento de excesivo de datos y el uso para fines distintos a los incialmente indicados.

4. **Seguridad de los datos:** Las organizaciones deben implementar medidas de seguridad para proteger los datos, como el cifrado y las políti-

cas de acceso restringido. Estas medidas están orientadas a minimizar el riesgo de filtraciones de datos y accesos no autorizados.

5. **Notificación de brechas de seguridad:** Si ocurre una violación de seguridad que afecte datos personales, las empresas deben notificar a las autoridades competentes y a los afectados en un plazo de 72 horas.

Estas medidas buscan proteger los derechos de privacidad de los usuarios y obligan a las empresas a adoptar prácticas de manejo de datos que respeten la seguridad, confidencialidad y transparencia.

3.4. Estudio de usabilidad y accesibilidad

3.4.1. Estudio de usabilidad

La **usabilidad** se refiere a la facilidad con la que los usuarios pueden interactuar con un producto o sistema, para alcanzar sus objetivos de manera eficiente y satisfactoria. La usabilidad se define como ün atributo de calidad que evalúa qué tan fáciles de usar son las interfaces de usuario"(Nielsen, 1993).

Las principales características de la usabilidad son (Staff, 2023):

1. **Facilidad de aprendizaje:** Capacidad del sistema para permitir que nuevos usuarios realicen tareas básicas la primera vez que lo utilizan.
2. **Eficiencia:** Una vez que los usuarios han aprendido el diseño, deben poder realizar tareas con rapidez y precisión.
3. **Memorabilidad:** Los usuarios deben poder retomar el uso del sistema después de un período de inactividad sin necesidad de reaprender todo.
4. **Manejo de errores:** El sistema debe minimizar la frecuencia y gravedad de los errores, y permitir una recuperación fácil cuando ocurran.
5. **Satisfacción:** El uso del sistema debe de ser agradable, proporcionando una experiencia positiva al usuario.

Para garantizar una buena usabilidad en una aplicación web, es fundamental aplicar principios de diseño que optimicen la experiencia del usuario. (Nielsen, 1993) (Chisholm y Vanderheiden, 1999).

- **Estilo y Consistencia.** La coherencia en la disposición de elementos, tipografías y espacios en blanco facilita la navegación y el aprendizaje del usuario. Según Jakob Nielsen, la consistencia y los estándares permiten que los usuarios comprendan mejor la interfaz sin necesidad de reaprender.
- **Uso del Color.** Los colores influyen en la percepción del usuario y pueden guiar su atención. Es recomendable emplear contrastes adecuados para mejorar la legibilidad y cumplir con las pautas de accesibilidad web. Además, los colores deben transmitir el propósito de la aplicación; por ejemplo, el rojo se asocia con urgencia, mientras que el verde transmite seguridad y confirmación.
- **Tipografía.** Para mejorar la legibilidad, se recomienda el uso de fuentes simples y tamaños adecuados que faciliten la lectura en distintos

dispositivos. Mantener una cantidad limitada de tipografías contribuye a la coherencia visual y evita distracciones innecesarias.

- **Espaciado y Distribución.** Un diseño limpio con suficiente espacio en blanco permite que los usuarios procesen la información de manera más efectiva, evitando la sobrecarga cognitiva y mejorando la comprensión del contenido.
- **Iconografía y Elementos Visuales.** El uso de iconos intuitivos y representativos ayuda a los usuarios a comprender funciones o secciones sin necesidad de leer texto adicional. Para garantizar una experiencia homogénea, es importante que estos elementos sean coherentes en toda la interfaz.

La usabilidad es fundamental para garantizar que los productos o sistemas sean accesibles y efectivos para los usuarios previstos. Un alto nivel de usabilidad conduce a una mayor satisfacción del usuario, reduce los errores y aumenta la eficiencia en la realización de tareas. Además, mejora la adopción del producto y puede proporcionar una ventaja competitiva en el mercado. Una buena usabilidad asegura que los usuarios puedan navegar e interactuar con el sistema de manera intuitiva, encontrando la información que buscan y completando acciones sin dificultades (Adverthia, 2019).

3.4.2. Estudio de accesibilidad

La **accesibilidad** en el desarrollo de software se refiere al diseño de aplicaciones que puedan ser utilizadas por el mayor número de personas, incluyendo aquellas con discapacidades físicas, cognitivas o sensoriales. Diseñar software implica garantizar que los programas y sus funcionalidades estén disponibles para una amplia gama de usuarios, independientemente de sus capacidades (ThoughtWorks, 2020).

Las principales características de la accesibilidad son: (Burriel, 2024)

1. **Perceptible:** La información y los componentes de la interfaz deben ser presentados de manera que los usuarios puedan percibirlos. Esto incluye proporcionar alternativas textuales para contenido no textual y asegurar que el contenido sea adaptable a diferentes formatos.
2. **Operable:** Los componentes de la interfaz y la navegación deben ser utilizables por todos los usuarios. Por ejemplo, garantizar que todas las funciones sean accesibles mediante teclado y proporcionar suficiente tiempo para que los usuarios realicen tareas.
3. **Comprensible:** La información y el funcionamiento de la interfaz deben ser claros y coherentes, facilitando la comprensión y el uso del

sistema. Esto implica utilizar un lenguaje sencillo y predecible, así como proporcionar asistencia cuando sea necesario.

4. **Robusto:** El contenido debe ser lo suficientemente robusto como para ser interpretado de manera fiable por una amplia gama de agentes de usuario, incluyendo tecnologías de asistencia. Esto asegura que el contenido siga siendo accesible a medida que las tecnologías avanzan.

Segun (Carpena, 2023) algunos principios y prácticas recomendadas para mejorar la accesibilidad en el desarrollo de software son:

- **Uso de colores y contrastes adecuados:** Los colores deben facilitar la lectura y navegación, considerando a usuarios con dificultades visuales o daltonismo. Es recomendable utilizar combinaciones de colores que aseguren un contraste suficiente entre el texto y el fondo.
- **Diseño adaptable (Responsive Design):** El diseño debe adaptarse a diferentes dispositivos y tamaños de pantalla, garantizando una experiencia de usuario consistente y accesible en móviles, tabletas y ordenadores.
- **Navegación sencilla y coherente:** Una estructura de navegación clara y predecible facilita que los usuarios encuentren información y funciones sin dificultad. Es importante mantener una jerarquía lógica en los menús y enlaces.
- **Uso correcto de HTML y etiquetas semánticas:** Emplear etiquetas HTML adecuadas y semánticas mejora la accesibilidad y comprensión del contenido, tanto para usuarios como para tecnologías de asistencia.
- **Provisión de alternativas para contenido multimedia:** Ofrecer subtítulos, transcripciones o descripciones para contenido audiovisual asegura que todos los usuarios puedan acceder a la información, independientemente de sus capacidades sensoriales.
- **Evitar contenido intermitente o parpadeante:** El uso de elementos que parpadean o se mueven puede ser molesto o incluso perjudicial para algunos usuarios. Es recomendable evitar este tipo de contenido o proporcionar opciones para desactivarlo.
- **Optimización de formularios y controles:** Los formularios deben ser fáciles de completar, con etiquetas claras y campos de entrada accesibles. Además, es importante proporcionar mensajes de error comprensibles y sugerencias para corregirlos.

- **Pruebas de accesibilidad regulares:** Realizar evaluaciones y pruebas de accesibilidad durante el desarrollo y mantenimiento del software ayuda a identificar y corregir posibles barreras, asegurando que la aplicación sea accesible para todos los usuarios.

Implementar la accesibilidad en el desarrollo de software garantiza que todas las personas, independientemente de sus capacidades, puedan interactuar con las aplicaciones de manera efectiva. Esto no solo cumple con normativas legales y éticas, sino que también amplía la base de usuarios potenciales y mejora la experiencia general del usuario. Además, la accesibilidad beneficia a otros grupos, como personas mayores o usuarios con limitaciones temporales, y mejora la usabilidad en diferentes contextos de uso (Mozilla Developer Network, 2024).

Capítulo 4

Análisis de tecnologías

4.1. Análisis de las tecnologías Frontend

En la actualidad existen multitud de opciones en el mercado para el desarrollo de tecnologías móviles. A continuación se describen las tecnologías más populares y utilizadas en la actualidad para el desarrollo de aplicaciones móviles.

4.1.1. Desarrollo nativo

En primer lugar tenemos las tecnologías de **desarrollo nativo**, es decir, las tecnologías que permiten la creación de aplicaciones específicamente diseñadas para un sistema operativo en concreto. Dichas tecnologías son:

- **Kotlin.** Kotlin es un lenguaje que permite desarrollar aplicaciones para todas las plataforams soportadas por Android, siendo Android Studio el entorno de desarrollo integrado (IDE) principal para la creación de aplicaciones nativas en dicho sistema operativo. Una de las ventajas más destacas es su total interoperabilidad con Java, lo que facilita el uso de bibliotecas escritas en Java dentro de proyectos desarrollados en Kotlin. Además, el lenguaje proporciona mecanismos avanzados de seguridad frente a valores nulos, lo que ayuda a prevenir errores durante la ejecución de las aplicaciones. Entre sus deventajas destaca que, aunque la popularidad ha crecido, aún no alcanzar el nivel de adopción de Java, lo que puede dificultar encontrar soluciones a problemas específicos. Asimismo, aunque existen iniciativas para su uso multiplataforma, Kotlin sigue estando principal,emte orientado al desarrollo de aplicaciones en Android. (JetBrains, 2025)
- **Swift.** Es un lenguaje, diseñado específicamente para crear aplicacio-



Figura 4.1: Logo de Kotlin

nes de alto rendimiento y con una sintaxis clara y moderna. En este caso el IDE es XCode utilizado para el desarrollo nativo en las plataformas de Aple, como iOS y macOS. Swift se caracteriza por su seguridad, facilidad de uso y capacidades de optimización, lo que lo convierte en una buena opción para el desarrolo de aplicaciones en el ecosistema de Apple. Como principales carácteristicas de Swift podemos destacar una **sintaxis simple y moderna**, un **alto rendimiento** debido a su compilador avanzado permite ejecutar aplicaciones de manera eficiente, **seguridad** gracias a su sistema de tipos fuertes y a la prevención de errores en tiempo de ejecución, e **interoperabilidad con Objetive-C** lo que facilita la integración de código. Sin embargo, una de las principales deventajas de Swift es similar a la mencionada anteriormente en Kotlin, **la duplicidad de esfuerzos** en el desarrollo de aplicaciones multiplataforma. Las aplicaciones desarrolladas con Swift obligan a los desarrolladors a crear aplicaciones independientes para otras plataformas. (Apple Inc., 2025)



Figura 4.2: Logo de Swift

4.1.2. Desarrollo Híbrido

Las tecnologías de desarrollo híbrido representan una alternativa eficaz en el ámbito de aplicaciones multiplataforma. Estas tecnologías permiten crear una única base de código que puede ser ejecutada tanto en dispositivos con sistema operativo iOS como en Android. La principal ventaja de este enfoque es la optimización de los recursos, tanto económicos como operativos, ya que reduce significativamente los costes asociados al desarrollo y mantenimiento de las aplicaciones. Al utilizar un solo lenguaje y una única estructura de programación, se consigue implementar una solución funcional para ambas plataformas. Además, permite una mejor experiencia al usuario, garantizando que la aplicación tenga un comportamiento y apariencia similares. Algunos ejemplos de esta tecnología son:

- **Ionic.** Es un framework de código abierto que permite crear aplica-

ciones móviles, web y de escritorio utilizando tecnologías webs, como HTML, CSS y JavaScript. Se integra con diferentes frameworks frontend como Angular React o Vue, lo que permite a los desarrolladores utilizar el conjunto de herramientas y librerías de su preferencia. Además, Ionic tiene componentes nativos que proporcionan una biblioteca de elementos visuales optimizados para dispositivos móviles. Cuenta también con una **amplia comunidad** y herramientas adicionales para la integración con servicios de backend y otras tecnologías. (Ionic Framework, 2025)



Figura 4.3: Logo de Ionic

- **Flutter.** Flutter es un framework de código abierto desarrollado por Google que permite crear aplicaciones nativas multiplatadorma con una única base de código. Utiliza el lenguaje de programación Darrt y proporciona un enfoque único para desarrollar aplicaciones móviles, web y de escritorio. Tiene un **alto rendimiento** debido a que utiliza un motor gráfico y compila directamente a código nativo, lo que garantizando un rendimiento similar al de aplicaciones nativas. Además , cuenta con *Hot Reload* que facilita el desarrollo rápido al permitir que lo cambios en el código se reflejen instantáneamente en la aplicación sin tener que recompilar por completo. (Google, 2025b)



Figura 4.4: Logo de Flutter

- **React Native.** React Native es un framework de código abierto desarrollado por Facebook que permite crear aplicaciones móviles nativas utilizando JavaScript y React. A diferencia de otros frameworks híbridos, React Native no se limita a emular una experiencia nativa, en su lugar, **genera componentes nativos**, lo que permite un rendimiento más cercano al de una aplicación pura. Además cuenta con una **amplia comunidad** y una gran cantidad de librerías y herramientas que facilitan el desarrollo de aplicaciones. (Meta Platforms, 2025)

- **Xamarin.** Es un framework de código abierto desarrollado por Microsoft que permite crear aplicaciones móviles nativas para iOS y Android



Figura 4.5: Logo de React Native

utilizando C# y .NET. Xamarin permite compartir hasta un 90 % del código. Además, proporciona acceso a las API nativas de cada plataforma, lo que permite a los desarrolladores crear aplicaciones con una **experiencia de usuario nativa**. También se integra a la perfección con herramientas del ecosistema de Microsoft, como Visual Studio, facilitando el desarrollo y la depuración de aplicaciones. (Microsoft, 2025)



Figura 4.6: Logo de Xamarin

4.1.3. Tabla comparativa de tecnologías front-end

La siguiente tabla ofrece una comparación entre las tecnologías más populares en el desarrollo frontend de aplicaciones móviles, evaluando aspectos clave como el lenguaje de programación, rendimiento, disponibilidad de componentes de UI, tamaño de la comunidad, soporte de funcionalidad *Hot Reload* e integración con otros sistemas y herramientas. Esta información proporciona una visión global de las fortalezas y limitaciones de cada opción, facilitando así la selección de la tecnología que mejor se ajuste a los requisitos específicos del proyecto.

Tecnología	Lenguaje	Rendimiento	Componentes UI	Comunidad	Hot Reload	Integración
Kotlin	Java	Nativo	Componentes Nativos	Amplia	No	Herramientas Nativas
Swift	Swift	Nativo	Componentes Nativos	Amplia	No	Herramientas Nativas
Ionic	JavaScript	Bueno	Basado en Web	Amplia	Sí	Buen Soporte
Flutter	Dart	Cercano a Nativo	Personalizados	Activa	Sí	Buen Soporte
React Native	JavaScript	Bueno	Componentes Nativos	Amplia	Sí	Buen Soporte
Xamarin	C#	Bueno	Componentes Nativos	Amplia	Sí	Integración Microsoft

Cuadro 4.1: Comparativa de tecnologías para desarrollo de aplicaciones móviles.

En conclusión, la tabla destaca cómo cada tecnología ofrece un conjunto único de beneficios. Las opciones nativas como Kotlin y Swift son preferibles para aplicaciones que requieran un rendimiento óptimo y un uso extensivo de componentes específicos de la plataforma. Sin embargo, soluciones como Flutter y React Native se presentan como alternativas atractivas para el desarrollo multiplataforma debido a su rendimiento cercano al nativo, su comunidad activa y sus herramientas avanzadas de desarrollo rápido como *Hot Reload*.

4.2. Analisis de las tecnologías Backend

Otra de las decisiones que hay que tomar es la tecnología back-end que permita la comunicación entre la aplicación y la base de datos. A continuación vamos a ver las diferentes tecnologías que hay para ello:

4.2.1. Firebase

Firebase es una plataforma de Google que ofrece una amplia variedad de servicios backend, lo que incluye **bases de datos en tiempo real** como FireStore y Realtime Database. Además, proporciona servicios de autenticación, almacenamiento de archivos, notificaciones push, análisis y etc. No requiere gestión de servidores debido a que es completamente serverless. Cuenta con una **fácil integración** con Flutter debido a que tiene SDks nativos para dicha tecnología, lo que simplifica mucho el desarrollo en comparación con otras tecnologías que requieren la integración manual. Sin embargo, Firebase tiene un **coste** asociado al uso de sus servicios, lo que puede ser un inconveniente para proyectos con presupuestos limitados. También tiene una **dependencia de Google**, porque todo esta alojados en la infraestructura de Google, lo que puede generar dependecia a largo plazo. (Google, 2025a)



Figura 4.7: Logo de Firebase

4.2.2. Node.js con Express

Node.js es un entorno de ejecución de JavaScript que permite ejecutar código JavaScript en el backend. Express es un framework ligero que permite construir aplicaciones web y APIs REST de manera rápida y sencilla. Además, es bastante útil para aplicaciones que manejan un alto número de solicitudes simultáneas. Para el almacenamiento de datos se pueden usar bases de datos realacionales como **MySQL** o **PostgreSQL** o bases de datos NoSQL como **MongoDB**. Gracias a npm (*Node Package Manager*), hay miles de paquetes disponibles que permiten acelerar el proceso de desarrollo. También es una tecnología **flexible**, porque permite elegir el tipo de base de datos. Por el contrario tiene una **Complejidad de gestión del servidor**. Aunque Express es minimalista, la creación y gestión de servidores, puede

ser más compleja en comparación con soluciones serverless. Además, la **curva de aprendizaje** puede ser más pronunciada para desarrolladores que no tienen experiencia previa con JavaScript. (Node.js Foundation, 2025)



Figura 4.8: Logo de Node.js

4.2.3. Django

Es un framework web de alto nivel escrito en Python que facilita la creación de aplicaciones webs seguras y escalables. Junto con **Django REST Framework**, permite exponer APIs REST de forma rápida y eficiente. Se integra bien con bases de datos relacionales como **PostgreSQL** y **MySQL**. Django proporciona un **rápido desarrollo** debido a que incluye una gran cantidad de herramientas listas para usar (*autenticación, administración, etc.*). Además, tiene carácteristicas de seguridad incorporadas, como protección CSRF, gestión de contraseñas seguras y más. También proporciona **gestión automática de bases de datos** sin tener que escribir consultas SQL Manuales. Sin embargo, Django puede ser algo pesado para proyectos más pequeños o simples. Además, la **curva de aprendizaje** puede ser pronunciada para desarrolladores que no tienen experiencia previa con Python. (Foundation, 2025)



Figura 4.9: Logo de Django

4.2.4. Ruby on Rails

Ruby on Rails es un framework MVC (*Modelo-Vista-Controlador*) escrito en Ruby, que permite desarrollar aplicaciones web y APis de manera rápida. Rails se integra perfectamente con bases de datos relacionales como **PostgreSQL** o **MySQL**. Una facilidad que incorpora Ruby on Rails es la **Convención sobre configuración**, lo que significa que muchas decisiones están predefinidas para que el desarrollador no tenga que configurarlas

manualmente, acelerando el proceso de desarrollo. También **facilita la interacción** con la base de datos relacionales. Sin embargo, Aunque Rails es excelente para aplicaciones web, no es tan común en el desarrollo de APIs para aplicaciones móviles. (Ruby on Rails, 2025)



Figura 4.10: Logo de Ruby on Rails

4.2.5. Tabla comparativa de tecnologías back-end

La tabla siguiente compara algunas de las tecnologías back-end más comunes en el desarrollo de aplicaciones web y móviles, evaluando aspectos como el lenguaje de programación, las bases de datos soportadas, la gestión del estado, la integración con APIs y la popularidad general.

Tecnología	Lenguaje	Base de Datos Soportada	Gestión del Estado	Integración con API	Popularidad
Firebase	-	Firebase, Realtime DB	Serverless, en tiempo real	APIs nativas de Firebase	Muy alta
Node.js	JavaScript	MongoDB, MySQL, PostgreSQL	Depende de la implementación	Muy flexible	Muy alta
Django	Python	PostgreSQL, MySQL	ORM Propio	Django REST Framework	Alta
Ruby on Rails	Ruby	PostgreSQL, MySQL	ActiveRecord ORM	RESTful API fácil de implementar	Media

Cuadro 4.2: Comparativa de tecnologías para el desarrollo back-end.

En conclusión, esta tabla muestra cómo cada tecnología back-end aporta ventajas específicas. Firebase destaca en aplicaciones en tiempo real y serverless, ideal para proyectos rápidos que requieren menos mantenimiento de servidores. Node.js es muy flexible y compatible con diferentes bases de datos, adaptándose bien a aplicaciones escalables. Django y Ruby on Rails ofrecen una estructura robusta y sencilla de implementar, siendo opciones populares para aplicaciones que necesitan un backend consolidado.

Capítulo 5

Análisis de metodologías de desarrollo

Las metodologías de desarrollo software han evoluciado con el tiempo para adaptarse a las diferentes necesidades y contextos de proyectos. En función de las características del proyecto y del equipo de desarrollo, será más conveniente usar una u otra.

En este capítulo se analizarán las metodologías de desarrollo de software más utilizadas en la actualidad. Se describirán las características de cada una de ellas, así como sus ventajas y desventajas. Además, se realizará una comparativa entre las metodologías más populares para determinar cuál es la más adecuada para el desarrollo de la aplicación.

5.1. Metodologías de desarrollo de software

Las metodologías de desarrollo de software son un conjunto de prácticas y técnicas que se utilizan para planificar, diseñar, implementar y probar un proyecto de software. A continuación, se describen las metodologías más utilizadas en la actualidad:

5.1.1. Modelo en cascada (Waterfall)

Una de las metodologías más tradicionales y conocidas es el **Modelo en Cascada** (Waterfall), el cual sigue un enfoque secuencial en el que cada fase del desarrollo debe completarse antes de avanzar a la siguiente. Esta metodología se compone principalmente de las etapas de recopilación de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento. Su principal ventaja es la claridad de los requisitos desde el inicio, lo que facilita una bue-

na planificación y documentación. Sin embargo, su principal inconveniente es la falta de flexibilidad, ya que cualquier cambio en los requisitos implica volver a fases anteriores, lo que puede generar retrasos y costos adicionales. Esta metodología sigue siedno utilizada en proyectos donde los requisitos están bien definidos y no se esperan cambios significativos. (Sommerville, 2015)

5.1.2. Modelo en espiral

El **Modelo en Espiral** introduce un enfoque iterativo que prioriza la gestión de riegos. En cada iteración se realiza actividades de planificación, análisis de riegos, implementación y evaluación, lo que permite detectar y mitigar problemas antes de que se conviertan en fallos grandes. Esta metodología es útil en proyectos grandes y complejos, donde la incertidumbre es alta. No obstante su aplicación suele ser costosa y requiere un gestión constante del proceso, lo que lo hace menos adecuado para proyectos pequeños. (Boehm, 1988)

5.1.3. Metodología iterativa

La **metodología iterativa** se basa en el desarrollo por ciclos repetitivos, donde en cada iteración se revisa, prueba y ajusta el software hasta alcanzar el resultado deseado. A diferencia del modelo en cascada, no requiere que todas las fases de desarrollo se completen antes de volver a una anterior, lo que permite una mayor flexibilidad y adaptabilidad. Esta metodología es adecuada para proyectos donde los requisitos no están bien definidos o pueden cambiar a lo largo del tiempo. (Atlassian, 2025)

5.1.4. Desarrollo iterativo e Incremental

El **Desarrollo Iterativo e Incremental** es otra alternativa que busca dividir el desarrollo en incrementos funcionales, donde cada iteración añade nuevas carácteristicas al sistema. A diferencia del modelo en cascada, este enfoque permite obtener retroalimentación continua y realizar ajustes en cada fase del proyecto. Su flexibilidad es una de sus mejores ventajas, ya que se pueden introducir mejoras en el software en función de las necesidades del usuario. Sin embargo, gestionar adecuadamente un proyecto basado en interaciones puede ser complejo y es díficil predecir costos y tiempos de finalización con precisión. (Larman, 2003)

5.1.5. Metodología Ágil

Recientemente, las **metodologías ágiles** han ganado popularidad en la industria del software debido a su capacidad de adaptación y entrega continua de software funcional. Estas metodologías incluyen enfoques como *Scrum*, que se basa en ciclos cortos de desarrollo llamadas *sprints*, y *Extreme Programming* (XP), que se centra en la calidad del código y la colaboración entre los miembros del equipo. También destaca *Kanban*, que utiliza tableros visuales para gestionar el flujo de trabajo. (Beck, 1999)

5.1.6. DevOps

Por último, el enfoque **DevOps** ha emergido como una metodología clave en la industria del software, ya que integra el desarrollo y las operaciones en un único flujo de trabajo. Su principal objetivo es mejorar la eficiencia del desarrollo y la entrega de software, así como la colaboración entre los equipos de desarrollo y operaciones. Sin embargo, su adopción puede ser un reto para organizadores que no cuentan con la infraestructura y cultura adecuada. (Kim, Humble, Debois, y Willis, 2016)

5.2. Comparativa de metodologías

A continuación, se presenta una comparativa entre las metodologías de desarrollo de software más utilizadas en la actualidad:

Metodología	Flexibilidad	Gestión de Riesgos	Coste	Tiempo	Adaptabilidad	Documentación
Waterfall	Baja	Baja	Bajo	Largo	Baja	Alta
Modelo en Espiral	Alta	Muy Alta	Alto	Largo	Alta	Media
Iterativo	Alta	Media	Medio	Medio	Alta	Media
Iterativo e Incremental	Alta	Media	Medio	Medio	Alta	Media
Ágil	Muy Alta	Alta	Variable	Corto	Muy Alta	Baja
DevOps	Alta	Alta	Variable	Variable	Muy Alta	Baja

Cuadro 5.1: Comparativa de metodologías de desarrollo de software.

Los aparatados de la tabla 5.1 se describen a continuación:

1. **Flexibilidad:** Indica la capacidad de la metodología para adaptarse a cambios durante el proceso de desarrollo. Por ejemplo, las metodologías Ágil y DevOps tienen una flexibilidad muy alta, lo que significa que pueden adaptarse rápidamente a los cambios.

2. **Gestión de Riesgos:** Se refiere a la capacidad de la metodología para identificar, evaluar y mitigar riesgos durante el desarrollo del proyecto. El Modelo en Espiral tiene una gestión de riesgos muy alta, lo que significa que se enfoca mucho en la identificación y mitigación de riesgos.

3. **Coste:** Representa el costo asociado con la implementación de la metodología. Puede variar desde bajo hasta alto. Por ejemplo, el Modelo en Espiral tiene un coste alto debido a su enfoque en la gestión de riesgos y la iteración.

4. **Tiempo:** Indica la duración del proceso de desarrollo. Puede variar desde corto hasta largo. Por ejemplo, el Modelo en Cascada tiene un tiempo largo debido a su enfoque secuencial.

5. **Adaptabilidad:** Se refiere a la capacidad de la metodología para adaptarse a diferentes tipos de proyectos y entornos. Las metodologías Ágil y DevOps tienen una adaptabilidad muy alta, lo que significa que pueden ser aplicadas en una amplia variedad de contextos.

6. **Documentación:** Indica el nivel de documentación requerido por la metodología. Por ejemplo, la metodología Waterfall requiere una alta cantidad de documentación, mientras que Ágil y DevOps requieren menos documentación.

En conclusión, la elección de una metodología de desarrollo de software dependerá de las características del proyecto, el equipo de desarrollo y las necesidades del cliente.

Capítulo 6

Propuesta

6.1. Explicación de la propuesta

La propuesta consiste en el desarrollo de una aplicación móvil destinada a la gestión del hogar compartido, facilitando la organización de tareas domésticas y la administración de gastos comunes entre los miembros del hogar. La aplicación permitirá a los usuarios crear un hogar compartido y realizar funciones como la creación y asignación de tareas, la gestión de listas de compra y el seguimientos de gastos compartidos, ofreciendo una solución centralizada y colaborativa para la convivencia en grupo.

Después de hacer un análisis de todas las metodologías de desarrollo en el Capitulo 5 he decidido usar la metodología iterativa, ya que permite una mayor flexibilidad y adaptabilidad a lo largo del desarrollo. Dado que los requisitos del proyecto pueden cambiar y evolucionar con el tiempo, esta metodología me permitirá revisar, probar y ajustar el software en cada iteración hasta alcanzar el resultado deseado. Además, la posibilidad de realizar ajustes continuos y obtener retroalimentación en cada ciclo de desarrollo facilitará la mejora constante del proyecto.

6.2. Análisis de la aplicación

6.2.1. Especificación de requisitos

La especificación de requisitos es una etapa clave en el desarrollo de un proyecto, ya que en ella se definen las funcionalidades y condiciones que debe cumplir el sistema. Su objetivo principal es establecer de manera clara y estructurada las expectativas del producto final.

En esta sección se identifican y detallan los requisitos, tomando en cuen-

ta las necesidades y restricciones detalladas por los usuarios y demás partes interesadas. Para ello, es fundamental realizar un análisis que permita comprender qué características son esenciales para el correcto funcionamiento del software.

Cada requisito contará con un código único que permitirá clasificarlo y referenciarlo fácilmente a lo largo del documento. Este método de organización facilita su seguimiento y garantiza que todas las especificaciones sean consideradas durante el desarrollo del proyecto.

6.2.1.1. Requisitos funcionales

Los requisitos funcionales son especificaciones detalladas que describen las funciones y comportamientos que un sistema debe cumplir para satisfacer las necesidades del usuario. Estos requisitos definen las acciones que el sistema debe ser capaz de realizar, incluyendo las interacciones con el usuario y las respuestas a entradas específicas. Estos requisitos son ensenciales para guiar el desarrollo del sistema y asegurar que cumpla con las expectivas del usuario. (PMO Informática, 2017)

Vamos a clasificar estos requisitos en diferentes grupos según su funcionalidad.

- **Grupo 1:** Gestión de sesión. En este grupo se incluyen los requisitos relacionados con la autenticación y la gestión de la sesión del usuario.
- **Grupo 2:** Gestión del hogar. En este grupo se incluyen los requisitos relacionados con la creación y gestión de un hogar compartido.
- **Grupo 3:** Gestión de tareas. En este grupo se incluyen los requisitos relacionados con la creación y asignación de tareas en el hogar.
- **Grupo 4:** Gestión de gastos. En este grupo se incluyen los requisitos relacionados con la creación y gestión de gastos en el hogar.
- **Grupo 5:** Gestión de compra. En este grupo se incluyen los requisitos relacionados con la creación y gestión de listas de compra.

A continuación, se detallan los requisitos funcionales de cada grupo.

Grupo 1: Gestión de sesión

RF_01	**Registro**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario registrarse.
Criterios de aceptación	1. La pantalla de registro debe pedir el nombre, apellidos, correo electrónico y contraseña. 2. Verificar que toda la información esté completa. 3. Comprobar que la información proporcionada sea correcta. 4. Si hay algún error, mostrar un mensaje de advertencia. 5. Tras el registro, redirigir al usuario a la pantalla de inicio de sesión.

Cuadro 6.1: RF_01

RF_02	**Inicio de sesión**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario iniciar sesión.
Criterios de aceptación	1. La pantalla de inicio de sesión debe pedir el correo electrónico y contraseña. 2. Verificar que toda la información esté completa. 3. Comprobar que la información proporcionada sea correcta. 4. Si hay algún error, mostrar un mensaje de advertencia. 5. Tras el inicio de sesión, se redirigirá a la pantalla principal.

Cuadro 6.2: RF_02

RF_03	**Inicio de sesión con Google**
Prioridad	Baja
Descripción	El sistema debe permitir al usuario iniciar sesión con Google
Criterios de aceptación	1. La pantalla de inicio de sesión debe permitir iniciar sesión con Google. 2. Si el usuario ya ha iniciado sesión con Google, se redirigirá a la pantalla principal de la aplicación. 3. Si el usuario no ha iniciado sesión con Google, se registrará automáticamente y se redirigirá a la pantalla principal de la aplicación.

Cuadro 6.3: RF_03

RF_04	**Cerrar Sesión**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario cerrar sesión
Criterios de aceptación	1. Debe exitir una opción en la interfaz para cerrar sesión. 2. Tras cerrar sesión, se redirigirá a la pantalla de bienvenida.

Cuadro 6.4: RF_04

RF_05	**Ver datos de usuario**
Prioridad	Media
Descripción	El sistema debe permitir al usuario ver sus datos personales.
Criterios de aceptación	1. Debe existir una sección en la interfaz para ver los datos personales del usuario. 2. El sistema debe mostrar la información del usuario

Cuadro 6.5: RF_05

RF_06	**Editar datos de usuario**
Prioridad	Media
Descripción	El sistema debe permitir al usuario editar sus datos personales.
Criterios de aceptación	1. Debe existir una opción en la interfaz para editar los datos personales del usuario. 2. Verificar que los campos obligatorios no estén vacíos y que la información sea válida. 3. Tras la edición de los datos, se actualizará la información en la base de datos.

Cuadro 6.6: RF_06

Grupo 2: Gestión del hogar

RF_07	**Crear hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear un hogar compartido.
Criterios de aceptación	1. Debe exitir una opción en la interfaz para crear un hogar. 2. El sistema debe pedir el nombre del hogar. 3. Tras la creación del hogar, se redirigirá a la pantalla principal del hogar.

Cuadro 6.7: RF_07

RF_08	**Unirse a un hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario unirse a un hogar compartido.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una opción para unirse a un hogar. 2. El sistema debe pedir el código del hogar. 3. Verificar que el código del hogar sea válido. 4. Si la información no es válida , mostrar un mensaje de error. 5. Tras unirse al hogar, se redirigirá a la pantalla principal del hogar.

Cuadro 6.8: RF_08

RF_09	**Abandonar hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario abandonar un hogar compartido.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una opción para abandonar el hogar. 2. Si el usuario confirma, abandonará el hogar y se redirigirá a la pantalla principal de la aplicación.

Cuadro 6.9: RF_09

RF_10	**Ver datos del hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver los datos del hogar.
Criterios de aceptación	1. Debe existir una sección en la interfaz para ver los datos del hogar. 2. Al acceder a la sección, se mostrará la información del hogar.

Cuadro 6.10: RF_10

RF_11	**Editar hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario editar los datos del hogar.
Criterios de aceptación	1. Debe existir una opción en la interfaz para editar los datos del hogar. 2. Verificar que los campos obligatorios no estén vacios y que la información sea válida. 3. Si los campos no son válidos, mostrar un mensaje de error. 4. Tras la edición de los datos, se actualizará la información en la base de datos.

Cuadro 6.11: RF_11

Grupo 3: Gestión de tareas

RF_12	**Crear tarea**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear una tarea.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una opción para crear una tarea. 2. El sistema nos pedirá la información de la tarea. 3. Verificar que los campos obligatorios no estén vacíos y que la información ssea válida. 4. Si la información no es válida, mostrar un mensaje de error.

Cuadro 6.12: RF_12

RF_13	**Consultar planificación semanal**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder consultar las tareas asignadas por semanas.
Criterios de aceptación	1. Debe existir una sección en la interfaz para consultar la planificación semanal. 2. Al acceder a la sección, se mostrarán las tareas asignadas por semanas. 3. Si no hay tareas asignadas, mostrar un mensaje indicando que no hay tareas asignadas.

Cuadro 6.13: RF_13

RF_14	**Marcar tarea como realizada**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder marcar una tarea como realizada.
Criterios de aceptación	1. Debe existir una opción en la interfaz para marcar una tarea como realizada. 2. Si el usuario confirma, la tarea se marcará como realizada y se actualizará la información en la base de datos.

Cuadro 6.14: RF_14

RF_15	**Editar Tarea**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder modificar una tarea.
Criterios de aceptación	1. Debe existir una opción en la interfaz para modificar una tarea. 2. Verificar que los campos obligatorios no estén vacíos y que la información sea válida. 3. Si los campos no son válidos, mostrar un mensaje de error. 4. Tras la modificación de la tarea, se actualizará la información en la base de datos.

Cuadro 6.15: RF_15

RF_16	Eliminar Tarea
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder eliminar una tarea.
Criterios de aceptación	1. Debe existir una opción en la interfaz para eliminar una tarea. 2. Si el usuario confirma, la tarea se eliminará y se actualizará la información en la base de datos.

Cuadro 6.16: RF_16

RF_17	Ver datos de una tarea
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver los datos de una tarea.
Criterios de aceptación	1. Debe existir una sección en la interfaz para ver los datos de una tarea. 2. Al acceder a la sección, se mostrarán los datos de la tarea.

Cuadro 6.17: RF_17

RF_18	Ver calendario de tareas
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver el calendario de tareas.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver el calendario de tareas. 2. Al acceder a la sección, se mostrará un calendario con las tareas asignadas.

Cuadro 6.18: RF_18

Grupo 4: Gestión de gastos

RF_19	**Crear gasto**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear un gasto.
Criterios de aceptación	1. Debe existir una opción en la interfaz para crear un gasto. 2. El sistema debe pedir la información del gasto. 3. Verificar que los campos obligatorios no estén vacíos y que la información sea válida. 4. El sistema debe recalcular las deudas pendientes.

Cuadro 6.19: RF_19

RF_20	**Editar gasto**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario modificar un gasto.
Criterios de aceptación	1. Deber existir una opción en la interfaz para modificar un gasto. 2. El sistema debe pedir la información del gasto. 3. Verificar que los campos obligatorios no estén vacíos y que la información sea válida. 4. Tras la modificación del gasto, se actualizará la información en la base de datos. 5. El sistema debe recalcular las deudas pendientes.

Cuadro 6.20: RF_20

RF_21	**Eliminar gasto**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario eliminar un gasto.
Criterios de aceptación	1. Debe existir una opción en la interfaz para eliminar un gasto. 2. Si el usuario confirma, el gasto se eliminará y se actualizará la información en la base de datos. 3. El sistema debe recalcular las deudas pendientes.

Cuadro 6.21: RF_21

RF_22	**Saldar deuda**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario saldar una deuda.
Criterios de aceptación	1. Debe existir una opción en la interfaz para saldar una deuda. 2. Si el usuario confirma, la deuda se marcará como saldada y se actualizará la información en la base de datos.

Cuadro 6.22: RF_22

RF_23	**Ver deudas**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver las deudas pendientes.
Criterios de aceptación	1. Debe existir una sección en la interfaz para ver las deudas pendientes. 2. Al acceder a la sección, se mostrarán las deudas pendientes. 3. Si no hay deudas pendientes, mostrar un mensaje indicando que no hay deudas pendientes.

Cuadro 6.23: RF_23

RF_24	**Ver detalles de un gasto**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver los detalles de un gasto.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver los detalles de un gasto. 2. Al acceder a la sección, se mostrarán los detalles del gasto. 3. Si no hay datos disponibles, mostrar un mensaje indicando que no hay datos disponibles.

Cuadro 6.24: RF_24

Grupo 5: Gestión de compra

RF_25	**Crear lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear una lista de la compra.
Criterios de aceptación	1. Debe existir una opción en la interfaz para crear una lista de la compra. 2. El sistema debe pedir el nombre de la lista de la compra.

Cuadro 6.25: RF_25

RF_26	**Añadir producto a la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario añadir un producto a la lista de la compra.
Criterios de aceptación	1. Debe existir una opción en la interfaz para añadir un producto a la lista de la compra. 2. El sistema debe la información del producto. 3. Verificar que los campos obligatorios no estén vacíos. 4. Tras añadir el producto, se actualizará la lista de la compra.

Cuadro 6.26: RF_26

RF_27	**Eliminar producto de la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario eliminar un producto de la lista de la compra.
Criterios de aceptación	1. Debe existir una opción en la interfaz para eliminar un producto de la lista de la compra. 2. Si el usuario confirma, el producto se eliminará de la lista de la compra y se actualizará la información en la base de datos.

Cuadro 6.27: RF_27

RF_28	**Eliminar lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario eliminar una lista de la compra.
Criterios de aceptación	1. Debe existir una opción en la interfaz para eliminar una lista de la compra. 2. Si el usuario confirma, la lista de la compra se eliminará y los productos asociados a ella y se actualizará la información en la base de datos.

Cuadro 6.28: RF_28

RF_29	**Ver productos de la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver los productos de una lista de la compra.
Criterios de aceptación	1. Debe existir una sección en la interfaz para ver los productos de la lista de la compra. 2. Al acceder a la sección, se mostrarán los productos de la lista de la compra. 3. Si no hay productos en la lista, mostrar un mensaje indicando que no hay productos en la lista.

Cuadro 6.29: RF_29

RF_30	**Editar producto de la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario editar un producto de la lista de la compra.
Criterios de aceptación	1. Debe existir una opción en la interfaz para editar un producto de la lista de la compra. 2. Verificar que los campos obligatorios no estén vacíos y que la información sea válida. 3. Tras la edición del producto, se actualizará la lista de la compra.

Cuadro 6.30: RF_30

6.2.1.2. Requisitos no funcionales

Los **requisitos no funcionales** son aquellos que describen las características y restricciones del sistema, pero no están directamentes relacionadas con sus funcionalidades. En lugar de especificar qué hace el sistema, establecen cómo debe hacerlo. Estos requisitos garantizan la calidad, el rendimiento, la seguridad, la usabilidad y otros aspectos clave del software. (Sommerville, 2015).

RNF_01	**Seguridad**
Prioridad	Alta
Descripción	El sistema protegerá los datos de los usuarios mediante autenticación con Firebase Authentication, reglas de seguridad en Firestore y cifrado en la comunicación de datos.

Cuadro 6.31: RNF_01

RNF_02	**Usabilidad**
Prioridad	Alta
Descripción	El sistema debe ser fácil de usar y comprender para los usuarios. Para ello, se implementará una interfaz intuitiva con una navegación clara y un diseño consistente. Además, se realizarán pruebas con usuarios reales para identificar mejoras en la experiencia de uso.

Cuadro 6.32: RNF_02

RNF_03	**Rendimiento**
Prioridad	Bajo
Descripción	El sistema optimizará el rendimiento mediante consultas eficientes en Firestore y una interfaz ligera para mejorar la velocidad de respuesta.

Cuadro 6.33: RNF_03

RNF_04	**Compatibilidad**
Prioridad	Media
Descripción	El software debe estar disponible para diferentes sistemas operativos (iOS, Android).

Cuadro 6.34: RNF_04

RNF_05	Accesibilidad
Prioridad	Baja
Descripción	El sistema implementará etiquetas accesibles y un diseño con buen contraste para mejorar la usabilidad para personas con discapacidades.

Cuadro 6.35: RNF_05

6.2.2. Modelos de casos de uso

Los modelos de de casos de uso nos permiten representar las iteracciones entre los usuarios y el sistema de forma clara. En este apartado, se presentan los casos de uso de la aplicación que reflejan las principales funcionalidades que la aplicación va a ofrecer.

Cada caso de uso describe una funcionalidad específica, estableciendo el actor involucrado. En este caso, el único actor es el usuario que es quien utiliza la aplicación.

6.2.2.1. Diagrama de casos de uso

A continuación, se muestran los diagramas de casos de uso, que se han dividido en paquetes (cada paquete corresponde a un grupo de funcionales que ya establecimos en el apartado de requisitos funcionales) y así poder representar de forma más clara las funcionalidades de la aplicación.

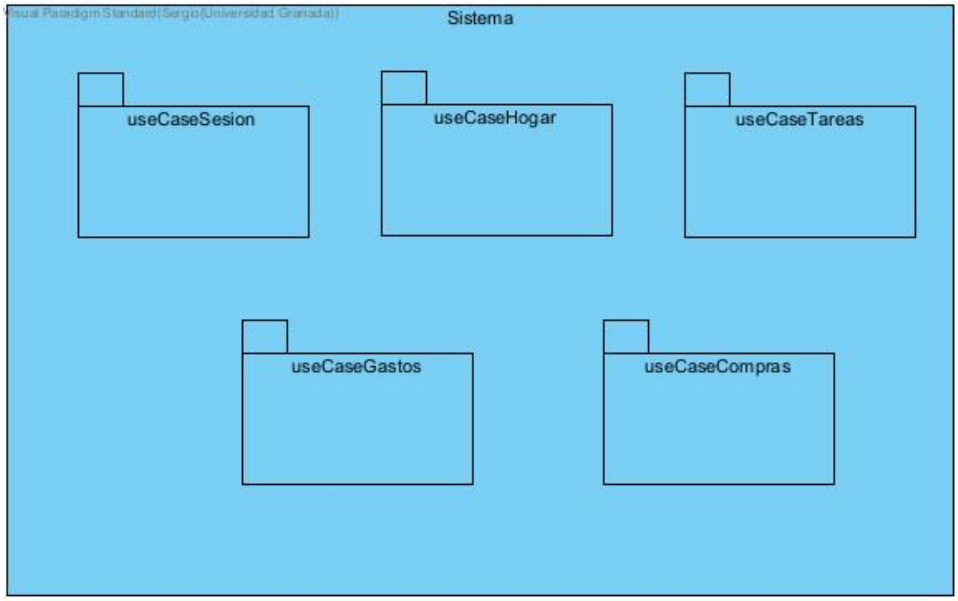


Figura 6.1: Diagrama de paquetes de casos de uso

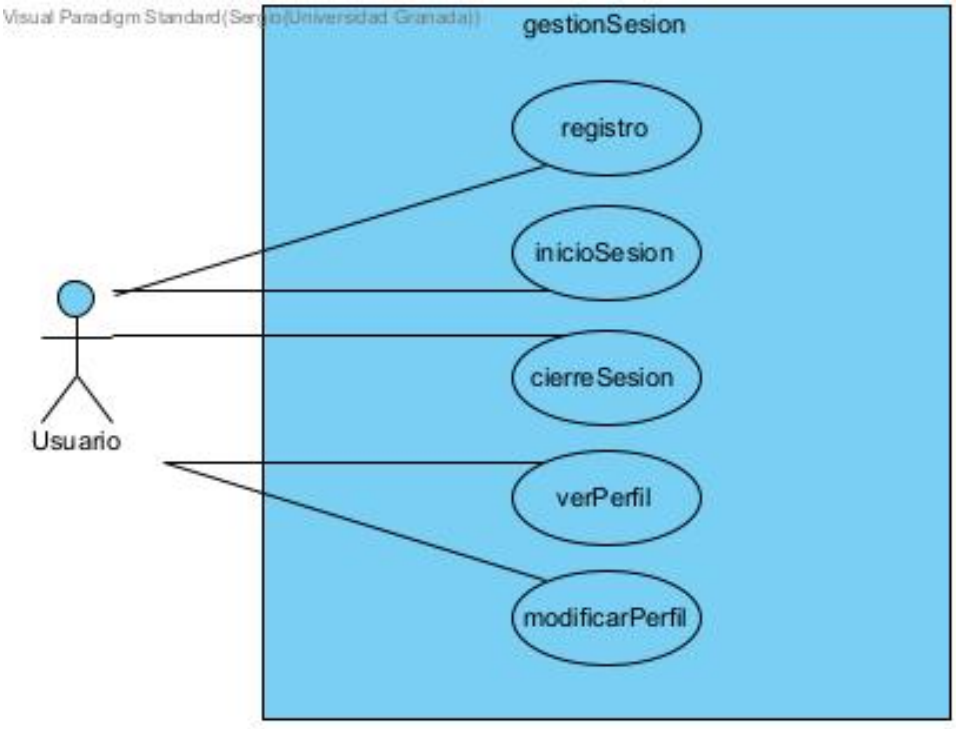


Figura 6.2: Diagrama de casos de uso de la gestión de sesión

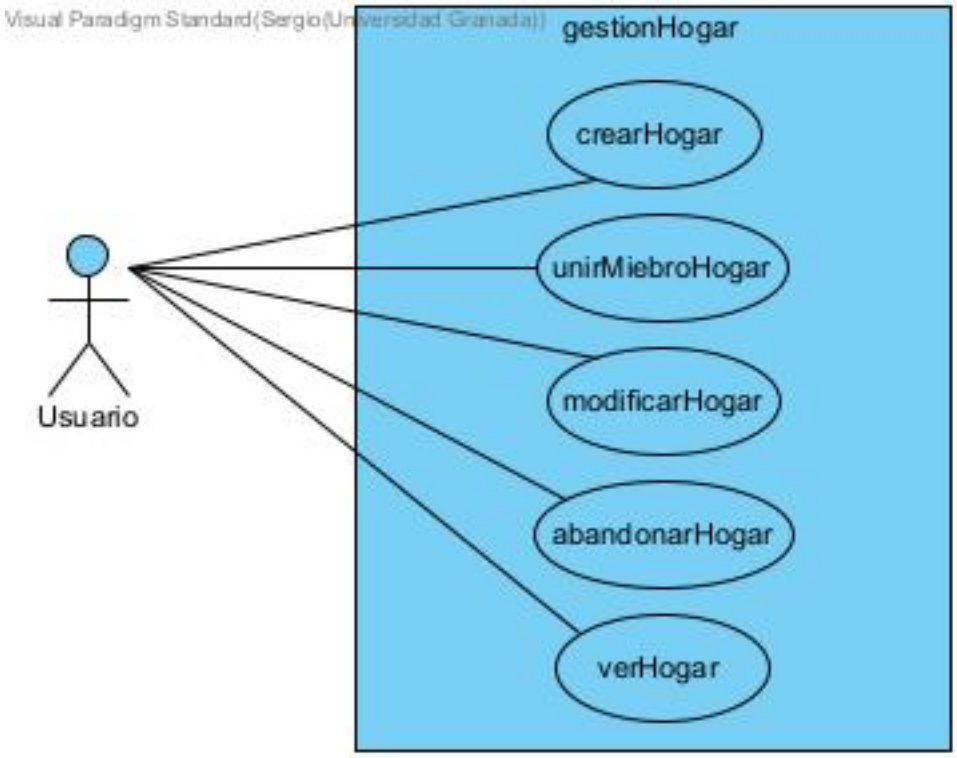


Figura 6.3: Diagrama de casos de uso de la gestion de hogar

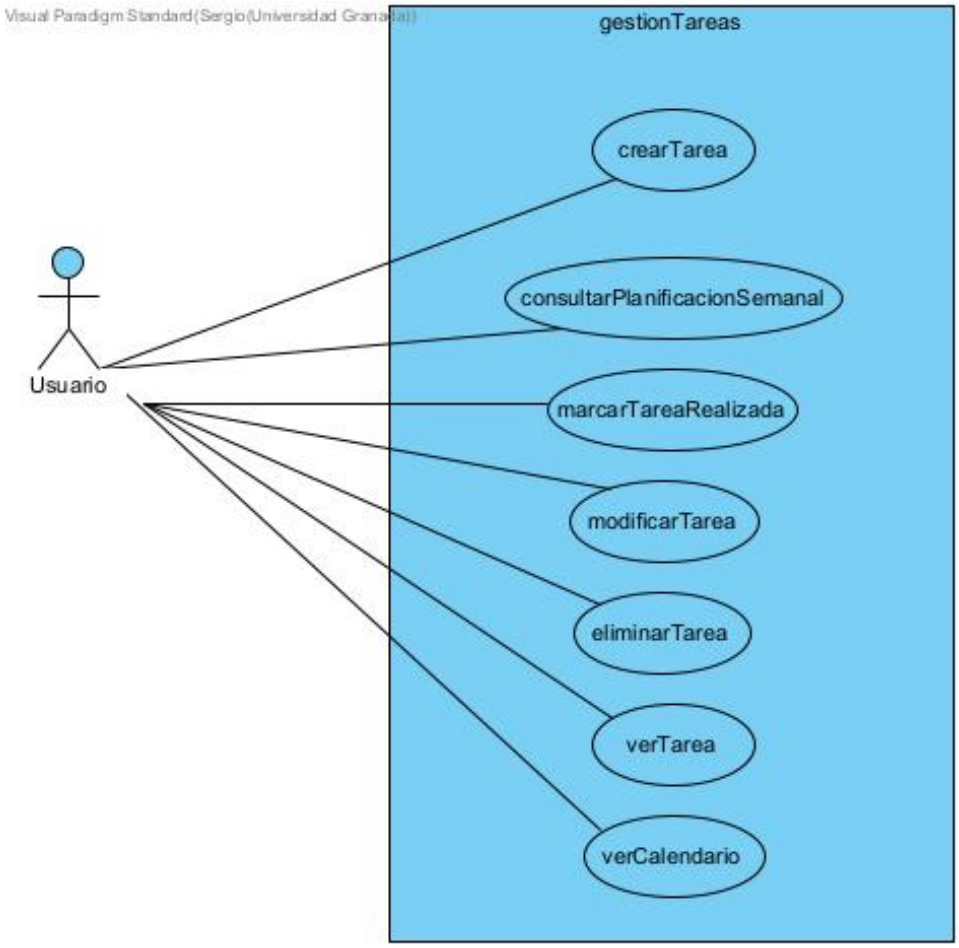


Figura 6.4: Diagrama de casos de uso de la gestion de tareas

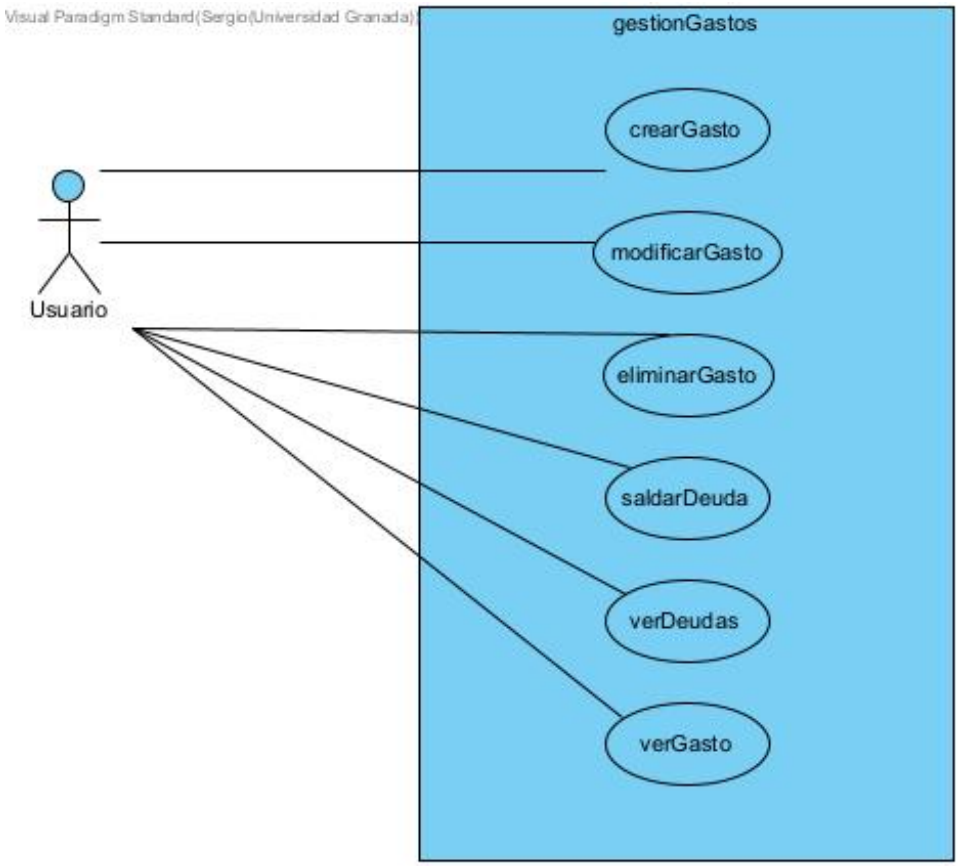


Figura 6.5: Diagrama de casos de uso de la gestion de gastos

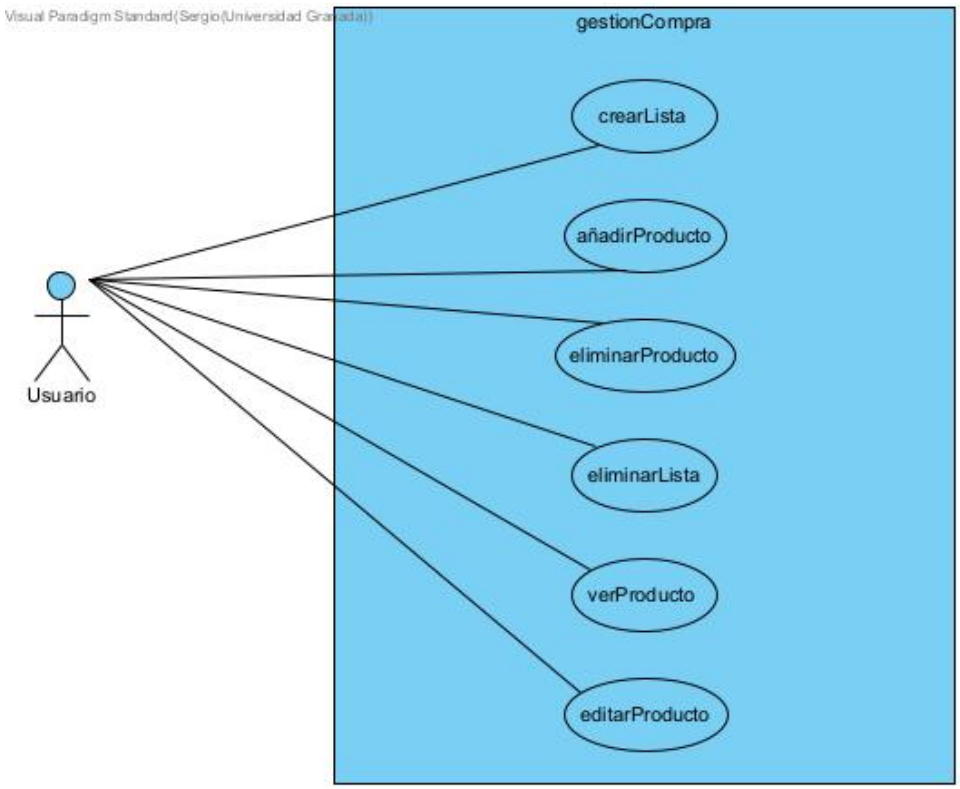


Figura 6.6: Diagrama de casos de uso de la gestion de compra

6.2.2.2. Descripción de los casos de uso

En esta sección se describen los casos de uso de la aplicación, indicando los actores involucrados, la descripción y los pasos a seguir para llevar a cabo la funcionalidad.

Caso de Uso	registro		CU1
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_01		
Precondición	El usuario no debe estar registrado en el sistema.		
Postcondición	El usuario queda registrado en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crearse una cuenta en el sistema.
Resumen	El usuario introduce los datos necesarios para crear una cuenta.

Curso normal			
1.	Usuario: Introduce su nombre, apellidos, correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el registro.	4.	El sistema verifica que el correo electrónico no esté registrado previamente.
5.	Usuario: Recibe un mensaje indicando que debe verificar su correo electrónico.		
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico ya está registrado.		
2c.	El sistema muestra un mensaje de error.		

Cuadro 6.36: Caso de uso - Registro

Caso de Uso	iniciarSesion		CU2
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_02		
Precondición	El usuario debe estar registrado en el sistema.		
Postcondición	El usuario accede al sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de iniciar sesión en el sistema.
Resumen	El usuario introduce su correo electrónico y contraseña para autenticarse.

Curso normal			
1.	Usuario: Introduce su correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el inicio de sesión.	4.	El sistema verifica que el correo electrónico y la contraseña sean correctos.
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico o la contraseña son incorrectos.		
2c.	El sistema muestra un mensaje de error.		

Cuadro 6.37: Caso de uso - Iniciar Sesión

Caso de Uso	cerrarSesion		CU3
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_04		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario cierra su sesión en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de cerrar su sesión en el sistema.
Resumen	El usuario cierra su sesión para finalizar su uso del sistema.

Curso normal			
1.	Usuario: Accede a la opción de cerrar sesión.	2.	El sistema muestra un mensaje de confirmación.
3.	Usuario: Confirma que desea cerrar sesión.	4.	El sistema cierra la sesión del usuario.

Cuadro 6.38: Caso de uso - Cerrar Sesión

La explicación de los casos de uso restantes se encuentra en el Apéndice A.

6.2.3. Modelo conceptual

A continuación, se muestra el modelo conceptual de la aplicación, que representa las entidades, las relaciones entre ellas y que información almacenan.

En este diagramas se pueden observar las entidades principales de la aplicación, que son:

- **Usuario**: Representa a los usuarios que va a usar la aplicación. De cada usuario se almacena información como el nombre, apellidos y correo electrónico.
- **Hogar**: Representa a los hogares de los usuarios. Almacena información como el nombre del hogar, la lista de usuarios que pertenecen a él y un código identificativo.

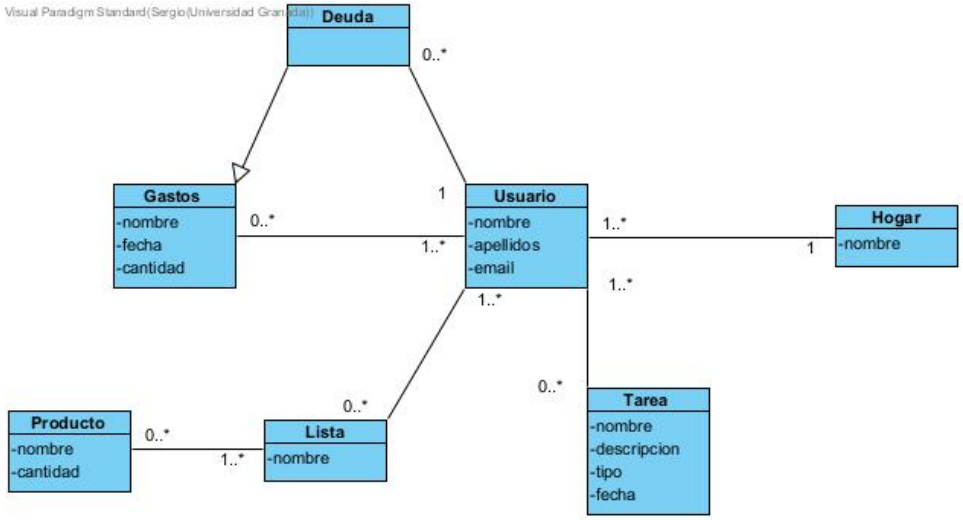


Figura 6.7: Modelo conceptual

- **Tarea**: Representa las tareas que pueden crear los usuarios. Almacena información relevante para la gestión de la misma.
- **Gasto**: Representa los gastos que se pueden generar entre los miembros del hogar o de forma individual. Almacena información como el nombre del gasto, la cantidad y la fecha de creación.
- **Deuda**: Representan las deudas que se generan a través de los gastos. Contiene información como quien debe dinero a quién y la cantidad.
- **Lista de la compra**: Representa las listas de la compra que se pueden crear. Almacena información como el nombre de la lista y la lista de productos asociados a ella.
- **Producto**: Representa los productos que se pueden añadir a una lista de la compra. Almacena información como el nombre del producto y la cantidad.

También se pueden observar las relaciones entre las entidades :

- **Usuario-Hogar:** Un usuario tiene que pertenecer a uno y solo un hogar. Un hogar puede tener varios usuarios.
- **Usuario-Tarea:** Un usuario puede crear y asignar varias tareas. Una tarea puede pertencer a uno o varios usuarios (como por ejemplo las tareas semanales).
- **Usuario-Lista:** Un usuario puede crear varias listas de la compra. Una lista de la compra puede pertenecer a uno o varios usuarios.

- **Usuario-Gasto:** Un usuario puede crear varios gastos. Un gasto puede pertenecer a uno o varios usuarios.
- **Usuario-Deuda:** Un usuario puede tener varias deudas. Una deuda puede pertenecer a uno o varios usuarios.
- **Lista-Producto:** Una lista de la compra puede tener varios productos. Un producto pertenece a una lista.
- **Gasto-Deuda:** Un gasto puede generar varias deudas. Una deuda se genera a partir de un gasto.

6.3. Diseño

6.3.1. Diseño de la arquitectura

La arquitectura de este sistema se basa en un modelo **cliente-servidor**, donde el usuario interactúa únicamente con la interfaz gráfica de la aplicación (cliente), la cual se comunica con la base de datos *Firebase Firestore* para obtener y almacenar la información. La interfaz gráfica se interactua directamente con la base de datos, sin necesidad de un servidor intermedio. Esta estructura simplifica el desarrolo, al delegar en Firebase la gestión de la base de datos y la sincronización en tiempo real.

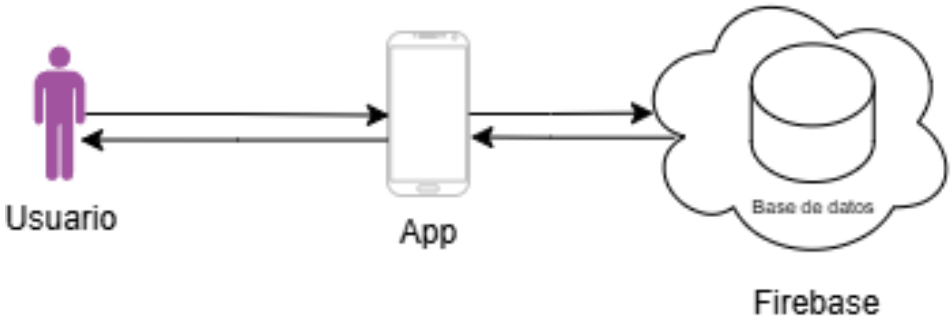


Figura 6.8: Diagrama de la arquitectura del sistema

6.3.1.1. Componentes Principales

- **Usuario:** El usuario es el componente principal de la aplicación, ya que es el que interactúa con la interfaz gráfica y realiza las acciones.
- **App (Interfaz gráfica):** La interfaz gráfica es la parte de la aplicación con la que el usuario interactúa. En este caso, la interfaz gráfica

se ha diseñado para ser intuitiva y fácil de usar, con el objetivo de que el usuario pueda realizar las tareas de forma rápida y sencilla.

- **Base de datos:** (Servidor) La base de datos es el componente encargado de almacenar y gestionar la información de la aplicación. En este caso, se ha utilizado Firebase Firestore como base de datos, ya que permite almacenar y sincronizar los datos en tiempo real, lo que facilita la gestión de la información.

6.3.1.2. Flujo de interacción

1. El usuario accede a la aplicación y realiza una acción.
2. La interfaz gráfica procesa la acción y envía la solicitud a la base de datos a través de las funciones de Firebase.
3. La base de datos recibe la solicitud, la procesa y devuelve la información necesaria
4. La interfaz gráfica recibe la información y la muestra al usuario.

6.3.2. Diseño de la interfaz gráfica

El diseño de la interfaz gráfica de la aplicación se ha centrado en crear un experiencia de usuario fluida, intuitiva y visualmente atractiva. A pesar de que la aplicación está orientada a un público joven, el objetivo es asegurar que cualquier usuario independientemente de su edad pueda navegar de forma cómoda y sencilla. La estética y la usabilidad son los pilares del diseño y se va a trabajar para que la aplicación sea moderna y fácil de usar.

La interfaz gráfica se va ha diseñar utilizando una paleta de colores coherente. El color principal es un verde menta (#4CAF97), un tono de verde fresco y tranquilo. Este color se va a utilizar en los elementos más importantes y de acción, como en los botones de “Registrar” e “Iniciar Sesión”. Además, se va a emplear en encabezados y otros elementos clave para dar coherencia al diseño y asegurar que la aplicación sea visualmente atractiva sin ser abrumadora.

Por otro lado, el color naranja cálido (#F4A261), se va a usar como color secundario. Este color se va a utilizar en elementos que se requieren resaltar sin competir con el verde. Se aplica en, icono y botones secundarios y así proporcionar un contraste con el color principal y asegurando que la aplicación mantenga un equilibrio visual adecuado.

Uno de los aspectos más destacados en el diseño de la interfaz es la tipográfia. Se va a utilizar la fuente *Poppins* de Google Fonts, una fuente

moderna que es altamente legible y adecuada para aplicaciones móviles. Tiene una estructura clara y amigable, lo que facilita la lectura incluso en dispositivos con las pantallas más pequeñas. La elección de esta tipografía no solo mejora la estética de la aplicación, sino que contribuye a la accesibilidad, ofreciendo una lectura cómoda para todos los usuarios.

El diseño de la interfaz sigue principios clave de usabilidad, los cuales son fundamentales para asegurar una experiencia fluida y agradable:

- **Simplicidad:** El diseño debe ser sencillo , evitando elementos innecesarios que puedan confundir al usuario.
- **Tamaño de los elementos:** Los elementos de la interfaz gráfica deben tener un tamaño adecuado para que el usuario pueda interactuar con ellos de forma cómoda.
- **Contraste:** Los colores y la tipografía deben tener un contraste adecuado para que el usuario pueda leer la información de forma clara.
- **Facilidad de navegación:** La interfaz gráfica debe ser fácil de navegar, permitiendo al usuario moverse de una pantalla a otra de forma rápida y sencilla.

A continuación, se van a exponer los bocetos másss importantes de la interfaz gráfica de la aplicación. Estos bocetos han sido creados con la herramienta de diseño *Figma* y representan las pantallas principales de la aplicación. Más adelante, dentro de la iteracción en la que se desarrollen, se presentarán algunos bocetos más que sean necesarios para completar la aplicación.

Los bocetos se han dividido en grupos, cada uno de los cuales representa una funcionalidad específica de la aplicación:

6.3.2.1. Bocetos de las interfaces de gráficas del Grupo 1: *Gestión de sesión*

En esta sección podemos ver todos los bocetos de las interfaces gráficas del Grupo 1: *Gestión de sesión.*



Figura 6.9: Diseño de la pantalla de inicio de sesión.

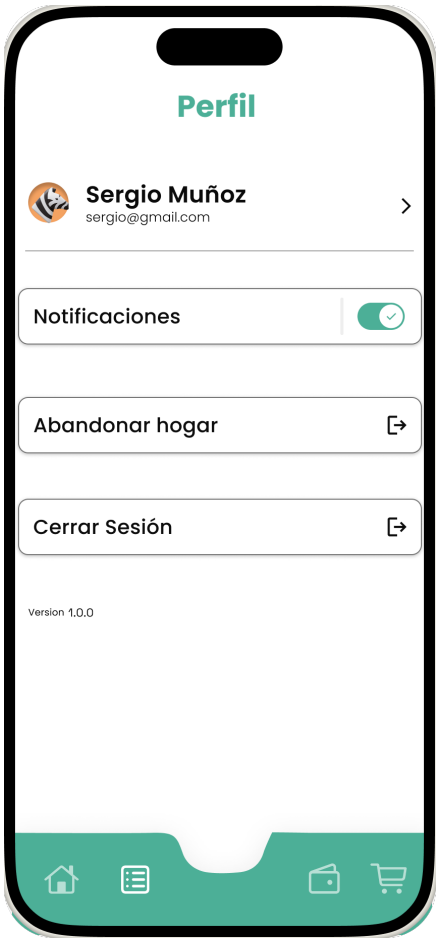


Figura 6.10: Diseño de la pantalla de cierre de sesión y datos de usuario.

6.3.2.2. Bocetos de las interfaces de gráficas del Grupo 2: *Gestión del hogar*



Figura 6.11: Diseño de la pantalla para unirte a un hogar o crear uno nuevo.



Figura 6.12: Diseño de la pantalla para unirte a un hogar.

6.3.2.3. Bocetos de las interfaces de gráficas del Grupo 3: *Gestión de tareas*

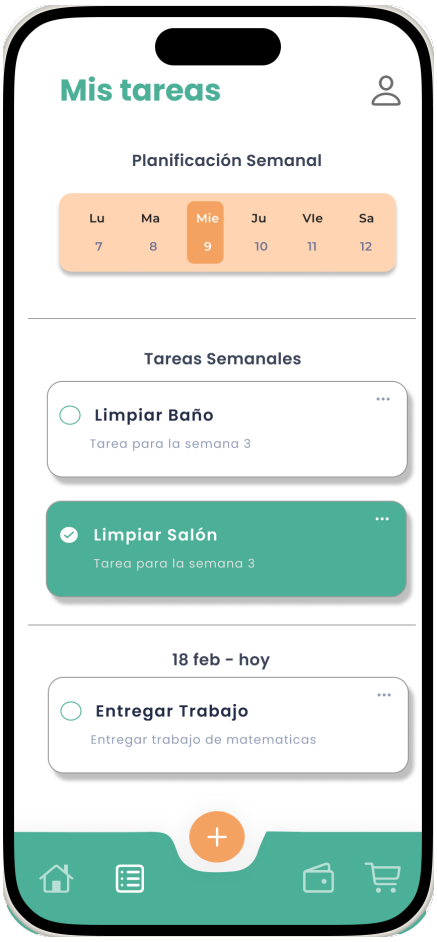


Figura 6.13: Diseño de la pantalla principal de tareas.

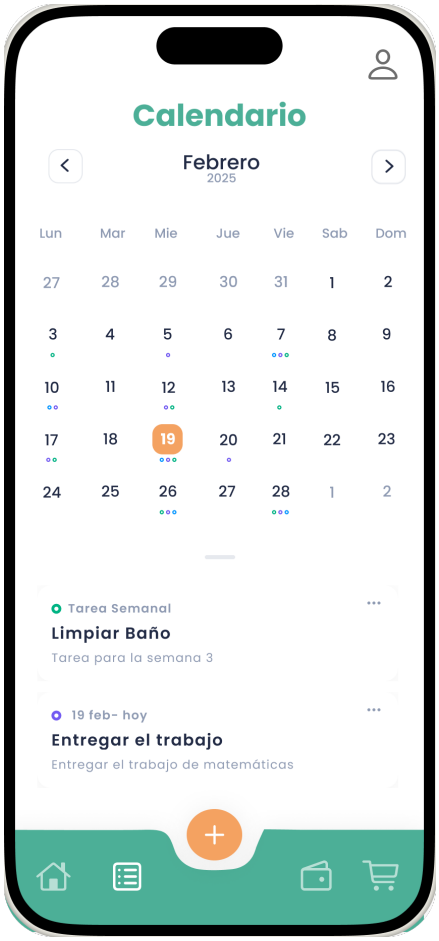


Figura 6.14: Diseño de la pantalla del calendario.

6.3.2.4. Bocetos de las interfaces de gráficas del Grupo 4: *Gestión de gastos*



Figura 6.15: Diseño de la pantalla principal de gastos.

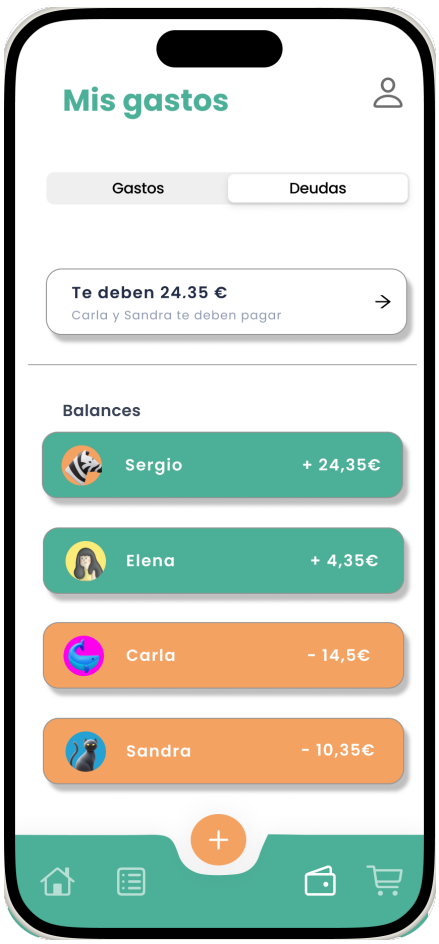


Figura 6.16: Diseño de la pantalla de deudas.

6.3.2.5. Bocetos de las interfaces de gráficas del Grupo 5: *Gestión de la compra*

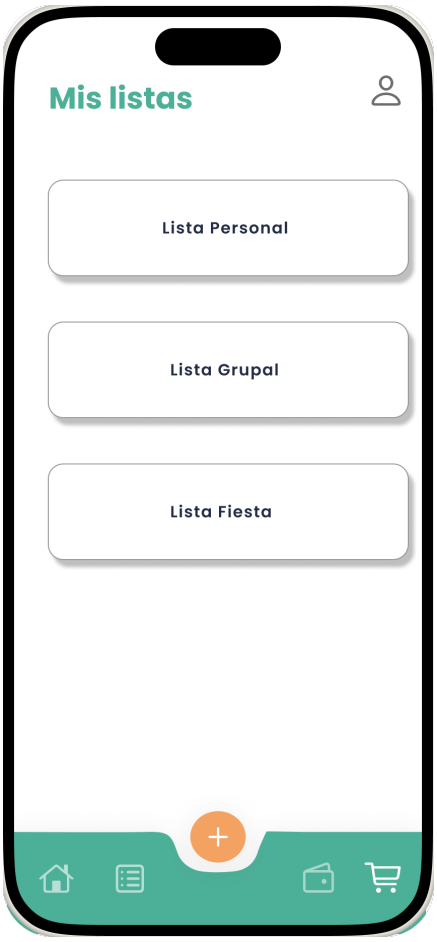


Figura 6.17: Diseño de la pantalla de la lista de la compra.

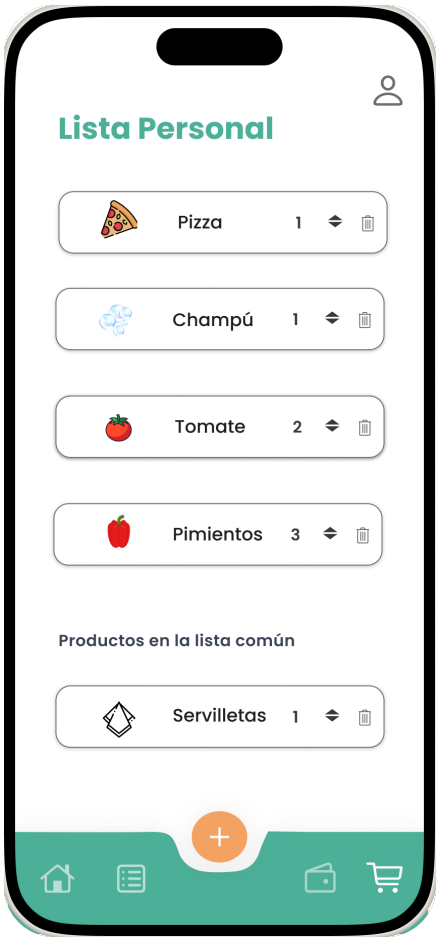


Figura 6.18: Diseño de la pantalla de productos.

Capítulo 7

Desarrollo e implementación de la aplicación

En este capítulo se va detallar el proceso de desarrollo de la aplicación, explicando las tecnologías y usadas y como se han estructurado las iteracciones para construir la aplicación de manera progresiva.

7.1. Elección de tecnologías

Tras analizar las diferentes tecnologías disponibles para el desarrollo de la aplicación en el Capítulo 4, he decidido utilizar **Flutter** como tecnología front-end porque presenta varias ventajas significativas frente al resto de alternativas. Una de las razones más destacadas es su capacidad de desarrollo multiplataforma, que permite crear una única base de código que se ejecuta tanto en Android, como en iOS, web y aplicaciones de escritorio, lo que ahorra tiempo y esfuerzo en comparación con desarrollar código nativo para cada plataforma por separado. Además, Flutter ofrece un rendimiento cercano al de las aplicaciones nativas, gracias a su motor gráfico Dart, que se compila en código nativo. Otra ventaja es el sistema de Hot Reload, que permite a los desarrolladores visualizar los cambios en tiempo real, sin tener que recompilar por completo. Tiene también una enorme capacidad de personalización, simplificando la tarea. Además, al estar respaldado por Google,, recibe soporte continuo.

En cuanto a la tecnología back-end, he decidido utilizar **Firebase** porque ofrece una serie de ventajas que lo hacen destacar frente a las otras alternativas. Para empezar, Firebase proporciona un backend completo, lo que significa que no es necesario gestionar servidores ni preocuparse por la configuración y el mantenimiento. La integración de la autenticación de usuarios es también uno de sus puntos fuertes, ya que Firebase Auth facilita

la implementación de los mismos. Otra razón para elegir Firebase es su base de datos en tiempo real, que permite sicronización instantánea de datos. Además, tiene una gran integración con servicios de Google, optimizando la monitorización del rendimiento.

7.2. Iteracciones de desarrollo

El desarrollo de la aplicación se ha estructurado en varias iteracciones, siguiendo una metología iterativa. En cada iteración se va a desarrollar una serie de funcionalidades, asociados a los requisitos especificados anteriormente. A continuación se detallan las iteracciones que se han llevado a cabo para el desarrollo de la aplicación:

7.2.1. Iteración 1

En esta primera iteracción se van a desarrollar los siguientes requisitos:

- RF_01: Registro.
- RF_02: Inicio de sesión.
- RF_03: Inicio de sesión con Google.
- RF_04: Cerrar sesión.
- RF_05: Ver datos de usuario.
- RF_06: Editar datos de usuario.
- RF_07: Crear hogar.
- RF_08: Unirse a un hogar.
- RF_09: Abandonar hogar.
- RF_10: Ver datos del hogar.
- RF_11: Editar hogar.

Antes de la implementación de los requisitos mencionados anteriomente es necesario realizar una configuración inicial del proyecto. Para ello, lo primero es configurar el entorno de desarrollo, asegurando que todas las herramieentas necesarias estén correctamente instaladas. Se utiliza Visual Studio Code como editor principal y se instala Flutter junto con las dependecias necesarias. También es necesario configurar un emulador para poder realizar las pruebas de la aplicación posteriormente.

Una vez configurado el entorno de desarrollo, se procede a la creación de un nuevo proyecto de Flutter. Se configura un sistema de control de versiones con Git y se sube el código a un repositorio en GitHub para llevar un seguimiento.

Otra de las tareas a realizar antes de empezar con la implementación de los requisitos es la configuración de Firebase. Para ello se crea un nuevo proyecto en Firebase y se configuran los servicios necesarios (como Firebase Auth para la autenticación de usuarios y Firebase Firestore para almacenar datos).

Por último antes de empezar con la implementación de los requisitos, se realizan pruebas de conexion entre la app y Firebase para confirmar que la base de datos está correctamente configurada y que la aplicación puede comunicarse con ella sin problemas.

Al usar Flutter como tecnología de desarrollo, se está cumpliendo el RNF_04 - Compatibilidad, ya que Flutter permite desarrollar aplicaciones multiplataforma que se ejecutan en Android e iOS. Además, Flutter también es compatible con aplicaciones web y de escritorio, lo que amplía aún más el alcance de la aplicación y asegura que pueda ser utilizada en una variedad de dispositivos y sistemas operativos. Esto no solo mejora la experiencia del usuario al permitirle acceder a la aplicación desde diferentes plataformas, sino que también optimiza los recursos de desarrollo al reducir la necesidad de mantener bases de código separadas para cada plataforma.

Firebase contribuye al cumplimiento del RNF_01 - Seguridad, gracias a las características de seguridad que ofrece. En primer lugar, Firebase Auth proporciona un sistema de autenticación de usuarios, permitiendo implementar métodos de inicio de sesión seguros, como autenticación mediante correo electrónico y contraseña, o autenticación con proveedores externos como Google.

Además, Firebase Firestore incluye reglas de seguridad que permiten definir políticas de acceso a los datos almacenados en la base de datos. Tambien utiliza conexiones seguras para la comunicación entre la aplicación y los servicios de backend, asegurando que los datos transmitidos estén cifrados y protegidos.

Para los datos relacionados con los usuarios y los hogares, se crearán colecciones en Firebase Firestore para almacenar la información de forma estructurada.

7.2.1.1. Implementacion

La implementación se ha realizado a partir de los bocetos iniciales presentados anteriormente, aunque ha hecho falta crear alguno más para detallar

mejor la interfaz de usuario a la hora editar los datos por tanto se han creado los siguientes bocetos:



Figura 7.1: Diseño de la pantalla de bienvenida.

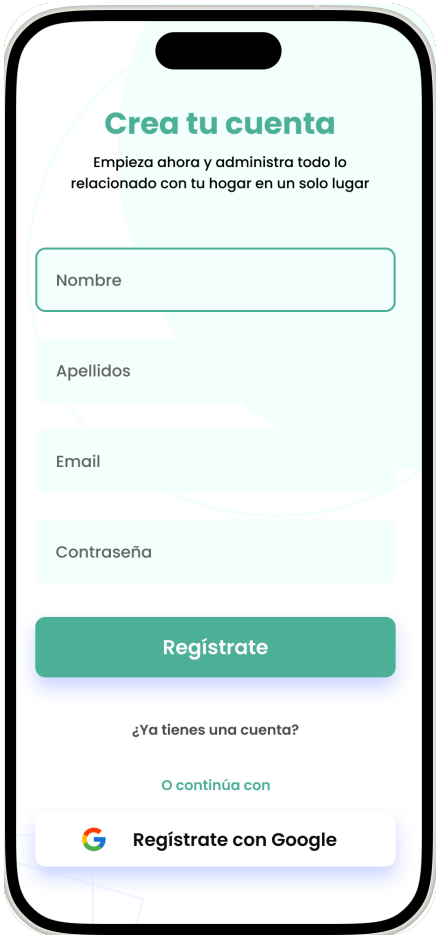


Figura 7.2: Diseño de la pantalla de registro.



Figura 7.3: Diseño de la pantalla de editar usuario.

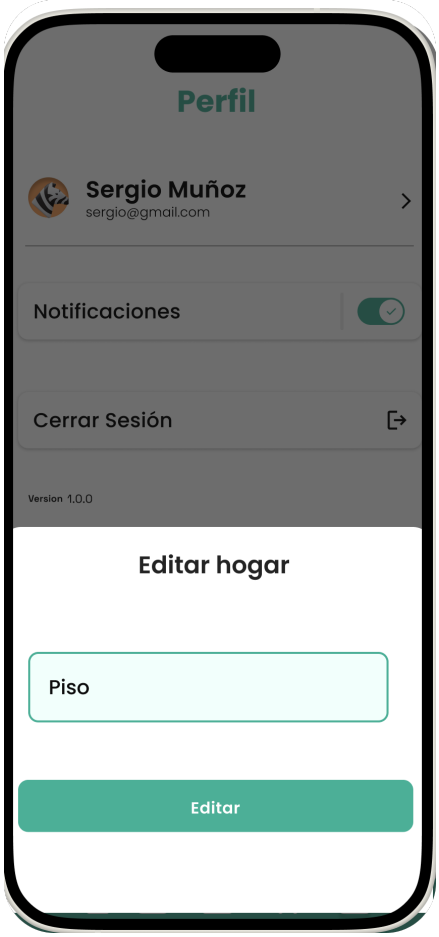


Figura 7.4: Diseño de la pantalla de editar hogar.

Una vez creados los bocetos, se ha procedido a la implementación de los requisitos. Para ello, se han creado las siguientes pantallas:

Pantalla de bienvenida

Esta pantalla se ha implementado con la clase *WelcomeScreen*, es la primera que ven los usuarios al abrir la aplicación. Su función principal es dar la bienvenida al usuario y ofrecerle dos opciones: registrarse o iniciar sesión.

No hay funciones muy interesantes en esta pantalla, ya que es bastante simple. Algunas funciones destacables son:

- **_buildTextContent :** Se encarga de mostrar los textos de bienvenida. Presentan el mensaje principal y la descripción.
- **_buildButtons :** Crea y anima los dos botones principales de la pantalla para que el usuario pueda registrarse o iniciar sesión.

A continuación se muestra el resultado de la implementación de la pantalla de bienvenida:



Figura 7.5: Interfaz gráfica pantalla bienvenida.

Pantalla de registro

Esta pantalla se ha implementado con la clase *RegistrationPage* que permite a los usuarios crear una nueva cuenta en la aplicación. Esta clase maneja todo el proceso de registro, tanto con email y contraseña como mediante Google.

Las funciones principales de esta clase son:

- **initState :** Este método se encarga de preparar las animaciones iniciales que hacen aparecer gradualmente el contenido cuando se abre la pantalla, creando una transición para el usuario.
- **_register :** Es la función encargada del registro con email y contraseña. Verfica primero que todos los campos estén correctamente completados, luego intenta crear la cuenta en Firebase. Si tiene éxito, guarda los datos adicionales del usuario y envía un correo de verificación.
- **_signInWithGoogle :** Gestiona el registro alternativo mediante una cuenta de Google. Inicia el proceso de autenticación con Google y si es un nuevo usuario crea automáticamente su perfil en la base de datos.
- **_createUserProfile :** Guarda la información del usuario (nombre y apellidos) en Firestore después de crear una cuenta exitosamente, asociándola al ID único.
- **_showRegistrationSuccessDialog :** Muestra un cuadro de diálogo cuando el registro se completa correctamente. Ofrece opciones para volver al inicio de sesión o reenviar el correo de verificación si es necesario.
- **_parseAuthError :** Traduce los códigos de error técnicos de Firebase en mensajes comprensibles para el usuario.

Aquí se muestra el resultado de la implementación de la pantalla de registro:

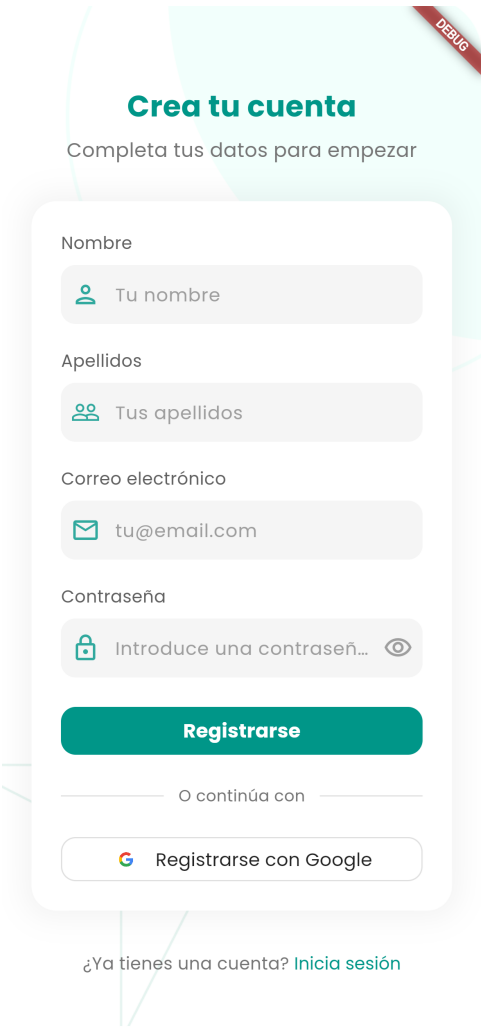


Figura 7.6: Interfaz gráfica pantalla de registro.

Pantalla de inicio de sesión

Esta pantalla se ha implementado con la clase *SignInEmail* que permite a los usuarios iniciar sesión en la aplicación. En esta pantalla, se puede iniciar sesión con email y contraseña o mediante Google.

Las funciones principales de esta clase son:

- **initState :** Este método se ejecuta al iniciar la pantalla. En ella se establece una animación que hace que el contenido aparezca de forma gradual cuando se abre la pantalla.
- **dispose :** Se encarga de liberar los recursos utilizados.

- **_signIn :** Este método se ejecuta cuando el usuario intenta iniciar sesión con su correo y contraseña. Verifica las credenciales con Firebase y luego decide a qué pantalla debe dirigirse según si el usuario pertenece o no a un hogar.
- **_signInWithGoogle :** Permite iniciar sesión mediante una cuenta de Google. Si el inicio de sesión tiene éxito, también verifica si el usuario está asociado a un hogar y lo redirige en consecuencia.
- **_handleAuth :** Es el método que decide a qué pantalla se envía al usuario después de una autenticación exitosa. Si su correo no está verificado, muestra un díalago informativo. Si ya está verificado, revisa si pertenece a un hogar para ver a qué pantalla se envía.
- **_handleAuthError :** Se encarga de interpretar y mostrar mensajes claros de error cuando el inicio de sesión falla.
- **_showEmailNotVerifiedDialog :** Muestra un cuadro de diálogo cuando el usuario intenta iniciar sesión sin hacer verificado su correo.

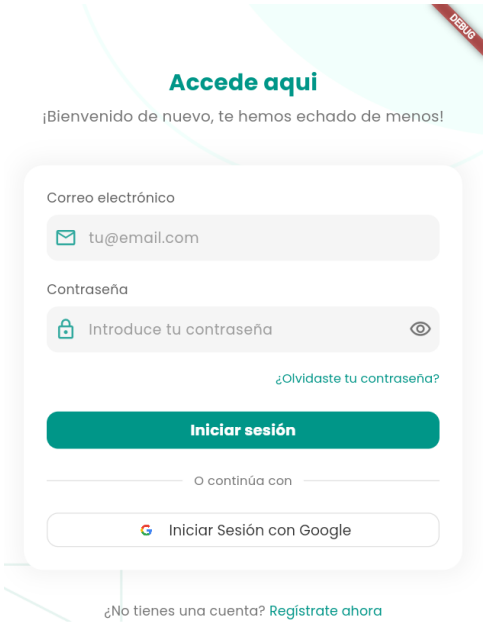


Figura 7.7: Interfaz gráfica pantalla de inicio de sesión.

Pantalla de ver datos de hogar y usuario

Esta pantalla se ha immplentado con la clase *ProfileScreen* que representa el perfil de usuario dentro de la aplicación. Muestra la información personal y del hogar al que pertenece el usuario. Su finalidad es mostrar la información personal y del hogar al que pertenece el usuario, así como permitirle editar sus datos, gestionar el nombre del hogar, ver a los miembros con los que convive, y realizar acciones como abandonar el hogar o cerrar sesión.

Entre sus funciones destacadas se encuentran:

- **initState :** Este método se ejecuta cuando se abre la pantalla. Inicializa el usuario actual y llama a otro método para cargar sus datos desde la base de datos.
- **_loadProfileData :** Carga la información del usuario desde Firebase, incluyendo su nombre, apellidos y correo. Luego busca el hogar al que pertenece y obtiene su nombre, código y los nombres de los demás miembros del hogar.
- **_updateProfile :** Permite guardar cambios en los datos del usuario, actualizandolos en la base de datos.
- **_showEditProfileModal :** Muestra un formulario en la parte inferior de la pantalla donde el usuario puede modificar sus datos.
- **_buildProfileHeader :** Construye la parte superior de la pantalla, donde se muestra el nombre y apellidos del usuario y su correo.
- **_buildProfileContent :** Genera el contenido del perfil debajo del encabezado. Incluye las secciones de configuración del hogar, preferencias y botones de accion.
- **_buidEditableHomeNameItem :** Muestra el nombre del hogar junto a un botón para editarlo.
- **_updateHomeName :** Guarda en la base de datos el nuevo nombre del hogar, si el usuario lo modifica.
- **_showEditHomeModal :** Abre un formulario donde se puede cambiar el nombre del hogar.
- **_buildSectionCard :** Crear una tarjeta que agrupa secciones dentro del perfil.
- **_buildInfoItem :** Muestra un ítem de información dentro de una sección.
- **_buildMembersList :** Muestra una lista con los nombres de los demás miembros del hogar.

- **_buildSwitchItem :** Crea un interruptor que permite activar o desactivar la opción de recibir notificaciones.
- **_buildActionItem :** Genera un ítem de acción dentro de una sección, como cerrar sesión o abandonar el hogar.
- **_confirmarAbandonoHogar :** Muestra un cuadro emergente que pregunta al usuario si realmente desea abandonar el hogar.
- **_confirmarCerrarSesión :** Muestra un cuadro emergente que pregunta al usuario si realmente desea cerrar sesión.
- **_abandonarHogar :** Elimina al usuario de la lista de miembros del hogar.

Aqui se muestra el resultado de la implementación de la pantalla:

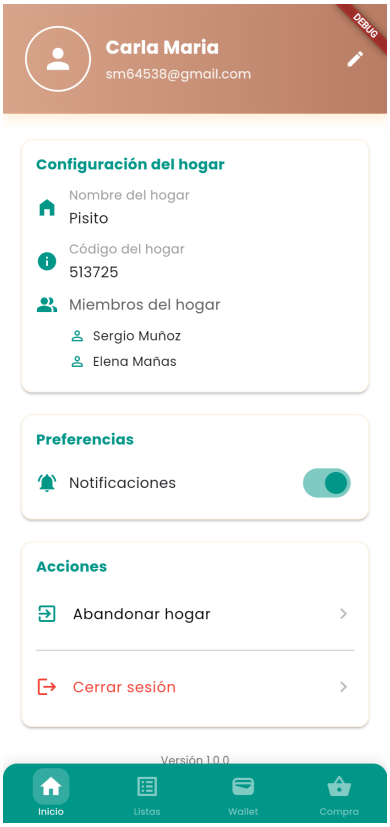


Figura 7.8: Interfaz gráfica pantalla de ver datos de hogar y usuario.

Pantalla de editar datos de usuario

Esta es la pantalla que se muestra cuando el usuario selecciona la opción de editar su perfil, con el método *_showEditProfileModal*, explicado anteriormente.

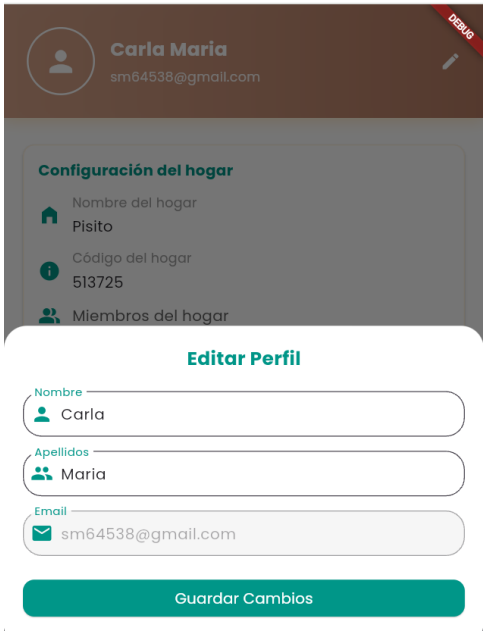


Figura 7.9: Interfaz gráfica pantalla de editar datos de usuario.

Pantalla de editar hogar

Esta es la pantalla que se muestra cuando el usuario selecciona la opción de editar el hogar, con el método *_showEditHomeModal*, explicado también anteriormente.

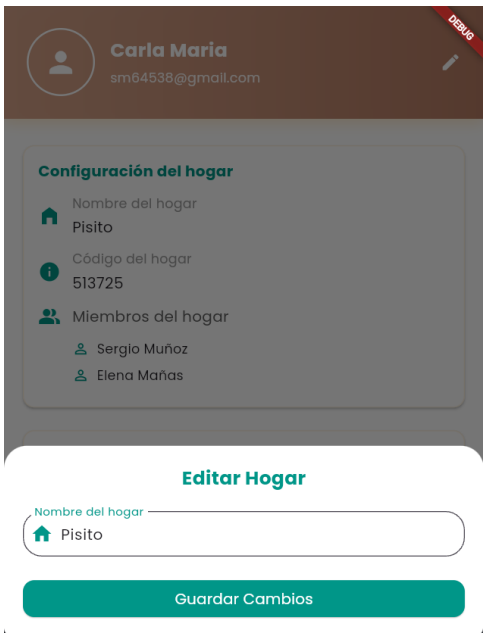


Figura 7.10: Interfaz gráfica pantalla de editar hogar.

Pantalla para participar en un hogar

La clase de esta pantalla se llama *ConenctHomeScreen* y esta diseñada para que un usuario se conecte a un hogar dentro de la aplicación, ya se creando uno nuevo o uniéndose a uno existente mediante un código. Esta pantalla se muestra principalmente a usuarios que han iniciado sesión pero aún no pertenecen a un hogar. Si el usuario escoge unirse, se abre un formulario que permite introducir un código de seis dígitos para validar la entrada al hogar. Si el código es correcto, se actualiza la base de datos para incluir al usuario dentro dele hogar. En caso de que el usuario decida crear un hogar, se redirige a la pantalla de creación de hogar.

Las funciones principales son:

- **_buildActionButton :** Crea el botón de acción que permite al usuario unirse a un hogar o crear uno nuevo. Si decide unirse, ejecuta una función y si decide crear un hogar, navega a la ruta definida.
- **_showJoinHomeModal :** Muestra un formulario en la parte inferior de la pantalla donde el usuario puede escribir el código para unirse a un hogar existente.
- **_buildCustomSnackBar :** Muestra un mensaje para informar al usuario si el código es incorrecto o si se ha unido correctamente.

- **_joinHomeWithCode :** Es la función que se encarga de validar el código introducido por el usuario.

El resultado de la implementación de esta pantalla es el siguiente:



Figura 7.11: Interfaz gráfica pantalla para participar en un hogar.

Pantalla para crear un hogar

En este caso la pantala se ha creado con la clase *CrearHogarScreen* y permite que el usuario cree un nuevo hogar dentro de la aplicación. Una vez que el usuario crea el hogar, se le asigna un código único de seis dígitos y se une a él automáticamente.

Las funciones que componen esta pantalla son:

- **generarCodigoHogar :** Genera un código aleatorio de 6 dígitos que se usará como identificador único para que otros usuarios puedan unirse al hogar creado.
- **crearHogar :** Esta función se encarga de crear el hogar en la base de datos de Firebase. Primero verifica que el nombre del hogar no esté vacío y luego guarda la información del hogar junto con el código generado.
- **_buildCustomSnackBar :** Muestra un mensaje para informar al usuario de errores o confirmaciones.

- **_buildFormSection :** Construye la sección del formulario donde el usuario puede ingresar el nombre del hogar.

El resultado de la implementación de la pantalla es el siguiente:

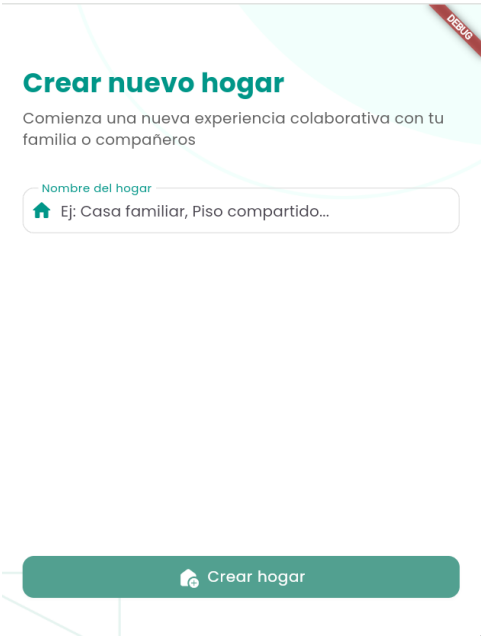


Figura 7.12: Interfaz gráfica pantalla para crear un hogar.

Pantalla de unirse a un hogar

Esta pantalla es la que se muestra cuando el usuario selecciona la opción de unirse a un hogar, con el método *_joinHomeWithCode*, explicado anteriormente.

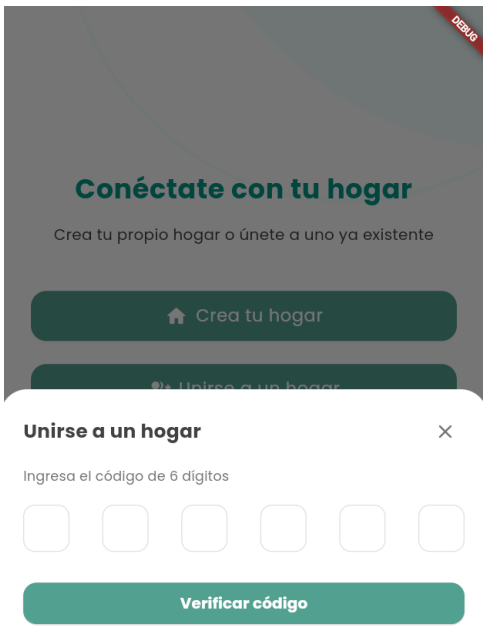


Figura 7.13: Interfaz gráfica pantalla de unirse a un hogar.

7.2.1.2. Base de datos

En esta primera iteracción ha sido necesario crear dos colecciones en la base de datos de Firebase, una para los usuarios y otra para los hogares.

Para la colección de **usuarios** se ha creado la siguiente estructura:

- **nombre**: Es el nombre que indica el usuario al registrarse.
- **apellidos**: Son los apellidos que indica el usuario al registrarse.

El resto de datos como el email y la contraseña se almacenan en Firebase Auth, por lo que no es necesario almacenarlos en la base de datos. El usuario se identifica por su ID único que genera Firebase Auth al registrarse, por tanto si queremos sacar datos adicionales del usuario como el correo electrónico, que no tenemos almacenado en la base de datos, se puede hacer una consulta a Firebase Auth a través del ID y obtenerlo.

Para la colección de **hogar** se ha creado la siguiente estructura:

- **nombre:** Es el nombre que indica el usuario al crear el hogar.
- **codigo:** Es el código único que se genera al crear el hogar.
- **miembros:** Es un array que contiene los ID de los usuarios que pertenecen a ese hogar.

- **fechaCreacion:** Es la fecha en la que se creó el hogar

7.2.1.3. Pruebas de funcionalidad

Para probar la implementación que se ha realizado se ha llevado a cabo una serie de pruebas de funcionalidad.

- Comprobación que el usuario puede registrarse correctamente.
- Comprobación de acceso a la aplicación con un usuario registrado.
- Comprobación de que en caso de que el usuario no este registrado, se muestre mensaje de error.
- Comprobación de que el usuario puede iniciar sesión con Google.
- Comprobación de que el usuario puede cerrar sesión.
- Comprobación de que el usuario puede ver sus datos.
- Comprobación de que el usuario puede editar sus datos.
- Comprobación de que el usuario puede crear un hogar.
- Comprobación de que el usuario puede unirse a un hogar.
- Comprobación de que el usuario puede ver los datos de su hogar.
- Comprobación de que el usuario puede editar los datos de su hogar.
- Comprobación de que el usuario puede abandonar un hogar.

7.2.1.4. Revisión de la iteracción

La aplicación ha sido probada por un usuario real, una mujer de 23 años, que comparte piso con otras tres personas. Durante todas las iteraciones se va a contar con su colaboración para realizar pruebas de funcionalidad y obtener nuevas mejoras.

Registro

El proceso de registro fue realizado con éxito por la usuaria, quien pudo crear su cuenta sin problemas. Sin embargo, depués de registrarse, la aplicación muestra un mensaje en la parte inferior indicando que debe verificar su correo y volver a iniciar sesión, pero la usuaria indicó que el mensaje no era claro y además aparecia durante un tiempo muy corto, lo que dificultaba su lectura.

Inicio de sesión

Después de indicarle a la usuaria lo que tenía que hacer después de registrarse, pudo iniciar sesión sin problemas.

Crear hogar

La usuaria tuvo algunas dificultades para crear un hogar, ya que el texto de ejemplo que aparece en el campo de nombre del hogar no era claro por tanto no sabía que tenía que escribir ahí. Después pudo crear el hogar sin problemas.

Visualización y edición de datos de usuario y hogar

La usuaria no tuvo ningún problema a la hora de identificar donde ver sus datos y los de su hogar, supo cómo editarlos. Lo único que comentó era que le resulta complicado encontrar como editar los datos de su hogar.

Abandonar hogar y unirse a otro hogar

La usuaria puedo abandonar sin problemas su hogar y unirse a otro hogar, mediante el código que le proporcionaron.

Cerrar sesión

La usuaria pudo cerrar sesión sin problemas, pero le gustaría que la aplicación le preguntara si realmente quiere cerrar sesión, para evitar cerrar sesion por error.

7.2.1.5. Plan para la siguiente iteracción

Para la siguiente iteracción se propone:

- Mejorar la intefaz de usuario, con las sugerencias y comentarios de la usuaria.
- Desarrollar los requisitos de la siguiente iteracción.

7.2.2. Iteración 2

En esta segunda iteracción se van a desarrollar los siguientes requisitos:

- RF_12: Crear tarea.
- RF_13: Consultar planificación semanal.

- RF_14: Marcar tarea como realizada.
- RF_15: Editar tarea.
- RF_16: Eliminar tarea.
- RF_17: Ver datos de una tarea.
- RF_18: Ver calendario de tareas.
- RF_25: Crear lista de la compra.
- RF_26: Añadir producto a la lista de la compra.
- RF_27: Eliminar producto de la lista de la compra.
- RF_28: Eliminar lista de la compra.
- RF_29: Ver productos de la lista de la compra.
- RF_30: Editar productos de la lista de la compra.

7.2.2.1. Implementacion

Durante la implementación de los requisitos se ha seguido el diseño propuesto en los bocetos iniciales, que mostramos en el cápitulo anterior. Al realizar la implementación, ha sido necesario añadir un requisito adicional:

RF_31	**Marcar producto como comprado**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario marcar un producto como marcado.
Criterios de aceptación	1. Debe existir una opción en la interfaz para marcar como comprado un producto. 2. Se actualizará la información en la base de datos.

Cuadro 7.1: RF_31

Este requisito era necesario para poder tener una lista de la compra totalmente funcional, ya que permite al usuario marcar los productos que ha comprado y que ya no necesita.

También se han creado algunos bocetos adicionales para detallar mejor la interfaz de usuario:



Figura 7.14: Diseño de la pantalla de crear tareas.

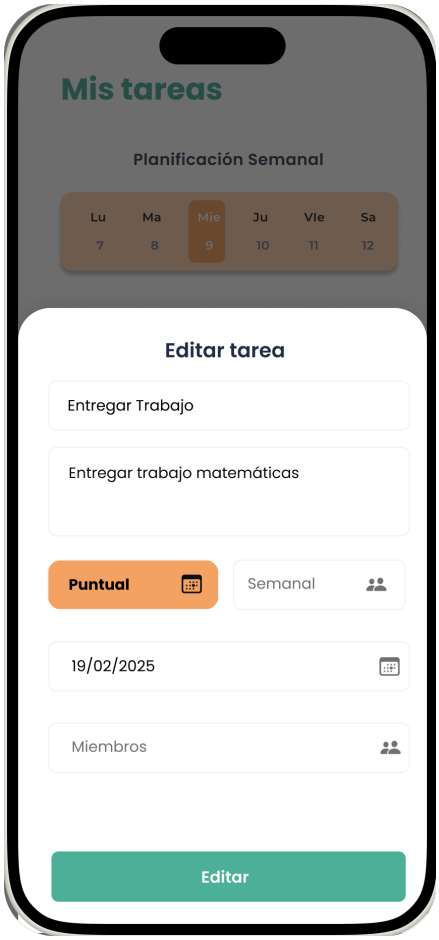


Figura 7.15: Diseño de la pantalla de editar tarea.

Tras completar la implementación, se han obtenido las siguientes pantallas:

Pantalla de tareas

La clase principal de esta pantalla se llama *TareasScreen* y se encarga de gestionar las tareas de los usuarios. Permite que se puedan visualizar las tareas de la semana y las del día actual, permite marcarlas como completadas, ver los detalles de estas, editarlas o eliminarlas. También permite crear nuevas tareas, tanto semanales como puntuales.

Al iniciar la pantalla se muestra widget iteractivo con la semana actual y dos secciones diferenciadas: una para las tareas que corresponde con la semana actual por defecto (si se cambia la semana en el widget mencionado anteriormente se mostraran las de esa semana) y otra para las tareas puntuales que corresponden por defecto al día actual.

Las funciones principales que hacen esto posible son:

- **initState :** Se ejecuta al abrir la pantalla. Inicializa las variables necesarias.
- **_loadInitialData :** Simula una pequeña carga para mostrar un indicdor visual.
- **dispose :** Libera los recursos cuando la pantalla se cierra.
- **_generarSemana :** A partir de una fecha dada, genera una lista de fechas que representa la semana correspondiente.
- **_obtenerSemanaMes :** Calcula a qué semana del mes pertenece una fecha concreta, con ajustes si la semana se cruza con el mes siguiente.
- **_buildLoadingIndicator :** Crea un indicador de carga que se muestra mientras se cargan los datos.
- **_buildMainContent :** Construye el contenido principal de la pantalla.
- **_buildWeekSelector :** Crea un selector de semanas que permite al usuario navegar entre las semanas.
- **_buildDayItem :** Muestra cada día de la semana con su número y abreviatura, destacando el seleccionado o el actual.
- **_buildTaksSection :** Agrupa las dos secciones de tareas: semanales y puntuales.
- **_buildSectionHeader :** Crea el encabezado de cada sección.
- **_buildWeeklyTaskList :** Busca en la base de datos las tareas semanales y muestra las tareas semanales asignadas al usuario para la semana y mes seleccionados.

- **_buildDailyTaskList :** Muestra las tareas puntuales del usuario que coinciden con el día seleccionado.
- **_buildTasksList :** Genera la lista de tareas, o un mensaje si no hay ninguna.
- **_buildTasksCard :** Crea una tarjeta para cada tarea, con su información básica.
- **_getShortWeekday :** Devuelve el nombre abreviado de los días de la semana.
- **_esSemanaCambioDeMes :** Determina si la semana actual se extiende al mess siguiente.
- **_handleRefresh :** Permite actualizar la pantalla al arrastrar hacia abajo.
- **_navigateWeek :** Cambia la semana mostrada hacia adelante o hacia atrás.
- **_mostrarDetallesTarea :** Muestra un modal con todos los detalles de una tarea.
- **_obtenerNombreMes :** Devuelve el nombre en español de un mes dado.
- **_mostrarDialogoConfirmacióonEliminar :** Muestra un cuadro de diálogo para confirmar la eliminación de una tarea.
- **_eliminarTarea :** Elimina una tarea de la base de datos y actualiza la pantalla.
- **_mostrarModalEditarTarea :** Muestra un formulario para editar una tarea existente.
- **_actualizarEstadoTarea :** Cambia el estado de una tarea a completada o no completada.
- **_mostrarModalCrearTarea :** Muestra un formulario para crear una nueva tarea.

El resultado de la implementación de esta pantalla es el siguiente:

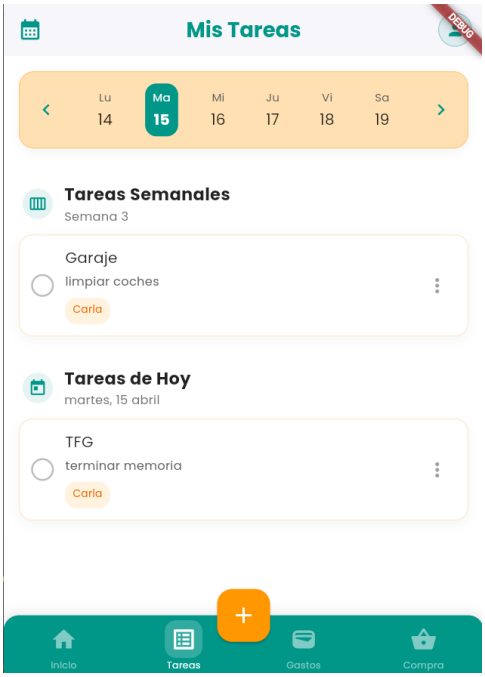


Figura 7.16: Interfaz gráfica pantalla de tareas.

Pantalla de crear tarea

Esta pantalla se muestra con el método *_mostrarModalCrearTarea*, aunque este método delega toda su funcionalidad en la clase *CrearTareaForm* que es la encargada de gestionar la creación de tareas y mostrar el formulario.

Dicha clase esta compuesta por los siguientes métodos:

- **initState :** Se ejecuta al abrir la pantalla y llama a los métodos _cargarMiembrosHogar y _verificarSemanaCambioMes que explicaremos más adelante.
- **_cargarMiembrosHogar :** Consulta en la base de datos para encontrar el hogar al que pertenece el usuario y obtiene la lista de miembros que comparten hogar.
- **_verificarSemanaCambioMes :** Si estamos en los últimos días del mes, cambia a la primera semana del siguiente mes para que se muestren las tareas semanales de dicha semana.
- **_seleccionarFecha :** Abre el calendario para que el usuario pueda seleccionar una fecha en caso de que la tarea sea puntual.

- **_toggleSemana :** Permite al usuario elegir las semanas en las que se realizará la tarea semanal.
- **_crearTarea :** Se encarga de validar que se haya rellenado correctamente todos los campos del formulario, crea un objeto con todos los datos de la tarea y los guarda en Firestore.
- **_mostrarError :** Muestra un mensaje en caso de que ocurra algún error.
- **_buildTaskTypeOption :** Crea los botones que permiten cambiar entre tarea puntual o semanal.

Por tanto la pantalla de crear tarea se divide en dos partes: una para crear tareas semanales y otra para crear tareas puntuales y se ven así:



Figura 7.17: Interfaz gráfica pantalla de crear tarea puntual.

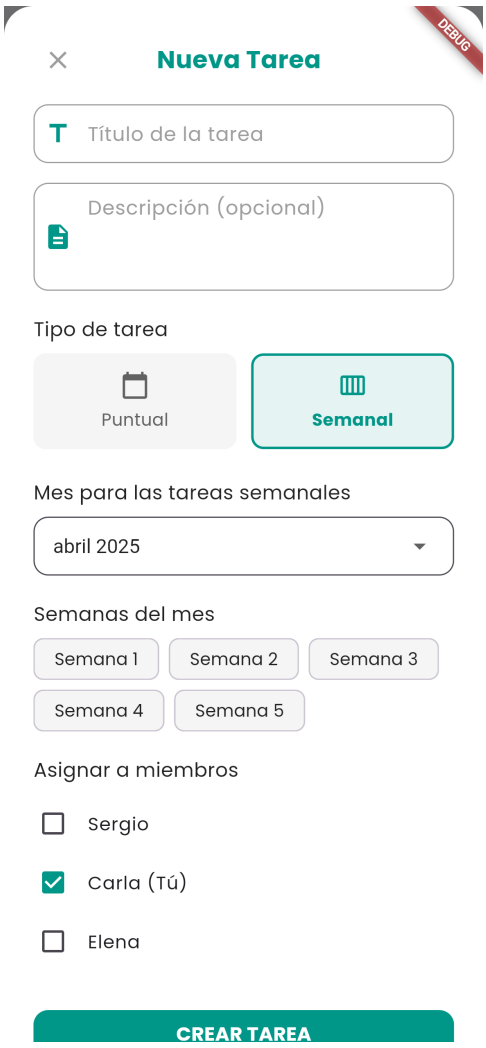


Figura 7.18: Interfaz gráfica pantalla de crear tarea semanal.

Pantalla de ver datos de una tarea

Esta pantalla se muestra al usuario cuando selecciona una tarea específica en la pantalla de tareas. Su implementación se realiza a través del método *_mostrarDetallesTarea*, el cual despliega un modal que contiene toda la información detallada de la tarea seleccionada. El resultado es el siguiente:



Figura 7.19: Interfaz gráfica pantalla de ver datos de una tarea.

Pantalla para editar o eliminar una tarea

Esta pantalla se muestra cuando el usuario selecciona el menú de opciones (representado por tres puntos) ubicado a la derecha de una tarea específica. Desde esta pantalla, el usuario puede optar por editar la tarea, lo que lo redirige a la pantalla de edición, o eliminarla, lo que muestra un cuadro de diálogo de confirmación antes de proceder. Esta implementado dentro de la clase *TareasScreen*, explicada anteriormente. El resultado es el siguiente:

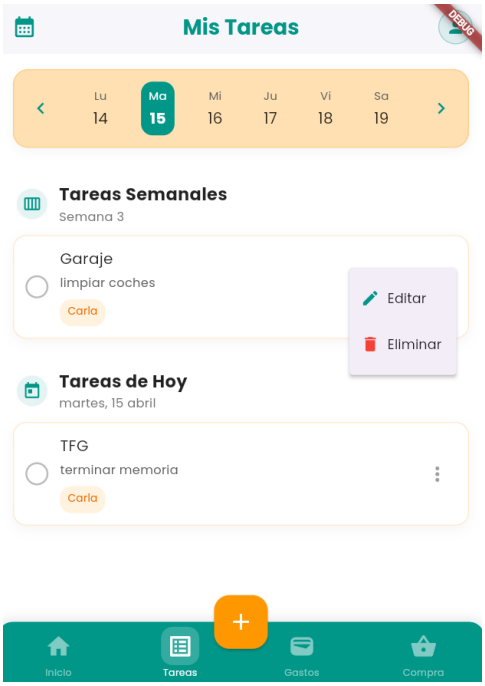


Figura 7.20: Interfaz gráfica pantalla para editar o eliminar una tarea.

Pantalla editar tarea

Esta pantalla se muestra con el método *_mostrarModalEditarTarea*, aunque este método delega toda su funcionalidad en la clase *EditarTareaForm* que es la encargada de gestionar la edición de tareas y mostrar el formulario. Esta clase tiene los mismos métodos que la clase *CrearTareaForm* explicada anteriormente, pero con la diferencia de que el método*_crearTarea* se llama _editarTarea y se encarga de editar la tarea en lugar de crearla y el formulario se inicia con los datos de la tarea seleccionada. El resultado es el siguiente:

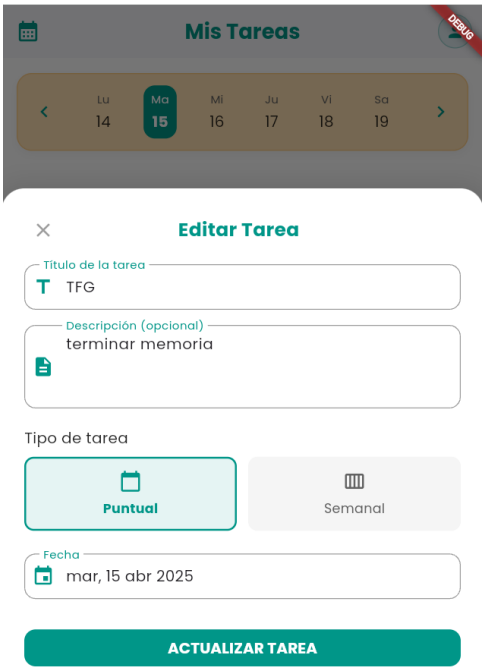


Figura 7.21: Interfaz gráfica pantalla de editar tarea.

Pantalla de calendario

Esta pantalla esta implementada dentro de la clase *CalendarioScreen* y su objetivo mostrar las tareas del usuario en un formato de calendario. Se mustra tanto las tareas puntuales como las semanales (aunque no estén asignadas al usuario) y permite ver claramente que días tiene tareas asignadas y los detalles de cada tarea.

Al iniciar la pantalla se carga el calendario junto con las tareas del hogar al que pertenece el usuario. Se muestran los días con tareas mediante formas circulares de colores que en función del color indican si la tarea es semanal o puntual. Al seleccionar un día concreto en el calendario, aparecen las tareas correspondientes a ese día, primero las semanales y luego las puntuales. Mientras que se visualizan las tareas, se pueden marcar como completada, editar o eliminar cualquier tarea. Además en todo momento se puede ver el botón que permite añadir una nueva tareaa, cuya lógica es la misma que la de la pantalla de tareas.

Los métodos que componen esta pantalla son:

- **initState :** Se ejecuta al abrirse la pantalla. Configura el calendario en español y selecciona el día actual por defecto.
- **dispose :** Libera los recursos cuando se cierra la pantalla.

- **_setupHouseholdListener :** Detecta cuando hay algun cambio en el hogar del usuario y recarga las tareas cuando esto ocurre.
- **_loadTasksFromFirestore :** Además de escuchar todos los cambios en la colección de tareas, carga las tareas y las filtra en tareas puntuales del usuario y semanales de cualquier miembro del hogar.
- **_loadUserNames :** Carga los nombres de los usuarios desde la colección de usuarios. Sirve para mostrar los nombres de los usuario en cada tarea.
- **_getDailyTasks :** Devuelve una lista con las tareas puntuales del usuario que ocurren en un día específico.
- **_getWeeklyTasksForDay :** Busca en la lista de las tareas semanales cuáles ocurren en un día específico.
- **_buildCalendar :** Se encarga de construir el calendario iteractivo y de mostrar las tareas en los días correspondientes.
- **_buildTaskCard :** Muestra un tarjeta con la información de una tarea, permitiendo ver los detalles de la tarea, marcarlas como completadas o acceder a las opciones de editarla o eliminarla.
- **_buildDetalleItem :** Muestra en detalle la información de una tarea seleccionada.
- **_obtenerNombreMes :** Devuelve el nombre del mes en español a partir de su número.
- **_toggleTaskCompletion :** Cambia el estado de una tarea entre completada y no completada en Firestore.
- **_handleTaskAction :** Comprueba si el usuario ha elegido editar o eliminar una tarea desde el menú y llama al método correspondiente.
- **_confirmDeleteTask :** Muestra un cuadro de confirmación antes de eliminar una tarea. Si se confirma, se borra la tarea de Firestore.
- **_mostrarModalCrearTarea :** Muestra un modal para crear una nueva tarea.
- **_mostrarModalEditarTarea :** Muestra un modal para editar una tarea existente.

El resultado de la implementación de esta pantalla es el siguiente:

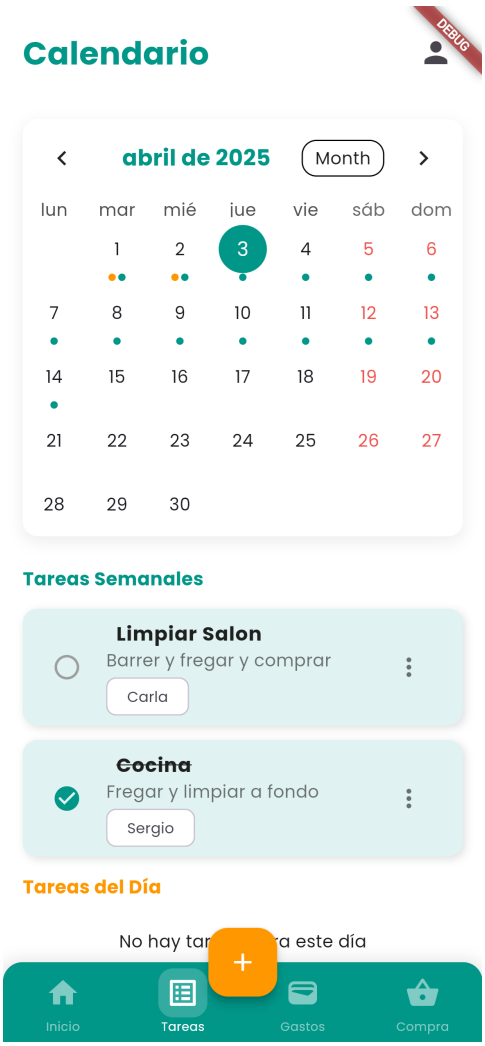


Figura 7.22: Interfaz gráfica pantalla de calendario.

Pantalla de lista de la compra

La clase principal de esta pantalla se llama *ListaScreen* y permite al usuario, ver, crear, buscar y eliminar listas de la compra, ya sean individuales o compartidas con los miembros del hogar. Esta pantalla muestra todas las listas activas del usuario y permite acceder a los productos que tiene cada una.

Los métodos que componen esta pantalla son:

- **dispose :** Libera los recursos cuando la pantalla se cierra.
- **_buildListaCompra :** Se conecta a la base de ddatos y obtiene todas

las listas donde el usuario está incluido y muestra un mensaje si no hay listas creadas o una lista con las tarjetas de cada lista.

- **_buildListaCard :** Crea una tarjeta para cada lista de la compra, muestra su nombre y si es individual o compartida.
- **_confirmarBorradaLista :** Muestra un cuadro de confirmación antes de eliminar una lista. Si se confirma, borra todos los productos asociados a esa lista en Firestore y luego borra la lista.
- **_mostrarModalCrearLista :** Muestra un modal para crear una nueva lista de la compra.
- **_crearLista :** Se encarga de validar que se haya rellenado correctamente todos los campos del formulario, crea un objeto con todos los datos de la lista y los guarda en Firestore.

Esta pantalla es bastante sencilla, y su resultado es el siguiente:

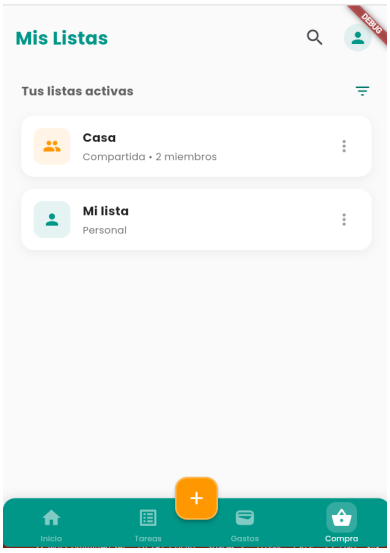


Figura 7.23: Interfaz gráfica pantalla de lista de la compra.

Pantalla de crear lista de la compra

Esta pantalla se muestra con el método *_mostrarModalCrearLista*, explicado anteriomente, que es el encargado de gestionar la creación de listas de la compra y mostrar el formulario.

El resultado de la implementación de esta pantalla es el siguiente:

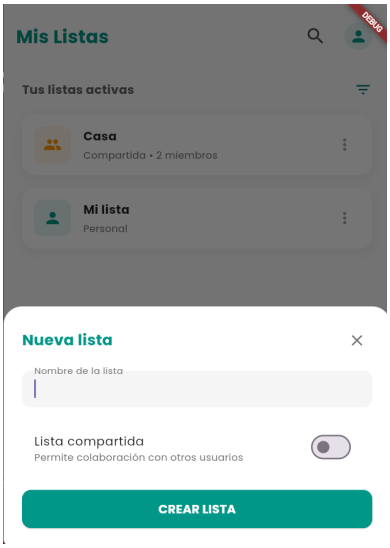


Figura 7.24: Interfaz gráfica pantalla de crear lista de la compra.

Pantalla para eliminar lista de la compra

Esta pantalla se muestra cuando el usuario selecciona el menú de opciones (representado por tres puntos) ubicado a la derecha de una lista específica. Desde esta pantalla, el usuario puede eliminar la lista, lo que muestra un cuadro de diálogo de confirmación antes de proceder. Esta implementado dentro de la clase *ListaScreen*, explicada anteriormente. El resultado es el siguiente:

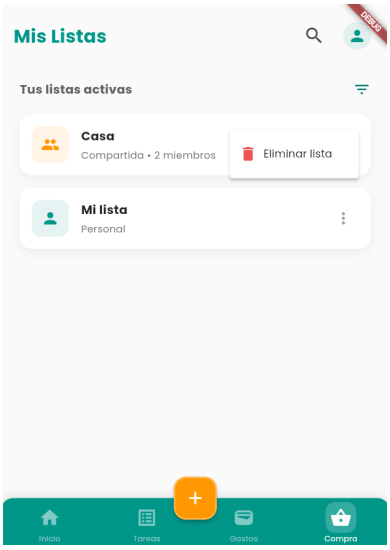


Figura 7.25: Interfaz gráfica pantalla para eliminar lista de la compra.

Pantalla de productos de la lista de la compra

La clase principal de esta pantalla es *ProductoListaScreen* y permite gestionar los productos que pertenecen a una lista de la compra específica. Esta pantalla permite ver, añadir, modificar la cantidad, marcar como comprado o eliminar productos dentro de una lista. Además incluye una opción de marcar todoss los productos como comprados de una sola vez.

Al abrir esta pantalla, se cargan automáticamente todos los productos relacionados con la lista seleccionada. Se organizan en una lista y se muestran diferenciado entre los productos comprados y los que no lo están. Incluye también un botón para añadir nuevos productos a la lista.

- **_buildProductossLista :** Obtiene todos los productos asociados a la lista desde Firestore y los ordena para mostrar primero los que aún no se han comprado.
- **_cambiarCantidad :** Permite aumentar o disminuir la cantidad de un producto en la lista.

- **_agregarProducto :** Añade un nuevo producto en la base de datos.
- **_marcarComoComprado :** Cambia el estado de un producto entre comprado y no comprado en Firestore.
- **_eliminarProducto :** Elimina un producto de la lista y de la base de datos.
- **_marcarTodosComoComprados :** Cambia el estado de todos los productos de la lista a comprados.
- **_mostrarModalAgregarProducto :** Muestra un modal para añadir un nuevo producto a la lista.
- **_eliminarProductosCompradosExpirados :** Se encarga de eliminar los productos que ya han sido comprados y que llevan más de 24 horas en ese estado.

La pantalla se muestra de la siguiente manera:

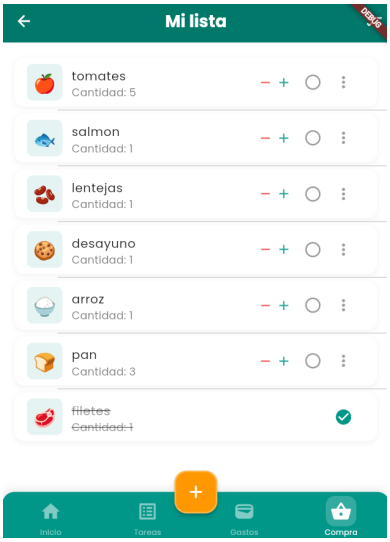


Figura 7.26: Interfaz gráfica pantalla de productos de la lista de la compra.

Pantalla de añadir producto a la lista de la compra

Esta pantalla se muestra con el método *_mostrarModalAgregarProducto*, explicado anteriomente.

El resultado es el siguiente:

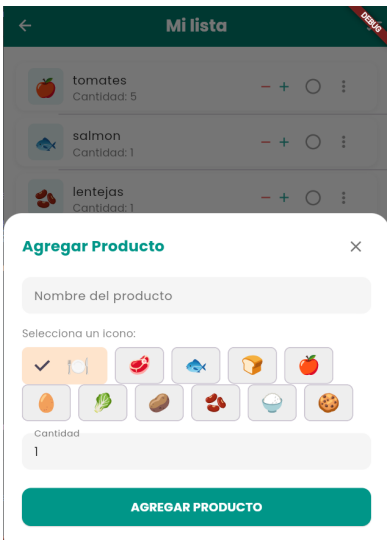


Figura 7.27: Interfaz gráfica pantalla de añadir producto a la lista de la compra.

7.2.2.2. Base de datos

Como resultado de la implementación de esta iteracción, se han creado las colecciones tareas, lista y producto en la base de datos de Firestore.

Para la colección de **tareas** se ha crado la siguiente estructura:

- **asignados**: Contiene los usuarios asignados a la tarea (en caso de que sea semanal).
- **año**: Almacena el año en el que se va a realizar la tarea.
- **completada**: Almacena el estado de la tarea (si ha sido completada o no).
- **descripcion**: Es la descripción de la tarea.
- **fecha**: Es la fecha en la que se va a realizar la tarea (en caso de que sea puntual).
- **fechaActualizacion**: En caso de que la tarea se modifique, almacena la fecha de la última modificación.
- **fechaCreacion**: Almacena la fecha en la que se creó la tarea.
- **mes**: Es el mes en el que se va a realizar la tarea. (en caso de que sea semanal).

- **semanas:** Es una lista con las semanas del mes en las que se va a realizar la tarea (en caso de que sea semanal).
- **tipo**: El tipo de tarea (semanal o puntual).
- **titulo**: Es el título de la tarea.
- **usuarioId**: Es el id del usuario que ha creado la tarea.

La colección de **lista** almacena la siguiente información:

- **esGrupal**: Es un booleano que almacena true en caso de que la lista sea compartida con el hogar y false en caso de que sea individual.
- **nombre**: Es el nombre de la lista.
- **usuario**: Es el id de los usuarios que participan en la lista.

Por último, la colección de **producto** almacena la siguiente información:

- **cantidad**: Es la cantidad del producto que se va a comprar.
- **comprado**: Es un booleano que almacena true en caso de que el producto haya sido comprado y false en caso de que no lo haya sido.
- **fechaComprado**: Si se ha comprado el producto, almacena la fecha en la que se compró.
- **fechaCreacion**: Es a l fecha en la que se creó el producto.
- **icono**: Es el icono del producto.
- **listaId**: Es el id de la lista a la que pertenece el producto.
- **nombre**: Es el nombre del producto.

7.2.2.3. Pruebas de funcionalidad

Para probar la implementación que se ha realizado se ha llevado a cabo una serie de pruebas de funcionalidad.

- Comprobación de la creación de una nueva tarea. Esto incluye que la tarea se almacene correctamente en la base de datos con los datos correctos y que se muestre en la intefaz de tareas.
- Comprobación de la visualización de la planificación semanal. Esto incluye que las tareas semanales se muestren correctamente en la interfaz de tareas y que se actualicen al seleccionar un día específico.

- Comprobación de la edición de una tarea.
- Comprobación de la eliminación de una tarea.
- Comprobación de la visualización de los datos de una tarea.
- Comprobación de la visualización del calendario de tareas. Esto incluye comprobar que todas las tareas semanales y puntuales se muestren correctamente en el calendario y que se actualicen al seleccionar un día específico.
- Comprobación de marcar una tarea como realizada.
- Comprobación de la creación de una lista de la compra. Esto incluye que la lista se almacene correctamente en la base de datos y que se muestre en la interfaz de listas de la compra.
- Comprobación de la visualización de los productos de la lista de la compra. Esto incluye que todos los productos se muestren correctamente en la interfaz de productos de la lista de la compra.
- Comprobación de la creación de un producto en la lista de la compra. Esto incluye que el producto se almacene correctamente en la base de datos y que se muestre en la interfaz de productos de la lista de la compra.
- Comprobación de cambiar la cantidad de un producto en la lista de la compra.
- Comprobación de la eliminación de un producto de la lista de la compra.
- Comprobación de marcar un producto como comprado.
- Comprobación de la eliminación de una lista de la compra.

7.2.2.4. Revisión de la iteracción

La aplicación ha sido probada por la misma usuaria que en la iteracción anterior. Ha probado todas las funcionalidades nuevas que se han implementado y las mejoras implementadas con los comentarios que realizó en la iteracción anterior.

Crear una tarea

La usuaria pudo crear una tarea sin problemas, pero le gustaría que la aplicación le permitiera elegir el día de la semana en el que se va a realizar

la tarea (en caso de que sea semanal) y en el caso de no saber el día de la semana en el que se va a realizar, que se asigne automáticamente a todos los días de la semana.

Consultar planificación semanal

La usuaria pudo consutar la planificación semanal sin problemas, pero le costó averiguar que esa pantalla mostraba solo las tareas para la semana actual y que si quería ver las tareas de otras semanas, tenía que ir al calendario.

Visualización, edición y eliminación de tareas

La usuaria no tuvo ningún problema a la hora de ver, editar y eliminar tareas y comentó que la interfaz era muy intuitiva y fácil de usar.

Visualización del calendario de tareas

La usuaria no tuvo ningún problema a la hora de ver el calendario de tareas, entendió perfectamente como funcionaba, pero indicó que le gustaría que la aplicación le mandará recordatorios de las tareas que tiene que realizar.

Crear lista de la compra, añadir productos y eliminar productos

La usuaria pudo crear una lista, añadir productos, eliminarlos, cambiar la cantidad de un producto y marcarlo como comprado sin problemas. Estuvo muy satisfecha con la interfaz y el diseño de la aplicación, ya que le resultó muy intuitivo y fácil de usar.

7.2.2.5. Plan para la siguiente iteracción

Para la siguiente iteracción se propone:

- Mejorar la intefaz de usuario, con las sugerencias y comentarios de la usuaria.
- Desarrollar los requisitos de la siguiente iteracción.
- Implementar una pantalla principal provisional, para hacer la aplicación más intuitiva.
- Corregir los posibles errores que se hayan detectado en la iteracción anterior

7.2.3. Iteración 3

En esta iteracción se van a desarrollar los siguientes requisitos:

- RF_19: Crear gasto.
- RF_20: Editar gasto.
- RF_21: Eliminar gasto.
- RF_22: Saldar deuda.
- RF_23: Ver deudas.
- RF_24: Ver detalles de un gasto.

7.2.3.1. Implementacion

Durante la implementación de los requisitos se ha seguido el diseño propuesto en los bocetos iniciales, aunque ha sido necesario realizar un boceto más adicional para detallar mejor la interfaz de usuario:



Figura 7.28: Diseño de la pantalla para saldar deudas.

Una vez se ha realizado el boceto, se ha implementado las siguientes pantallas:

Pantalla de gastos

La clase principal de esta pantalla se llama GastosScreen y se encarga de gestionar los gastos compartidos en un hogar, permite a los usuarios añadir, editar y eliminar gastos y además consultar el balance de deudas personales de cada miembro del hogar. La pantalla está dividida en dos pestañas principales, una para ver todos los gastos registrados y otra para ver las deudas pendientes. Los datos se organizan por fecha, y cada gasto incluye el nombre, el pagador, los participantes y el importe correspondiente a cada uno. Además, las deudas se actualizan automáticamente cada vez que se registra o elimina un gasto.

Los métodos que componen esta pantalla son:

- **initState :** Inicializa la pantalla y carga algunos datos iniciales.
- **dispose :** Se encarga de liberar recursos cuando la pantalla se destruye.
- **_cargarDatosHogar :** Busca y carga los fatos del hogar al que pertenece el usuario.
- **_cargarNombresMiebres :** A través de los IDs de los miembros del hogar, busca en la base de datos y carga los nombres de los miembros del hogar.
- **_mostrarModalAgregarGasto :** Abre un formulario para crear o editar un gasto. Permite introducir el título, monto, fecha, quién lo pagó y quiénes deben pagar. Calcula automáticamente el monto dividido entre los participantes.
- **_eliminarGasto :** Elimina un gasto de la base de datps y llama a la función correspondiente para ajustar las deudas.
- **_mostrarConfirmacionEliminar :** Muestra un cuadro de diálogo para confirmar la eliminación de un gasto.
- **_mostrarDetalleGasto :** Abre un modal con todos los detalles del gasto seleccionado.
- **_buildGastosTab :** Construye la pestaña de gastos, mostrando una lista de todos los gastos registrados, agrupados por fecha y también un resumen con el total de gastos y los del usuario.
- **_buildGastosCard :** Crea una tarjeta para cada gasto. Mostrando el título, el monto, quién lo pagó y si el usuario es uno de los participantes.
- **_buildDetailCard :** Muestra pequeños bloques de información con algunos detalles de los gastos.

La clase tiene más métodos pero se explicarán más adelante junto con las pantallas adecuadas. El resultado de la implementación de esta pantalla es el siguiente:

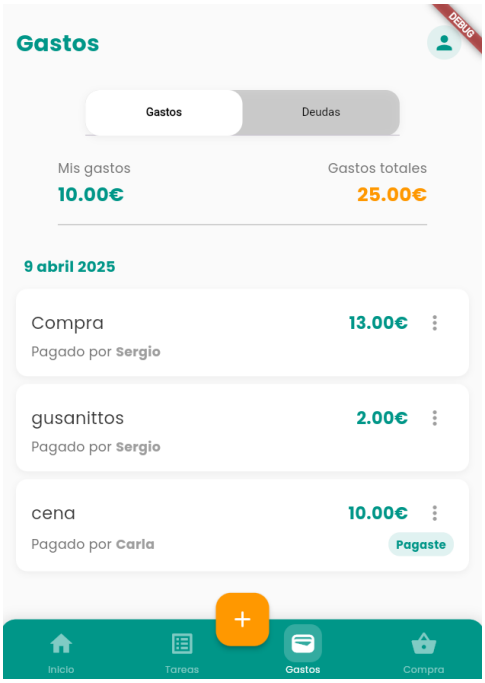


Figura 7.29: Interfaz gráfica pantalla de gastos.

Pantalla de crear gasto

Esta pantalla se muestra con el método *_mostrarModalAgregarGasto*, explicado anteriomente, que es el encargado de gestionar la creación de gastos y mostrar el formulario.

El resultado de la implementación de esta pantalla es el siguiente:



Figura 7.30: Interfaz gráfica pantalla de crear gasto.

Pantalla de deudas

Esta pantalla también está implementada dentro de la clase GastosScreen y permite al usuario ver las deudas pendientes de cada miembro del hogar. La pantalla muestra una lista de los miembros del hogar y el saldo que cada uno con respecto a las deudas. Así como un resumen de las deudas totales del usuario. Los métodos relacionados con esta pantalla son:

- **_actualizarDeudasDesdeGastos :** Cuando se crear o edita un gasto, este método se encarga de actualizar las deudas entre el pagador y los participantes, teniendo en cuneta si existen deudas previas o si deben compensarse.
- **_revertirDeudaDesdeGastos :** Cuando se elimina un gasto, ajusta o elimina las deudas.
- **_buildDeudasTab :** Construye la pestaña de deudas, donde se muestra el balance de deudas de todos los miembros del hogar. Tambien muestra un resumen de las deudas del usuario.
- **_buildListaDeudasCompleta :** Muestra todas las deudas del hogar, indicando si son deudas a favor o en contra del usuario y además muestra deudas que son entre otros miembros del hogar. También permite marcala como pagada o cobrada.

- **_marcarDeudaComoPagada :** Permite actualizar una deuda para marcarla como pagada y la registra en la colección de pagos.
- **_compensarDeudas :** Se encarga de reorganizar todas las deudas del hogar para simplificarlas. Elimina las deudas cruzadas y las convierte en una sola deuda entre dos miembros.
- **_calcularBalances :** Calcula cuánto debe o le deben a cada miembro del hogar.

El resultado de la implementación de esta pantalla es el siguiente:

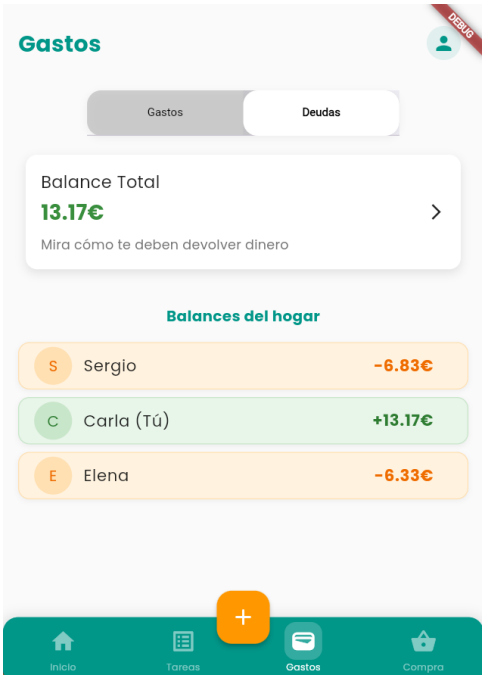


Figura 7.31: Interfaz gráfica pantalla de deudas.

Pantalla de ver detalles de un gasto

Esta pantalla se muestra con el método *_mostrarDetalleGasto*, que es el encargado de mostrar un modal con todos los detalles del gasto seleccionado. El resultado es el siguiente:

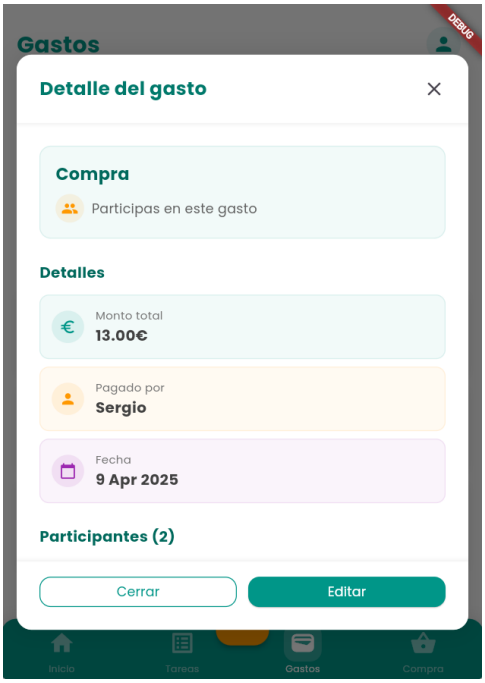


Figura 7.32: Interfaz gráfica pantalla de ver detalles de un gasto.

Pantalla saldar deuda

Esta pantalla se muestra con el método *_buildListaDeudasCompleta*, que es el encargado de mostrar un modal con todos los detalles de la deuda seleccionada. El resultado es el siguiente:

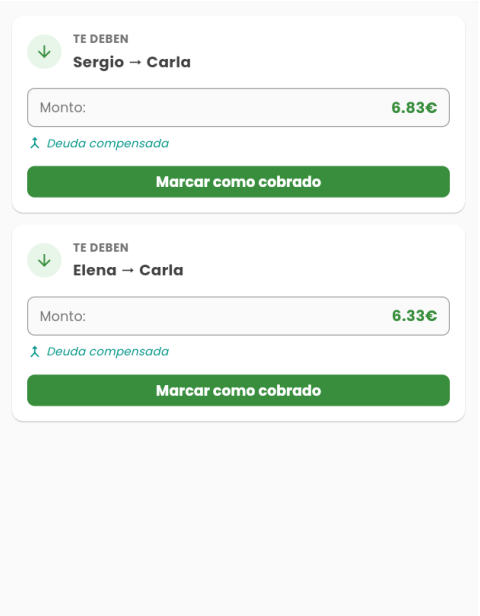


Figura 7.33: Interfaz gráfica pantalla para saldar deuda.

Pantalla principal de la aplicación

La clase principal de este archivo es *HomePage* y representa la pantalla de inicio de la aplicación. Esta pantalla ofrece un visión general de todo lo que el usuario puede hacer en la aplicación, tareas pendientes, deudas compartidas y productos en la lista de la compra. Funciona como un panel central donde el usuario puede consultar resumenes y acceder a las diferentes funcionalidades de la aplicación.

Las funciones que componen esta pantalla son:

- **initState :** Se ejecuta al abrir la pantalla y carga los datos iniciales del hogar, productos, deudas y tareas.
- **dispose :** Libera los recursos cuando se cierra la pantalla.
- **_loadInitialData :** Ejecuta en paralelo la carga de los datos del hogar, productos, deudas y tareas.
- **_cargarNombresUsuarios :** Obtiene a través de los IDs de los miembros del hogar, los nombres de los miembros del hogar.
- **_setupTareasListener :** Escucha los cambios en las tareas del usuario y calcula cuántas tareas debe realizar hoy, cuántas tareas tiene en la semana actual y cuántas tareas tiene pendientes.

- **_setupDeudasListener :** Escucha los cambios en las deudas del usuario. Actualiza la lista de deudas no pagadas.
- **_obtenerNombreHogar :** Busca el hogar al que pertenece el usuario actual y obtiene el nombre.
- **_obtenerProductosUsuario :** Busca y carga hasta 5 productos que pertenece a la lista de compra donde participa el usuario.
- **_obtenerDeudasPendientes :** Carga las 3 deudas que afectan al usuario, ya sea como deudor o acreedor. También obtiene los nombres de las personas implicadas.
- **_buildAppBar :** Construye la barra superior de la pantalla, donde se muestra el nombre del hogar y el botón del perfil.
- **_buildLoadingIndicator :** Muestra un indicador de carga mientras se cargan los datos iniciales.
- **_buildQuickSummary :** Muestra las dos tarjetas superiores con un resumen de las tareas y el balance de deudas del usuario.
- **_buildSummaryCard :** Construye una tarjeta que muestra un resumen de las tareas o deudas del usuario.
- **_buildWeekSection :** Sección de planificación semanal que enlaza con el calendario. Muestra los días de la semana resaltando el día actual.
- **_buildSectionHeader :** Cabecera de cada sección con opción de ver más.
- **_buildWeekPlanner :** Visualiza la semana actual como una fila de días, resaltando el día de hoy.
- **_buildDiaItem :** Dibuja cada día de la semana con su nombre abreviado y número.
- **_buildWeekStats :** Muestra estadísticas rápidas de las tareas de hoy y de la semana.
- **_buildStatItem :** Tarjeta pequeña para cada estadística.
- **_buildDebtsSection :** Sección que muestra las deudas pendientes del usuario. Permite ver detalles y acceder a la pantalla completa de gastos.
- **_buildDebtTile :** Tarjeta que muestra un resumen de cada deuda pendiente.

- **_buidShoppingListSection :** Sección que muestra los productos de la lista de la compra.
- **_buildProductTile :** Tarjeta de un solo producto, mostrando su icono, nombre y cantidad.
- **_buildEmptyState :** Muestra un mensaje cuando no hay datos para mostrar.

EL resultado de la implementación de esta pantalla es el siguiente:



Figura 7.34: Interfaz gráfica pantalla principal de la aplicación.

Barra de navegación

Ha sido necesario implementar una clase más correspondiente a la barra inferior de navvegación de la aplicación que se puede observar en la mayoría de las pantallas. Esta barra se ha implementado dentro de la clase *CustomBottomNavigationBar* y permite al usuario navegar entre las diferentes pantallas de la aplicación.

Los métodos que hacen esto posible son:

- **build :** Construye la barra de navegación y la muestra en la pantalla.
- **_buildNavItem :** Crea cada uno de los elementos de la barra de navegación.
- **_onBottomNavTapped :** Se ejecuta cuando el usuario pulsa un botón de la barra. Según el botón pulsado, navega a la pantalla correspondiente.

La barra de navegación se muestra de la siguiente manera:



Figura 7.35: Interfaz gráfica barra de navegación.

Clase Main

Aunque no lo hemos comentado hasta esta última iteración debido a que no era relevante, la clase *Main* es la encargada de inicializar la aplicación y redirigir al usuario a la pantalla de bienvenida o de inicio, dependiendo de si el usuario ya ha iniciado sesión o no. Esta clase se encarga también de inicializar la conexión con Firebase y de gestionar todas las rutas internas de la aplicación.

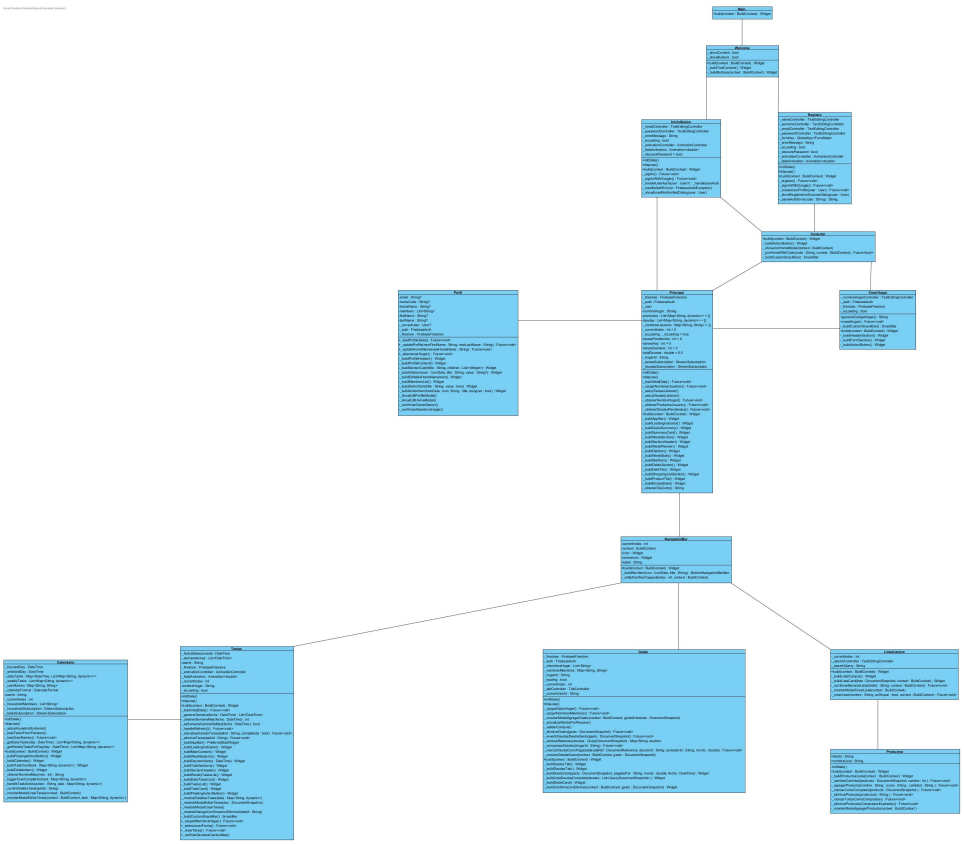


Figura 7.36: Diagrama final de clases.

7.2.3.2. Base de datos

Como resultado de esta última iteracción, se han creado las colecciones gastos, deudas y pagos en la base de datos de Firestore.

Para la colección de **gastos** se ha creado la siguiente estructura:

- **debenPagar**: Almacena los IDs de los usuarios que deben pagar el gasto.
- **fecha**: Almacena la fecha en la que se ha realizado el gasto.
- **fechaCreacion**: Almacena la fecha en la que se ha creado el gasto.
- **hogarId**: Almacena el ID del hogar al que pertenece el gasto.
- **monto**: Almacena el monto total del gasto.
- **montoPorPersona**: Almacena el monto que debe pagar cada persona.

- **pagado**: Almacena un booleano que indica si el gasto ha sido pagado o no.
- **pagadorId**: Almacena el ID del usuario que ha pagado el gasto.
- **titulo**: Almacena el título del gasto.

La colección de **deudas** almacena la siguiente información:

- **acreedor**: Almacena el ID del usuario que es el acreedor de la deuda.
- **creadoEn**: Almacena la fecha en la que se ha creado la deuda.
- **deudor**: Almacena el ID del usuario que es el deudor de la deuda.
- **esCompensada**: Almacena un booleano que indica si la deuda ha sido compensada o no.
- **fechaPago**: Almacena la fecha en la que se ha pagado la deuda en caso de que se haya pagado.
- **hogarId:** Almacena el ID del hogar al que pertenece la deuda.
- **monto**: Almacena el monto de la deuda.
- **pagado**: Almacena un booleano que indica si la deuda ha sido pagada o no.
- **ultimaActualizacion**: Almacena la fecha de la última actualización de la deuda.

Por ultimo la colección de **pagos** almacena la siguiente información:

- **acreedor**: Almacena el ID del usuario que es el acreedor del pago.
- **deudor**: Almacena el ID del usuario que es el deudor del pago.
- **fecha**: Almacena la fecha en la que se ha realizado el pago.
- **hogarId**: Almacena el ID del hogar al que pertenece el pago.
- **monto**: Almacena el monto del pago.
- **tipo**: Almacena el tipo de pago (si es un pago o una compensación).

Como esta es la ultima iteracción, ya tenemos la base de datos completa y podemos realizar el diagrama final de la base de datos:

Capítulo 8

Validación de la aplicación por usuarios reales

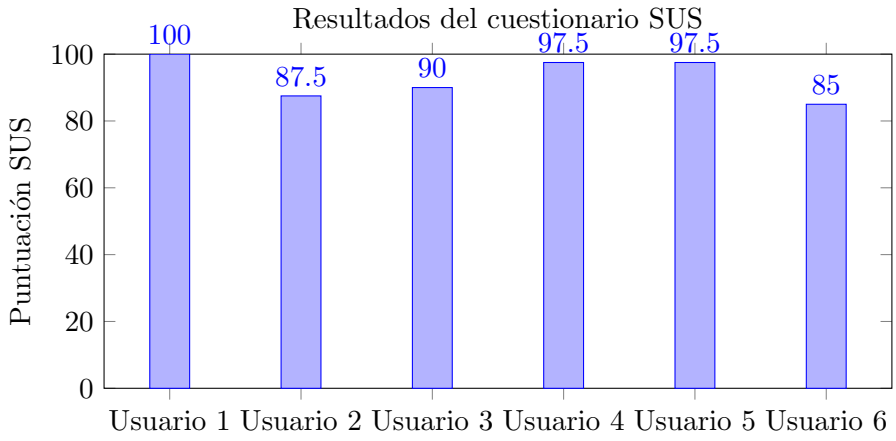


Figura 8.1: Puntuaciones individuales SUS

Apéndice A

Descripción de casos de uso

A.1. Casos de uso de la gestión de sesión

Caso de Uso	registro		CU1
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_01		
Precondición	El usuario no debe estar registrado en el sistema.		
Postcondición	El usuario queda registrado en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crearse una cuenta en el sistema.
Resumen	El usuario introduce los datos necesarios para crear una cuenta.

Curso normal			
1.	Usuario: Introduce su nombre, apellidos, correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el registro.	4.	El sistema verifica que el correo electrónico no esté registrado previamente.
5.	Usuario: Recibe un mensaje indicando que debe verificar su correo electrónico.		
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico ya está registrado.		
2c.	El sistema muestra un mensaje de error.		

Cuadro A.1: Caso de uso - Registro

Caso de Uso	iniciarSesion		CU2
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_02		
Precondición	El usuario debe estar registrado en el sistema.		
Postcondición	El usuario accede al sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de iniciar sesión en el sistema.
Resumen	El usuario introduce su correo electrónico y contraseña para autenticarse.

Curso normal			
1.	Usuario: Introduce su correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el inicio de sesión.	4.	El sistema verifica que el correo electrónico y la contraseña sean correctos.
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico o la contraseña son incorrectos.		
2c.	El correo electrónico no está verificado.		
2d.	El sistema muestra un mensaje de error.		

Cuadro A.2: Caso de uso - Iniciar Sesión

Caso de Uso	cerrarSesion		CU3
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_04		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario cierra su sesión en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de cerrar su sesión en el sistema.
Resumen	El usuario cierra su sesión para finalizar su uso del sistema.

Curso normal			
1.	Usuario: Accede a la opción de cerrar sesión.	2.	El sistema muestra un mensaje de confirmación.
3.	Usuario: Confirma que desea cerrar sesión.	4.	El sistema cierra la sesión del usuario.

Cuadro A.3: Caso de uso - Cerrar Sesión

Caso de Uso	verPerfil		CU4
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_05		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario visualiza la información de su perfil.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la información de su perfil.
Resumen	El usuario accede a la sección de perfil para visualizar su información personal.

Curso normal			
1.	Usuario: Ingresa a su área personal.	2.	El sistema muestra la información del perfil del usuario.

Cuadro A.4: Caso de uso - Ver Usuario

Caso de Uso	editarUsuario		CU5
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_06		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario actualiza su información en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de editar la información de su perfil.
Resumen	El usuario accede a la sección de perfil y modifica su información personal.

Curso normal			
1.	Usuario: Ingresa a su área personal.	2.	El sistema muestra la información del perfil del usuario.
3.	Usuario: Modifica la información deseada.	4.	El sistema verifica que la información introducida sea válida.
5.	Usuario: Guarda los cambios.	6.	El sistema actualiza la información del usuario.
Cursos alternos			
4a.	La información introducida no es válida.		
4b.	El sistema muestra un mensaje de error.		

Cuadro A.5: Caso de uso - Editar Usuario

A.2. Casos de uso de la gestión de hogar

Caso de Uso	crearHogar		CU6
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_07		
Precondición	El usuario no debe pertenecer a un hogar.		
Postcondición	El usuario crea un nuevo hogar en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crear un nuevo hogar en el sistema.
Resumen	El usuario introduce los datos necesarios para crear un hogar.

Curso normal			
1.	Usuario: Introduce el nombre del hogar.	2.	El sistema crea el hogar.

Cuadro A.6: Caso de uso - Crear Hogar

Caso de Uso	unirMiembroHogar		CU7
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_08		
Precondición	El usuario no debe pertenecer a un hogar.		
Postcondición	El usuario se une a un hogar existente en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de unirse a un hogar existente en el sistema.
Resumen	El usuario introduce un código de invitación para unirse a un hogar.

Curso normal			
1.	Usuario: Introduce el código del hogar.	2.	El sistema verifica que el código sea válido.
		3.	El sistema añade al usuario al hogar.
Cursos alternos			
2a.	El código no es válido.		
2b.	El sistema muestra un mensaje de error.		

Cuadro A.7: Caso de uso - Unir Miembro a Hogar

Caso de Uso	abandonarHogar		CU8
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_09		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario abandona el hogar en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de abandonar un hogar en el sistema.
Resumen	El usuario selecciona la opción para abandonar el hogar.

Curso normal			
1.	Usuario: Selecciona la opción de abandonar hogar.	2.	El sistema muestra un mensaje de confirmación.
3.	Usuario: Confirma que desea abandonar el hogar.	4.	El sistema elimina al usuario del hogar.

Cuadro A.8: Caso de uso - Abandonar Hogar

Caso de Uso	modificarHogar		CU9
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_11		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario modifica la información del hogar en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de modificar la información de un hogar en el sistema.
Resumen	El usuario accede a la sección de hogar y modifica la información del hogar.

Curso normal			
1.	Usuario: Ingresa al área personal.	2.	El sistema muestra la información del hogar.
3.	Usuario: Modifica la información deseada.	4.	El sistema verifica que la información introducida sea válida.
		5..	El sistema actualiza la información del hogar.
Cursos alternos			
4a.	La información introducida no es válida.		
4b.	El sistema muestra un mensaje de error.		

Cuadro A.9: Caso de uso - Modificar Hogar

Caso de Uso	verHogar		CU10
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_10		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario visualiza la información del hogar en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la información de un hogar en el sistema.
Resumen	El usuario accede a la sección de hogar para visualizar la información del hogar.

Curso normal			
1.	Usuario: Ingresa al área personal.	2.	El sistema muestra la información del hogar.

Cuadro A.10: Caso de uso - Ver Hogar

A.3. Casos de uso de la gestión de tareas

Caso de Uso	crearTarea		CU11
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_12		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario crea una nueva tarea en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crear una nueva tarea en el sistema.
Resumen	El usuario introduce los datos necesarios para crear una tarea.

Curso normal			
1.	Usuario: Introduce el nombre y la descripción de la tarea y el tipo de tarea.	2.	El sistema verifica que los datos sean válidos y guarda la tarea.

Cuadro A.11: Caso de uso - Crear Tarea

Caso de Uso	consultarPlanificaciónSemanal		CU12
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_13		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario visualiza la planificación semanal de tareas asociadas a él en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de consultar la planificación semanal de tareas en el sistema.
Resumen	El usuario accede a la sección de planificación semanal para visualizar las tareas programadas.

Curso normal			
1.	Usuario: Consulta la planificación semanal.	2.	El sistema muestra la planificación semanal de tareas.

Cuadro A.12: Caso de uso - Consultar Planificación Semanal

Caso de Uso	modificarTarea		CU13
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_15		
Precondición	El usuario debe pertenecer a un hogar y tener una tarea asignada.		
Postcondición	El usuario modifica la información de una tarea en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de modificar una tarea existente en el sistema.
Resumen	El usuario accede a la sección de tareas y modifica la información de una tarea existente.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar tareas.	2.	El sistema muestra la lista de tareas.
3.	Usuario: Selecciona la tarea a modificar.	4.	El sistema muestra la información de la tarea seleccionada.
5.	Usuario: Modifica la información deseada.	6.	El sistema verifica que la información introducida sea válida y actualiza la tarea.
Cursos alternos			
6a.	La información introducida no es válida.		
6b.	El sistema muestra un mensaje de error.		

Cuadro A.13: Caso de uso - Modificar Tarea

Caso de Uso	marcarTareaRealizada		CU14
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_14		
Precondición	El usuario debe pertenecer a un hogar y tener una tarea asignada.		
Postcondición	El usuario marca una tarea como realizada en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de marcar una tarea como realizada en el sistema.
Resumen	El usuario accede a la sección de tareas y marca una tarea como realizada.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar tareas.	2.	El sistema muestra la lista de tareas.
3.	Usuario: Selecciona la tarea a marcar como realizada.	4.	El sistema actualiza el estado de la tarea a realizada.

Cuadro A.14: Caso de uso - Marcar Tarea Realizada

Caso de Uso	eliminarTarea		CU15
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_16		
Precondición	El usuario debe pertenecer a un hogar y tener una tarea asignada.		
Postcondición	El usuario elimina una tarea en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de eliminar una tarea en el sistema.
Resumen	El usuario accede a la sección de tareas y elimina una tarea existente.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar tareas.	2.	El sistema muestra la lista de tareas.
3.	Usuario: Selecciona la tarea a eliminar.	4.	El sistema muestra un mensaje de confirmación.
5.	Usuario: Confirma la eliminación de la tarea.	6.	El sistema elimina la tarea.

Cuadro A.15: Caso de uso - Eliminar Tarea

Caso de Uso	verTarea		CU16
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_17		
Precondición	El usuario debe pertenecer a un hogar y tener una tarea asignada.		
Postcondición	El usuario visualiza la información de una tarea en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la información de una tarea en el sistema.
Resumen	El usuario accede a la sección de tareas para visualizar la información de una tarea.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar tareas.	2.	El sistema muestra la lista de tareas.
3.	Usuario: Selecciona la tarea a visualizar.	4.	El sistema muestra la información de la tarea seleccionada.

Cuadro A.16: Caso de uso - Ver Tarea

Caso de Uso	verCalendario		CU17
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_18		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario visualiza el calendario de tareas en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver el calendario de tareas en el sistema.
Resumen	El usuario accede a la sección de calendario para visualizar las tareas programadas.

Curso normal			
1.	Usuario: Ingresa al área destinada para el calendario.	2.	El sistema muestra el calendario de tareas.

Cuadro A.17: Caso de uso - Ver Calendario

A.4. Casos de uso de la gestión de gastos

Caso de Uso	crearGasto		CU18
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_19		
Precondición	El usuario debe pertenecer a un hogar.		
Postcondición	El usuario crea un nuevo gasto en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crear un nuevo gasto en el sistema.
Resumen	El usuario introduce los datos necesarios para crear un gasto.

Curso normal			
1.	Usuario: Introduce el nombre y la cantidad del gasto.		
2.	Usuario: Selecciona quien pagó el gasto y quien debe pagar.	3.	El sistema guarda el gasto.

Cuadro A.18: Caso de uso - Crear Gasto

Caso de Uso	modificarGasto		CU19
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_20		
Precondición	El usuario debe pertenecer a un hogar y tener un gasto registrado.		
Postcondición	El usuario modifica la información de un gasto en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de modificar un gasto existente en el sistema.
Resumen	El usuario accede a la sección de gastos y modifica la información de un gasto existente.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar gastos.	2.	El sistema muestra la lista de gastos.
3.	Usuario: Selecciona el gasto a modificar.	4.	El sistema muestra la información del gasto seleccionado.
5.	Usuario: Modifica la información deseada.	6.	El sistema verifica que la información introducida sea válida y actualiza el gasto.
Cursos alternos			
6a.	La información introducida no es válida.		
6b.	El sistema muestra un mensaje de error.		

Cuadro A.19: Caso de uso - Modificar Gasto

Caso de Uso	eliminarGasto		CU20
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_21		
Precondición	El usuario debe pertenecer a un hogar y tener un gasto registrado.		
Postcondición	El usuario elimina un gasto en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de eliminar un gasto en el sistema.
Resumen	El usuario accede a la sección de gastos y elimina un gasto existente.

Curso normal			
1.	Usuario: Accede a la sección de gastos.	2.	El sistema muestra la lista de gastos.
3.	Usuario: Selecciona el gasto a eliminar.	4.	El sistema muestra un mensaje de confirmación.
5.	Usuario: Confirma la eliminación del gasto.	6.	El sistema elimina el gasto.

Cuadro A.20: Caso de uso - Eliminar Gasto

Caso de Uso	saldarDeuda		CU21
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_22		
Precondición	El usuario debe pertenecer a un hogar y tener una deuda registrada.		
Postcondición	El usuario salda una deuda en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de saldar una deuda en el sistema.
Resumen	El usuario accede a la sección de deudas y salda una deuda existente.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar deudas.	2.	El sistema muestra la lista de deudas.
3.	Usuario: Selecciona la deuda a saldar.	4.	El sistema actualiza el estado de la deuda a saldada.

Cuadro A.21: Caso de uso - Saldar Deuda

Caso de Uso	verDeudas		CU22
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_23		
Precondición	El usuario debe pertenecer a un hogar y tener deudas registradas.		
Postcondición	El usuario visualiza la lista de deudas en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la lista de deudas en el sistema.
Resumen	El usuario accede a la sección de deudas para visualizar las deudas registradas.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar deudas.	2.	El sistema muestra la lista de deudas.

Cuadro A.22: Caso de uso - Ver Deudas

Caso de Uso	verGasto		CU23
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_24		
Precondición	El usuario debe pertenecer a un hogar y tener un gasto registrado.		
Postcondición	El usuario visualiza la información de un gasto en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la información de un gasto en el sistema.
Resumen	El usuario accede a la sección de gastos para visualizar la información de un gasto.

Curso normal			
1.	Usuario: Ingresa al área destinada para gestionar gastos.	2.	El sistema muestra la lista de gastos.
3.	Usuario: Selecciona el gasto a visualizar.	4.	El sistema muestra la información del gasto seleccionado.

Cuadro A.23: Caso de uso - Ver Gasto

A.5. Casos de uso de la gestión compra

Caso de Uso	crearLista		CU24
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_25		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario crea una nueva lista de compra en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crear una nueva lista de compra en el sistema.
Resumen	El usuario introduce los datos necesarios para crear una lista de compra.

Curso normal			
1.	Usuario: Introduce el nombre de la lista.	2.	El sistema guarda la lista.

Cuadro A.24: Caso de uso - Crear Lista

Caso de Uso	añadirProducto		CU25
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_26		
Precondición	El usuario debe tener una lista de compra creada.		
Postcondición	El usuario añade un producto a la lista de compra en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de añadir un producto a una lista de compra en el sistema.
Resumen	El usuario introduce los datos necesarios para añadir un producto a la lista de compra.

Curso normal			
1.	Usuario: Selecciona la lista de compra.	2.	Usuario: Introduce el nombre y la cantidad del producto.
3.	Usuario: Guarda el producto en la lista.	4.	El sistema actualiza la lista con el nuevo producto.

Cuadro A.25: Caso de uso - Añadir Producto

Caso de Uso	eliminarProducto		CU26
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_27		
Precondición	El usuario debe tener una lista de compra creada.		
Postcondición	El usuario elimina un producto de la lista de compra en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de eliminar un producto de una lista de compra en el sistema.
Resumen	El usuario selecciona el producto a eliminar de la lista de compra.

Curso normal			
1.	Usuario: Selecciona la lista de compra.	2.	Usuario: Selecciona el producto a eliminar.
3.	Usuario: Confirma la eliminación del producto.	4.	El sistema actualiza la lista eliminando el producto.

Cuadro A.26: Caso de uso - Eliminar Producto

Caso de Uso	eliminarLista		CU27
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_28		
Precondición	El usuario debe tener una lista de compra creada.		
Postcondición	El usuario elimina una lista de compra en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de eliminar una lista de compra en el sistema.
Resumen	El usuario selecciona la lista de compra a eliminar.

Curso normal			
1.	Usuario: Selecciona la lista de compra.	2.	Usuario: Confirma la eliminación de la lista.
3.	El sistema elimina la lista de compra.	4.	El sistema muestra un mensaje de confirmación.

Cuadro A.27: Caso de uso - Eliminar Lista

Caso de Uso	verProducto		CU28
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_29		
Precondición	El usuario debe tener una lista de compra creada.		
Postcondición	El usuario visualiza la información de un producto en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la información de un producto en el sistema.
Resumen	El usuario accede a la lista de compra para visualizar la información de un producto.

Curso normal			
1.	Usuario: Accede a la lista de compra.	2.	El sistema muestra la lista de productos.
3.	Usuario: Selecciona el producto a visualizar.	4.	El sistema muestra la información del producto seleccionado.

Cuadro A.28: Caso de uso - Ver Producto

Caso de Uso	editarProducto		CU29
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_30		
Precondición	El usuario debe tener una lista de compra creada.		
Postcondición	El usuario edita la información de un producto en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de editar la información de un producto en el sistema.
Resumen	El usuario accede a la lista de compra y modifica la información de un producto.

Curso normal			
1.	Usuario: Accede a la lista de compra.	2.	El sistema muestra la lista de productos.
3.	Usuario: Selecciona el producto a editar.	4.	El sistema muestra la información del producto seleccionado.
5.	Usuario: Modifica la información deseada.	6.	El sistema verifica que la información introducida sea válida y actualiza el producto.
Cursos alternos			
6a.	La información introducida no es válida.		
6b.	El sistema muestra un mensaje de error.		

Cuadro A.29: Caso de uso - Editar Producto

Referencias

Adverthia. (2019). *Usabilidad web: definición y características.* Descargado 2025-02-04, de https://adverthia.com/usabilidad-web-definicion-y-caracteristicas

American Time Use Survey. (2022). *Distributions of time for household management and shared living challenges.* Descargado de https://www.bls.gov/news.release/archives/atus_06222023.pdf

Apple Inc. (2025). *Swift programming language.* Descargado de https://www.swift.org/

Atlassian. (2025). *Guía de procesos iterativos.* Descargado de https://www.atlassian.com/es/work-management/project-management/iterative-process

Beck, K. (1999). *Extreme programming explained: Embrace change.* Addison-Wesley.

Boehm, B. (1988). A spiral model of software development and enhancement. *ACM SIGSOFT Software Engineering Notes*, *11*(4), 14-24.

Burriel, T. (2024). *Accesibilidad en aplicaciones móviles.* Descargado 2025-02-04, de https://torresburriel.com/weblog/accesibilidad-en-aplicaciones-moviles

Carpena, M. C. (2023). *9 buenas prácticas de accesibilidad web que puede imitar.* Descargado de https://www.seo.com/es/basics/technical/accessibility/

Chisholm, W., y Vanderheiden, G. (1999). *Guía de accesibilidad para el desarrollo de contenidos web.* Descargado de http://www.w3.org/TR/1999/WAI-WEBCONTENT-19990505/

Cozi Inc. (2025). *Cozi family organizer.* Descargado de https://www.cozi.com/

FC Research. (2024). *Pisos compartidos: Tendencias y perspectivas.* Descargado de https://s36360.pcdn.co/wp-content/uploads/2024/09/NdP-Pisos-compartidos-I-FC-Research.pdf

Foundation, D. S. (2025). *Django - the web framework for perfectionists with deadlines.* Descargado de https://www.djangoproject.com/

Google. (2025a). *Firebase.* Descargado de https://firebase.google.com/

Google. (2025b). *Flutter - build apps for any screen.* Descargado de https://flutter.dev/

Huffington Post. (2025). *El 87 % de los jóvenes emancipados comparte vivienda en alquiler al no poder hacer frente a los gastos mensuales.* Descargado de https://www.huffingtonpost.es/politica/el-87-jovenes-emancipados-comparte-vivienda-alquiler-frente-gastos-mensuales.html

Ionic Framework. (2025). *Ionic framework - the cross-platform app development leader.* Descargado de https://ionicframework.com/

Jefatura del Estado. (2018). *Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales.* Descargado de https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673

JetBrains. (2025). *Kotlin programming language.* Descargado de https://kotlinlang.org/

Kim, G., Humble, J., Debois, P., y Willis, J. (2016). *The devops handbook.* IT Revolution.

Larman, C. (2003). *Agile and iterative development: A manager's guide.* Addison-Wesley.

Meta Platforms, I. (2025). *React native · learn once, write anywhere.* Descargado de https://reactnative.dev/

Microsoft. (2025). *Xamarin.* Descargado de https://dotnet.microsoft.com/apps/xamarin

Ministerio de Igualdad. (2023). *Igualdad presenta 'metoca', la aplicación para repartir de forma corresponsable las tareas domésticas y de cuidados.* Descargado de https://www.igualdad.gob.es/comunicacion/notasprensa/igualdad-presenta-metoca-la-aplicacion-para-repartir-de-forma-corresponsable-las-tareas-domesticas-y-de-cuidados/

Mozilla Developer Network. (2024). *¿qué es la accesibilidad?* Descargado 2025-02-04, de https://developer.mozilla.org/es/docs/Learn_web_development/Core/Accessibility/What_is_accessibility

Nielsen, J. (1993). *Usability engineering* (1st ed.). San Diego: Academic Press.

Node.js Foundation. (2025). *Node.js - javascript runtime.* Descargado de https://nodejs.org/

OurHome. (2025). *Ourhome - organiza tu hogar fácilmente.* Descargado de https://ourhomeapp.com/

Parlamento Europeo. (2016). *Reglamento (ue) 2016/679 del parlamento europeo y del consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos.* Descargado de https://eur-lex.europa.eu/legal-content/ES/TXT/PDF/?uri=CELEX%3A32016R0679

PMO Informática. (2017). *Requerimientos funcionales: definición y ejemplos.* Descargado 2025-02-04, de https://www.pmoinformatica.com/2017/02/requerimientos-funcionales-ejemplos.html

Ruby on Rails. (2025). *Ruby on rails - web application framework.* Descargado de https://rubyonrails.org/

Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.

Staff, C. (2023). *What is usability and why it matters.* Descargado 2025-02-04, de https://www.coursera.org/mx/articles/what-is-usability-and-why-it-matters

Talent.com. (2025). *Salario de programador junior en españa.* Descargado de https://es.talent.com/salary?job=programador+junior

ThoughtWorks. (2020). *Manual de accesibilidad para profesionales.* Descargado de https://www.thoughtworks.com/content/dam/thoughtworks/documents/e-book/tw_ebook_accessibility_handbook_v4_spanish.pdf

Todoist. (2025). *Todoist - organiza tu vida y trabajo.* Descargado de https://todoist.com/es

Tricount. (2025). *Tricount - organiza los gastos del grupo.* Descargado de https://tricount.com/es-es

U.S. Department of Housing and Urban Development. (2022). *Desafíos financieros en viviendas estudiantiles fuera del campus: Un análisis de la carga financiera de los arrendamientos anuales y la gestión de los gastos compartidos.* Descargado 2025-02-04, de https://www.huduser.gov/portal/periodicals/insight/insight_2.pdf